

Universidade Federal de São Carlos - UFSCar
Departamento de Computação - DC
CEP 13565-905, Rod. Washington Luiz, s/n, São Carlos, SP

Aula 08 – Avaliação de algoritmos de classificação

Prof. Dr. Alan Demétrius Baria Valejo

1001524 - Aprendizado de Máquina 1

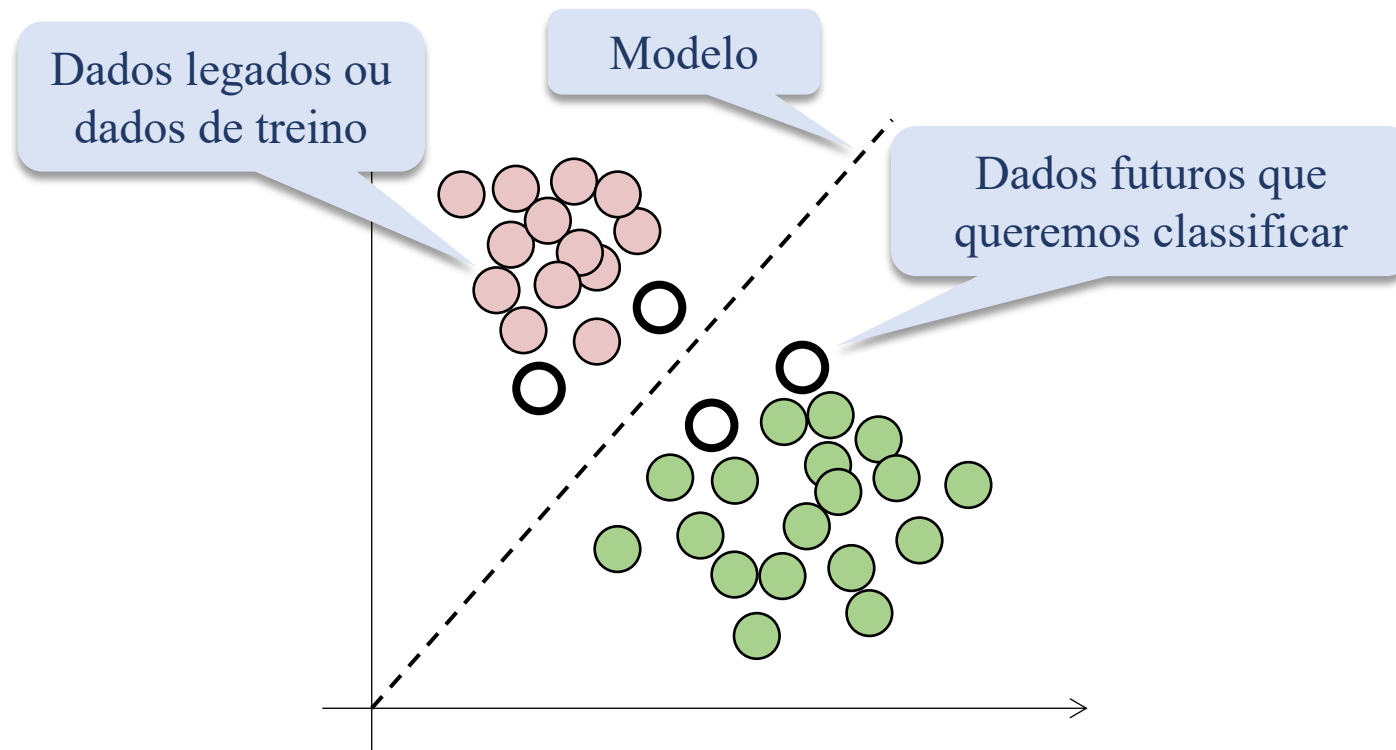
- Introdução
- Treinamento
- Matriz de confusão
- Acurácia e erro
- Precisão, Revocação e F1-Score
- Curva ROC-AUC (separado de avaliação pois é uma métrica mais complexa) ← não cai na prova
- Qual modelo subir em produção?
- Fluxograma
- Análise Visual ← não cai na prova
- Análise Estatística ← não cai na prova
- Outros cenários ← não cai na prova
- Análise paramétrica ← não cai na prova
- Próxima aula

Introdução

Introdução

Alguns conceitos básicos

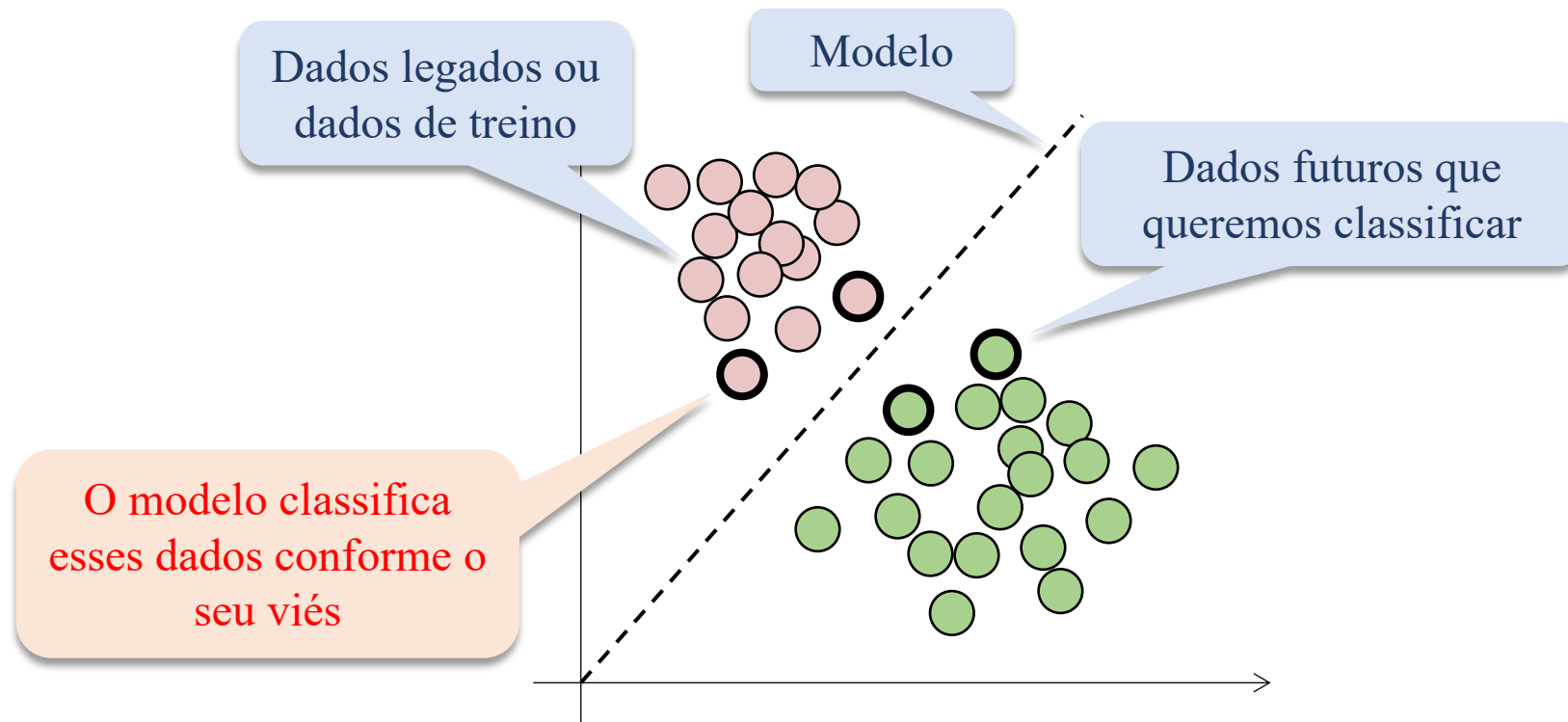
- Qual o cenário real?



Introdução

Alguns conceitos básicos

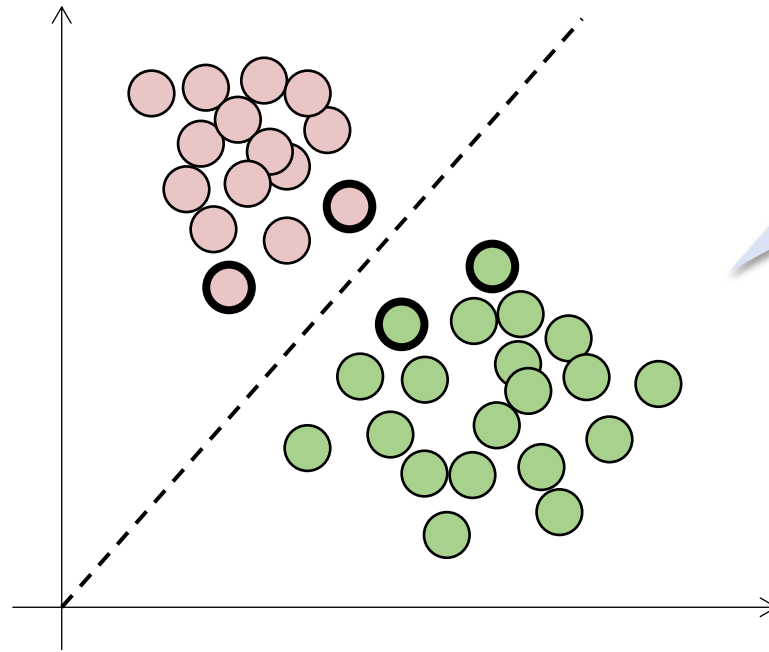
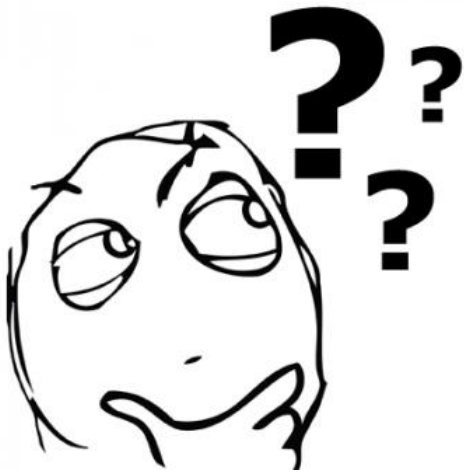
- Qual o cenário real?



Introdução

Alguns conceitos básicos

- Qual o cenário real?

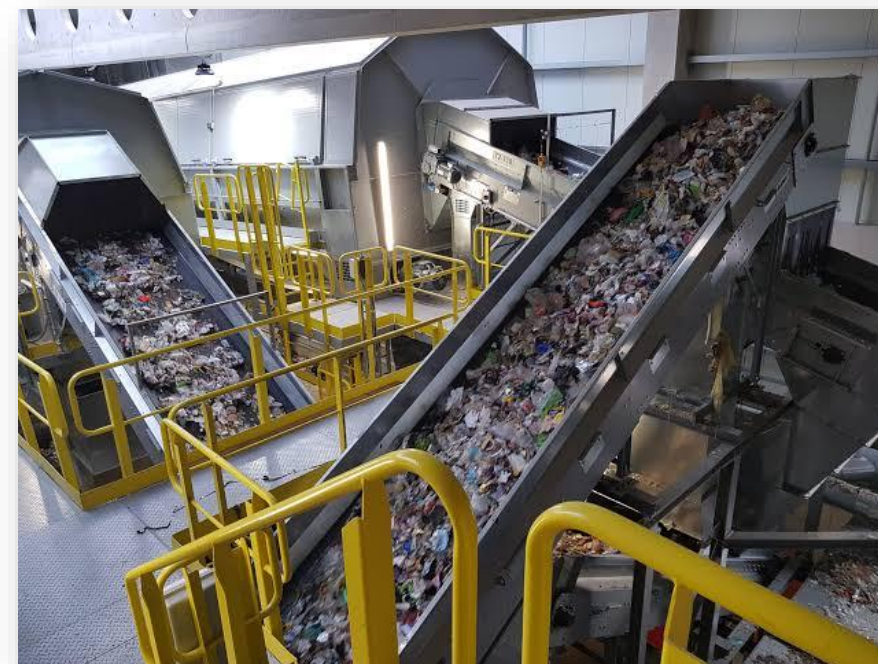


- Mas será que o algoritmo acertou esse resultado?

Introdução

Alguns conceitos básicos

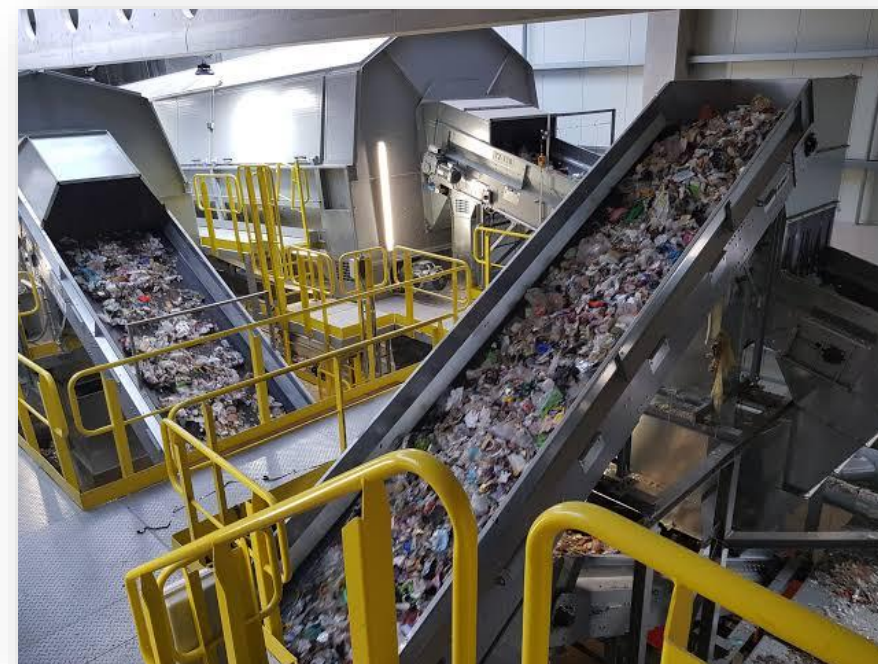
- Importante:
 - Depois de pronto, em muitas aplicações, não temos mais como saber se o modelo acertou ou errou quando avaliado em dados futuros;
 - Apenas confiamos;
- Imaginem uma máquina que separa lixo reciclado e não reciclado:
 - Diariamente, toneladas de lixo são separadas por essa máquina;
 - Se precisássemos conferir se a máquina acertou ou não, não faria sentido a sua utilização;
 - Claro que eventualmente, um lixo não reciclado passa batido ou ao contrário;
 - Porém, isso é tolerável.
 - O importante é saber antes de sua aplicação se o algoritmo costuma errar pouco ou muito.



Introdução

Alguns conceitos básicos

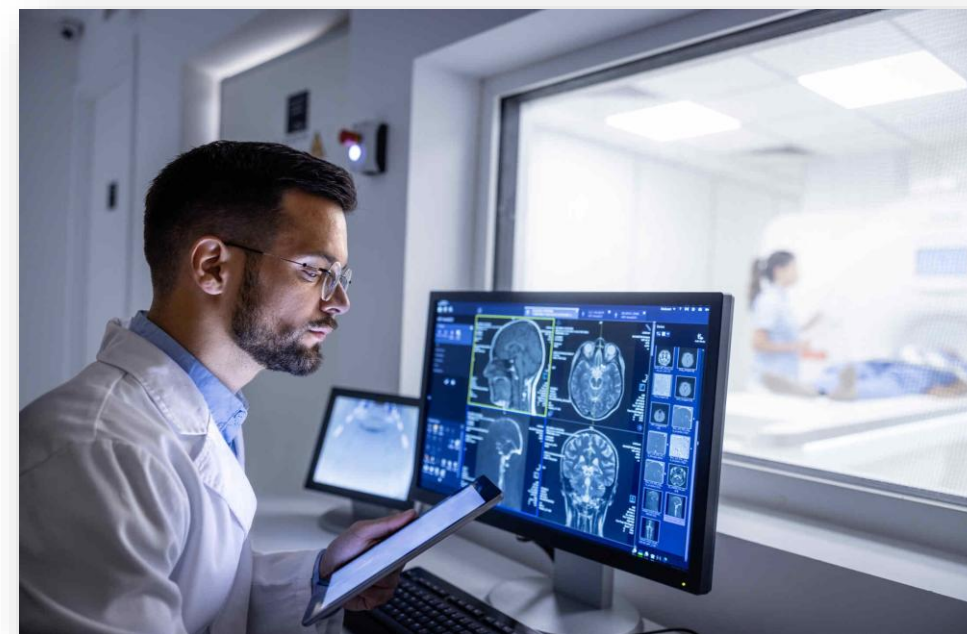
- Importante:
 - Depois de pronto, em muitas aplicações, não temos mais como saber se o modelo acertou ou errou quando avaliado em dados futuros;
 - Apenas confiamos;
- Imaginem uma máquina que separa lixo reciclado e não reciclado:
 - Diariamente, toneladas de lixo são separadas por essa máquina;
 - Se precisássemos conferir se a máquina acertou ou não, não faria sentido a sua utilização;
 - Claro que eventualmente, um lixo não reciclado passa batido ou ao contrário;
 - Porém, isso é tolerável.
 - O importante é saber antes de sua aplicação se o algoritmo costuma errar pouco ou muito.



Introdução

Alguns conceitos básicos

- Importante:
 - Depois de pronto, em muitas aplicações, não temos mais como saber se o modelo acertou ou errou quando avaliado em dados futuros;
 - Apenas confiamos;
- Logicamente, existem cenários críticos, como o **diagnóstico médico**:
 - **Nesse caso, cada diagnóstico é verificado;**
 - Porém, se o diagnóstico estiver separado em positivo e negativo é mais fácil verificar ou encaminhar o paciente para outros exames.



Introdução

Alguns conceitos básicos

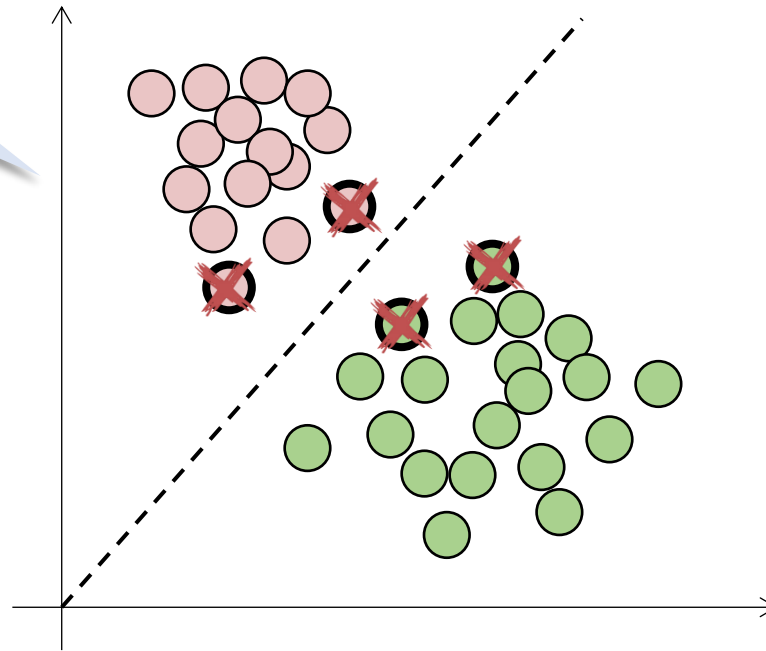
- Portanto:
 - Queremos saber, **antes que o modelo seja implementado em produção**, qual o grau de confiança em sua resposta ou o seu desempenho na aplicação;
- Antes do modelo ser aplicado em dados futuros, ele será avaliado.



Introdução

Alguns conceitos básicos

- Essa avaliação é feita no conjunto de dados legados e não em produção ou nos dados futuros.

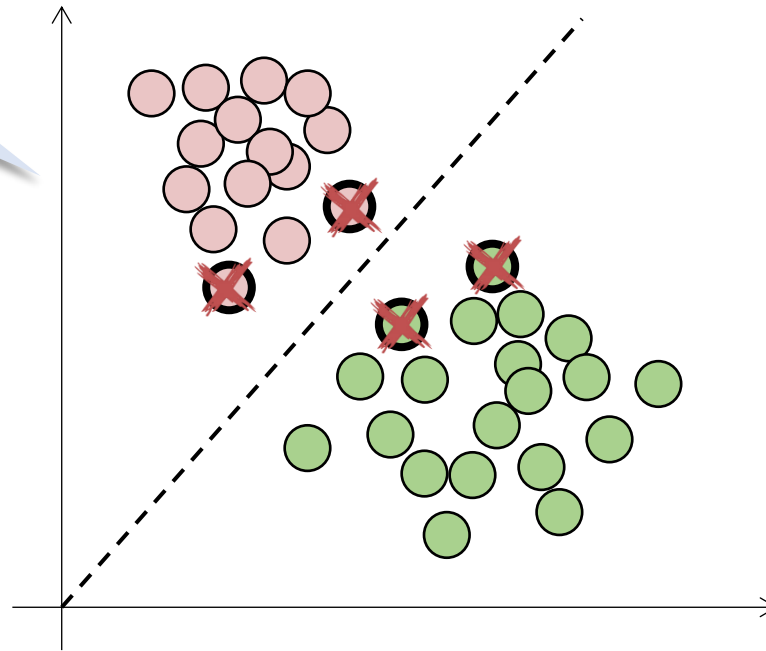


Introdução

Alguns conceitos básicos

- Essa avaliação é feita no conjunto de dados legados e não em produção ou nos dados futuros.

- Porque?

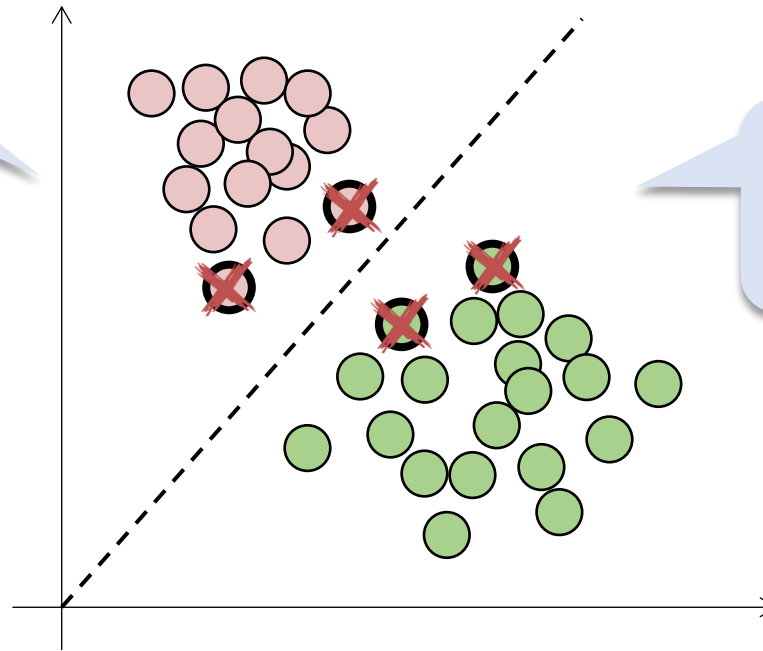


Introdução

Alguns conceitos básicos

- Essa avaliação é feita no conjunto de dados legados e não em produção ou nos dados futuros.

- Porque?



- Simples, eu conheço os meus dados legados ou eu tenho um supervisor na empresa.

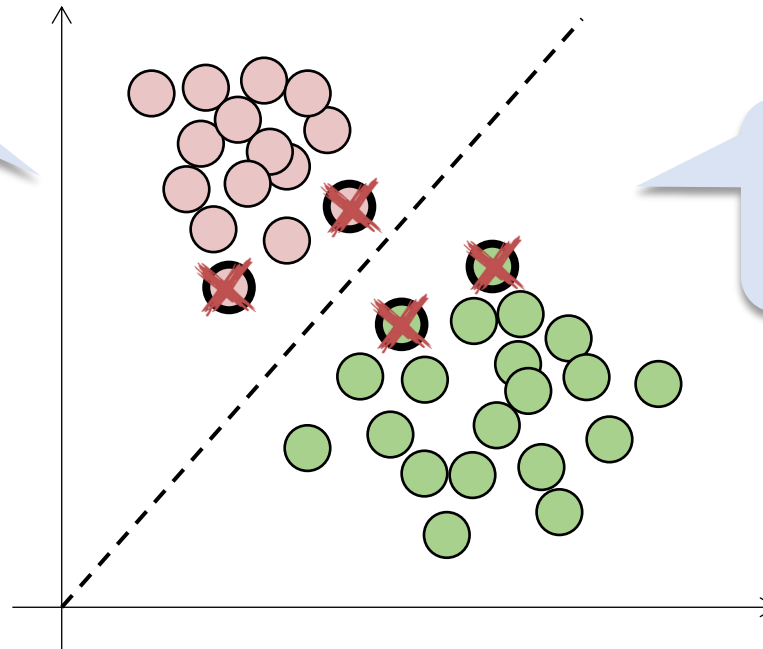
Introdução

Alguns conceitos básicos

- Essa avaliação é feita no conjunto de dados legados e não em produção ou nos dados futuros.

- E como fazemos isso?

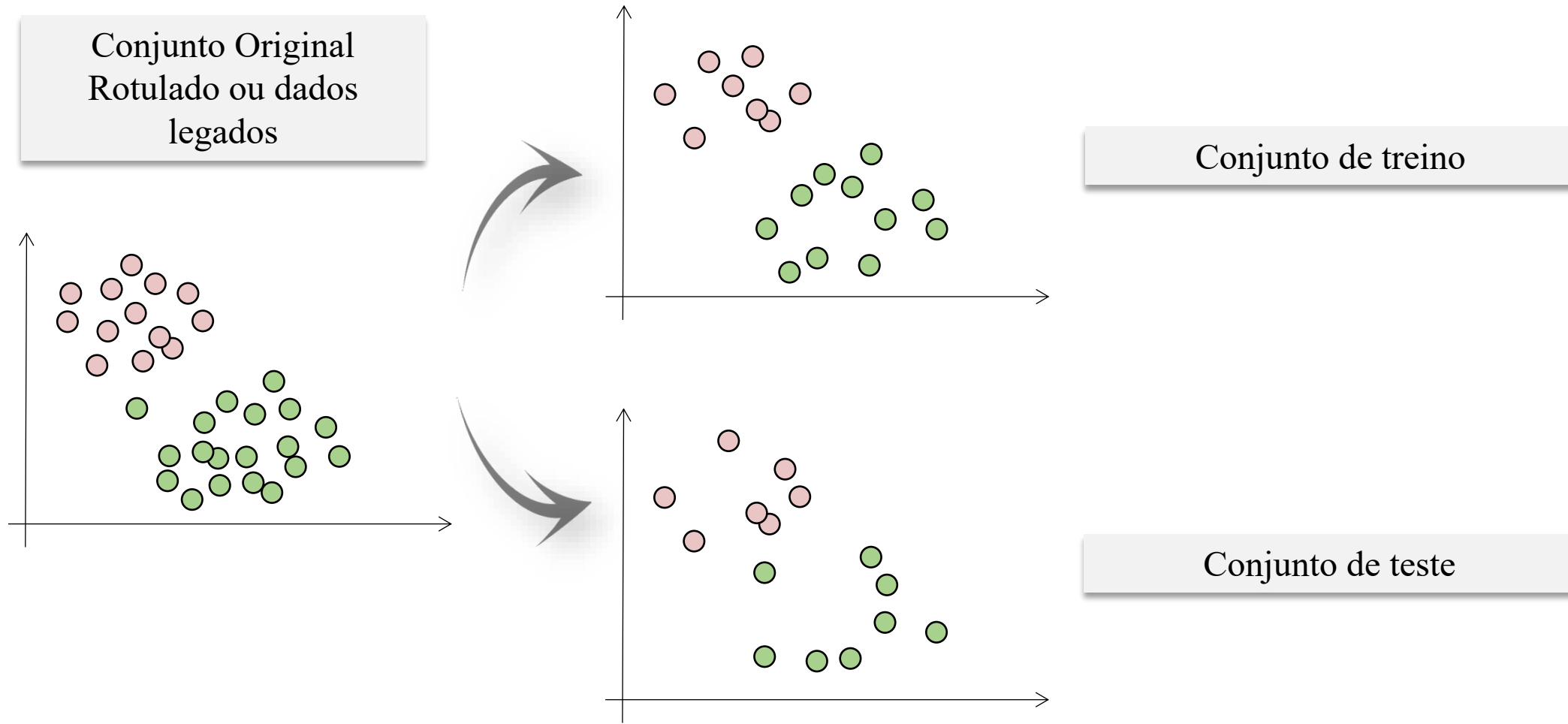
- Porque?



- Simples, eu conheço os meus dados legados ou eu tenho um supervisor na empresa.

Introdução

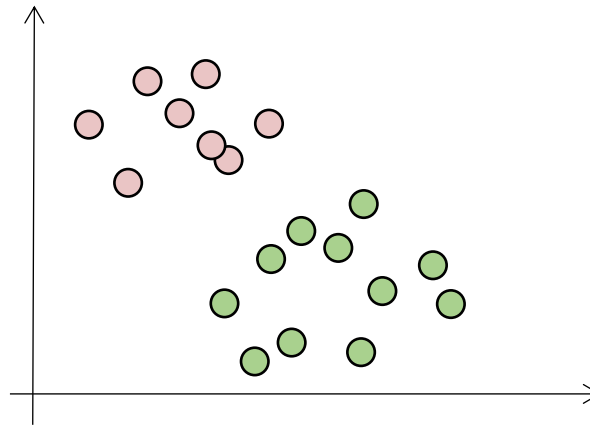
Alguns conceitos básicos



Introdução

Alguns conceitos básicos

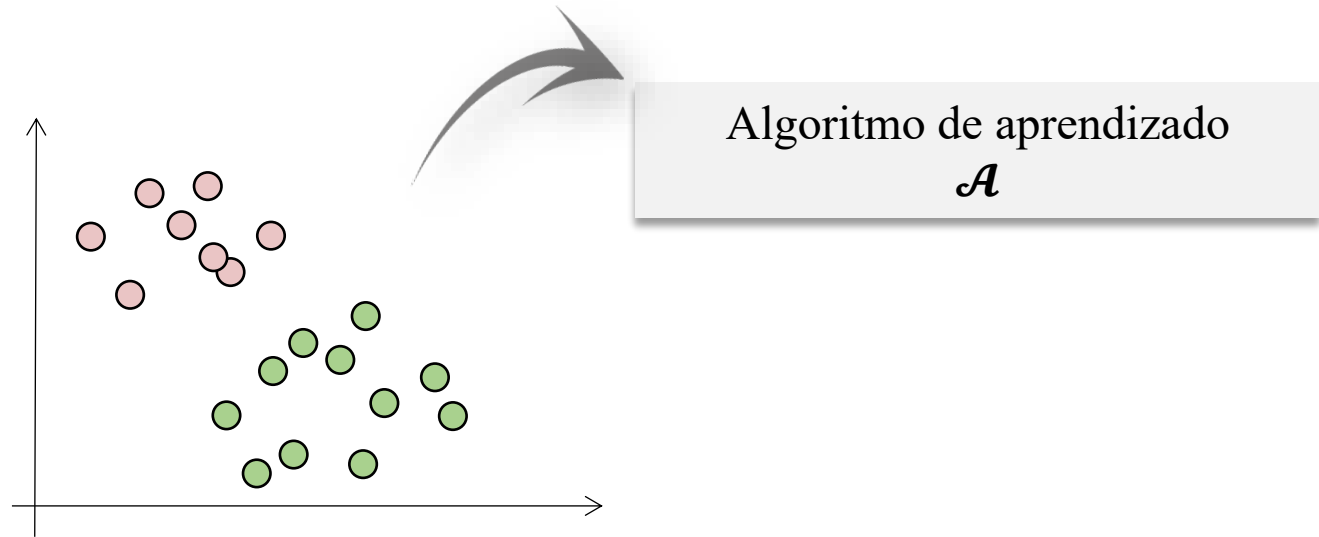
- O conjunto de treino é utilizado para treinar o modelo.



Introdução

Alguns conceitos básicos

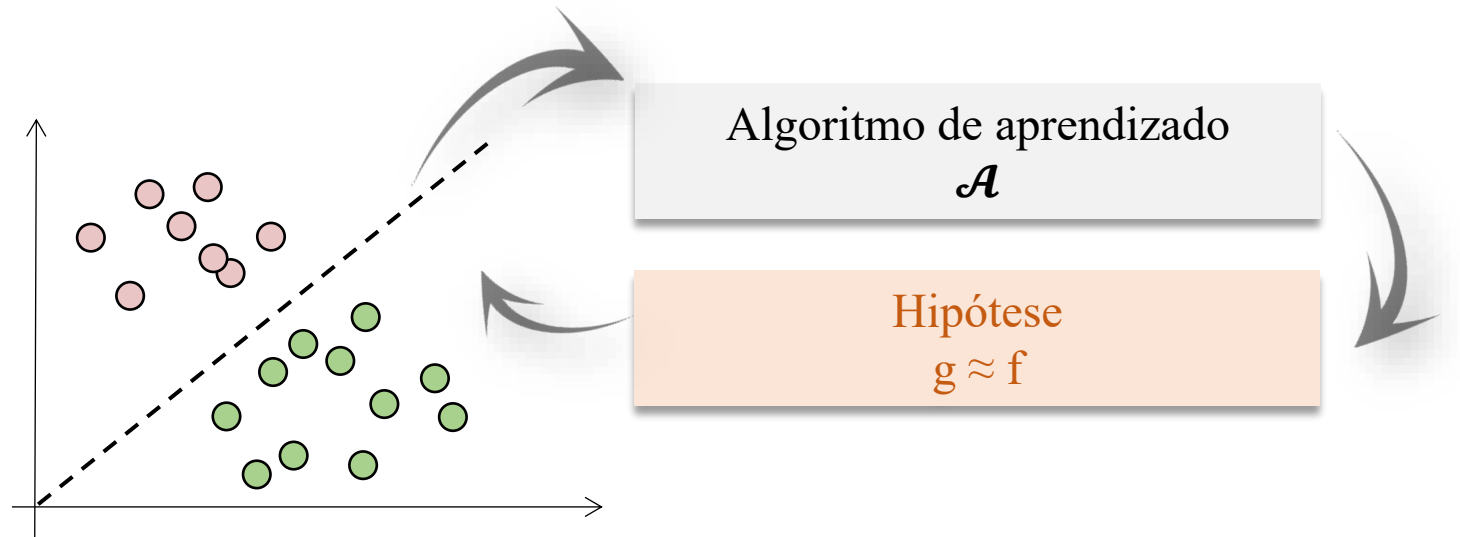
- O conjunto de treino é utilizado para treinar o modelo.



Introdução

Alguns conceitos básicos

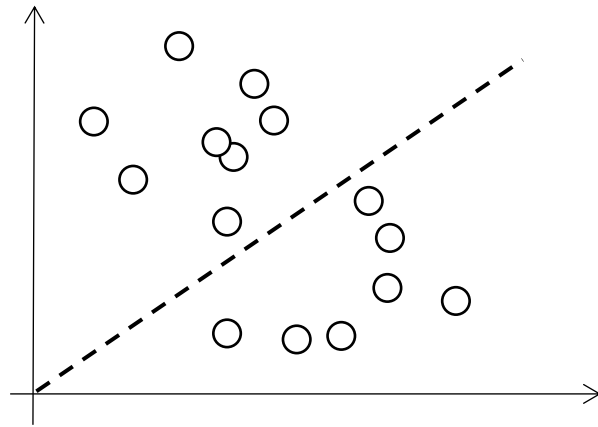
- O conjunto de treino é utilizado para treinar o modelo.



Introdução

Alguns conceitos básicos

- O conjunto de teste é utilizado para avaliar o modelo treinado.

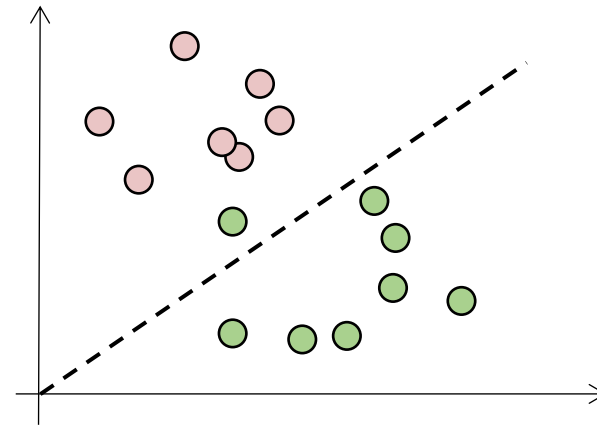
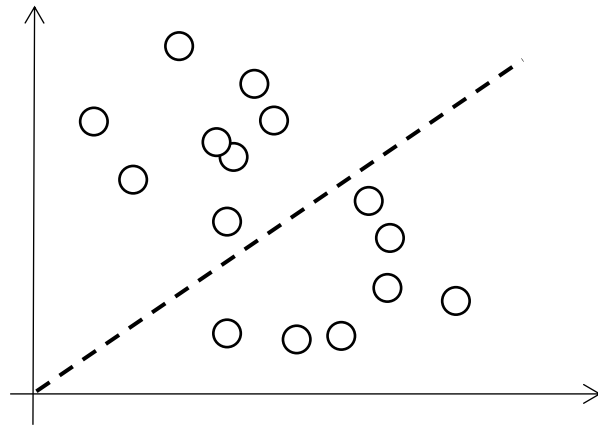


Introdução

Alguns conceitos básicos

- O conjunto de teste é utilizado para avaliar o modelo treinado.

Qual a grande sacada?

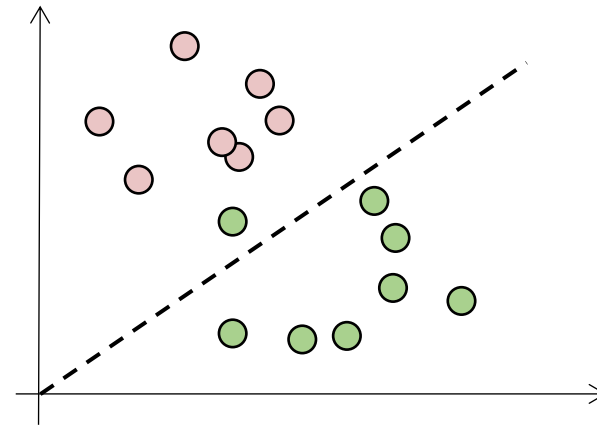
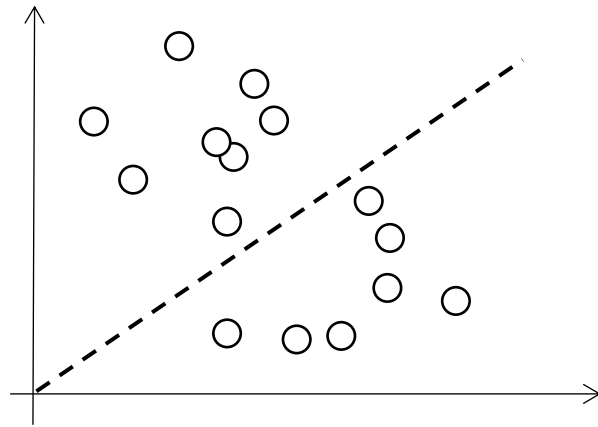


Introdução

Alguns conceitos básicos

- O conjunto de teste é utilizado para avaliar o modelo treinado.

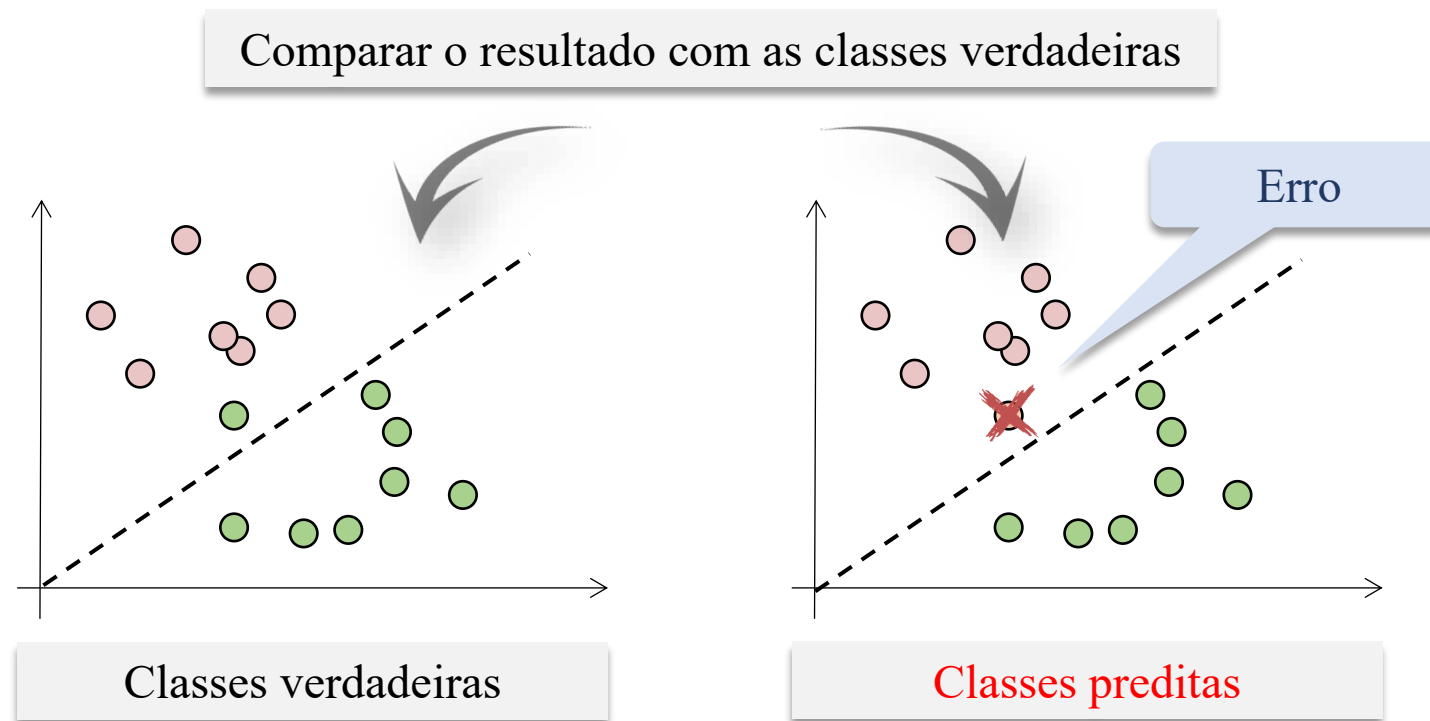
Como o conjunto de teste é oriundo dos meus dados legados eu conheço a resposta.



Introdução

Alguns conceitos básicos

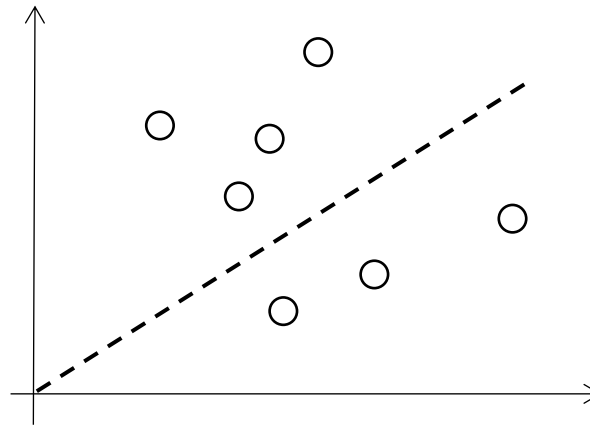
- O conjunto de teste é utilizado para avaliar o modelo treinado



Introdução

Alguns conceitos básicos

- Depois de avaliado e ajustado, o modelo pode ser colocado em produção.
- Ou seja, pode ser utilizado para classificar dados nunca antes vistos.

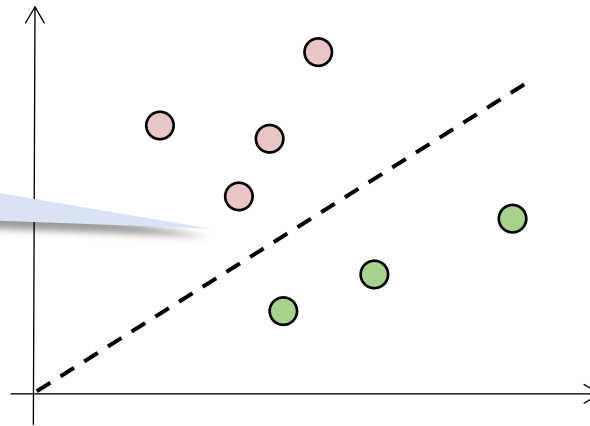


Introdução

Alguns conceitos básicos

- Depois de avaliado e ajustado, o modelo pode ser colocado em produção.
- Ou seja, pode ser utilizado para classificar dados nunca antes vistos.

Embora no conjunto de teste existiu 1 erro, em produção esse erro pode não ocorrer.

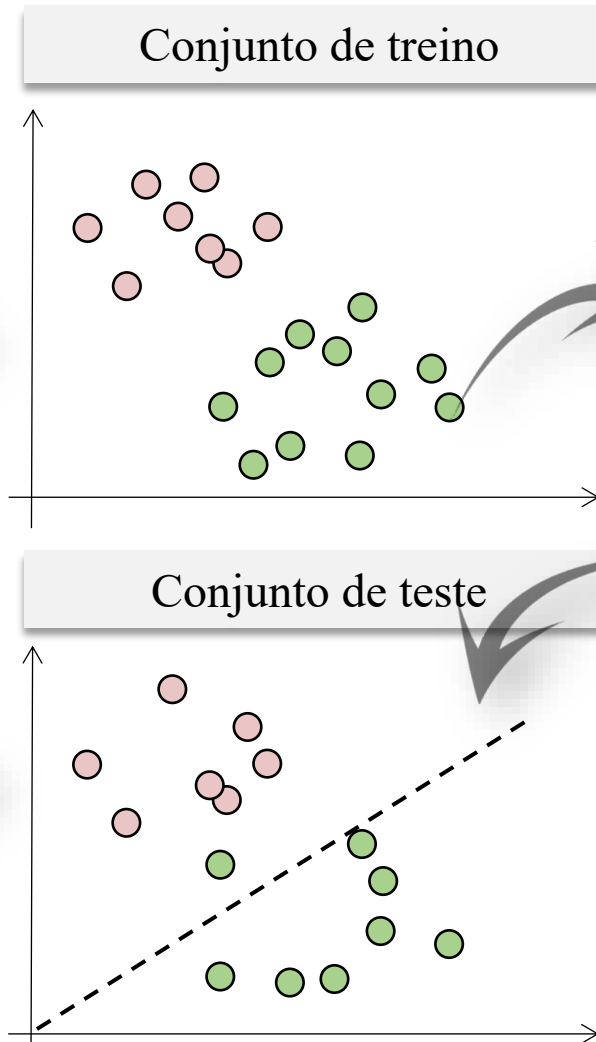
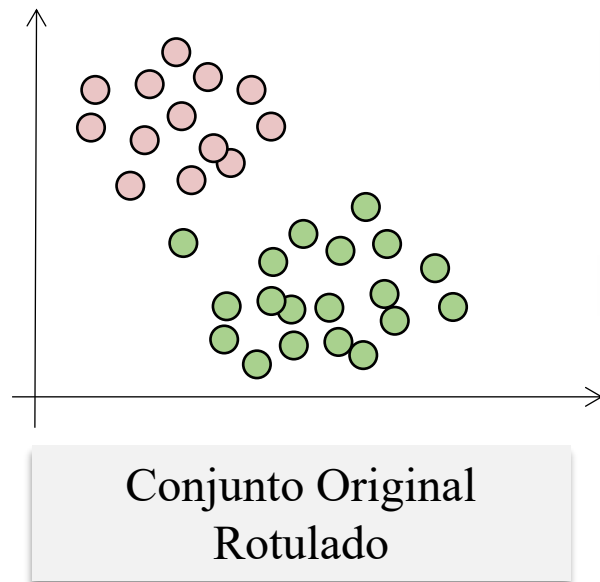


Treinamento

Conjunto de treino e teste

Como separar?

- O modelo não pode ver os dados de teste durante o treinamento.
- Caso isso ocorra, seria o equivalente a fazer uma prova sabendo de antemão quais são as questões e quais são as respostas.
- Ou seja, os testes devem ser feitos com dados que o modelo não viu anteriormente.



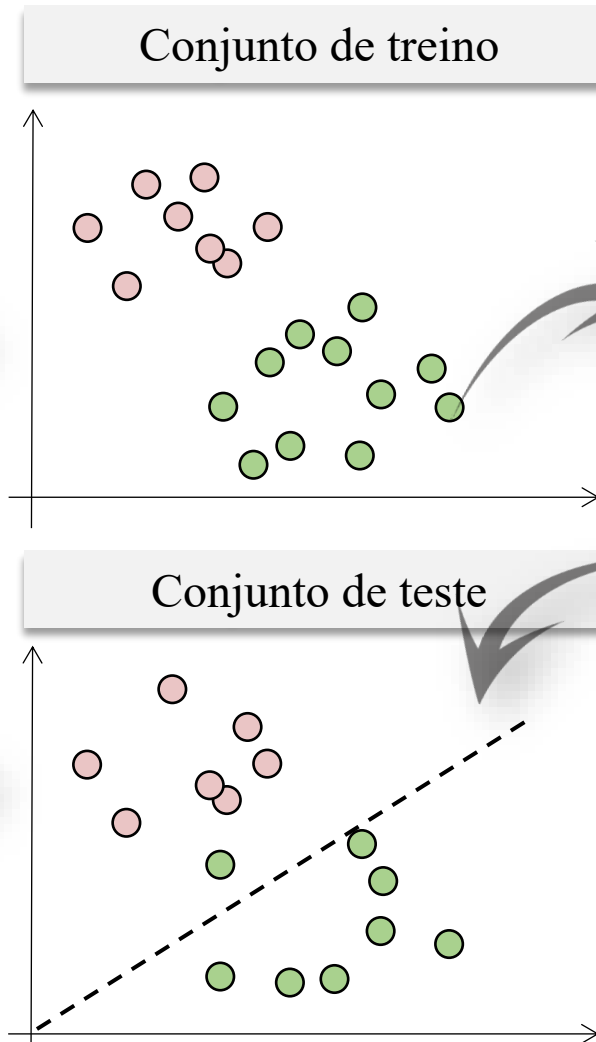
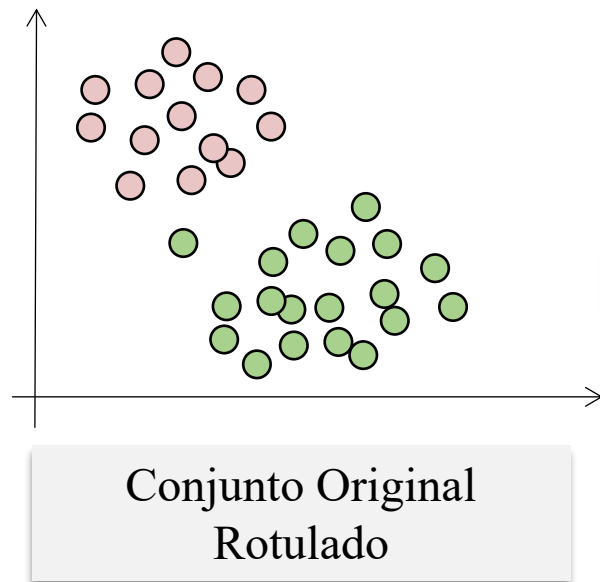
Algoritmo de aprendizado
 $\mathcal{A}$

Hipótese
 $g \approx f$

Conjunto de treino e teste

Como separar?

- O modelo não pode ver os dados de teste durante o treinamento.
- Caso isso ocorra, seria o equivalente a fazer uma prova sabendo de antemão quais são as questões e quais são as respostas.
- Ou seja, os testes devem ser feitos com dados que o modelo não viu anteriormente.



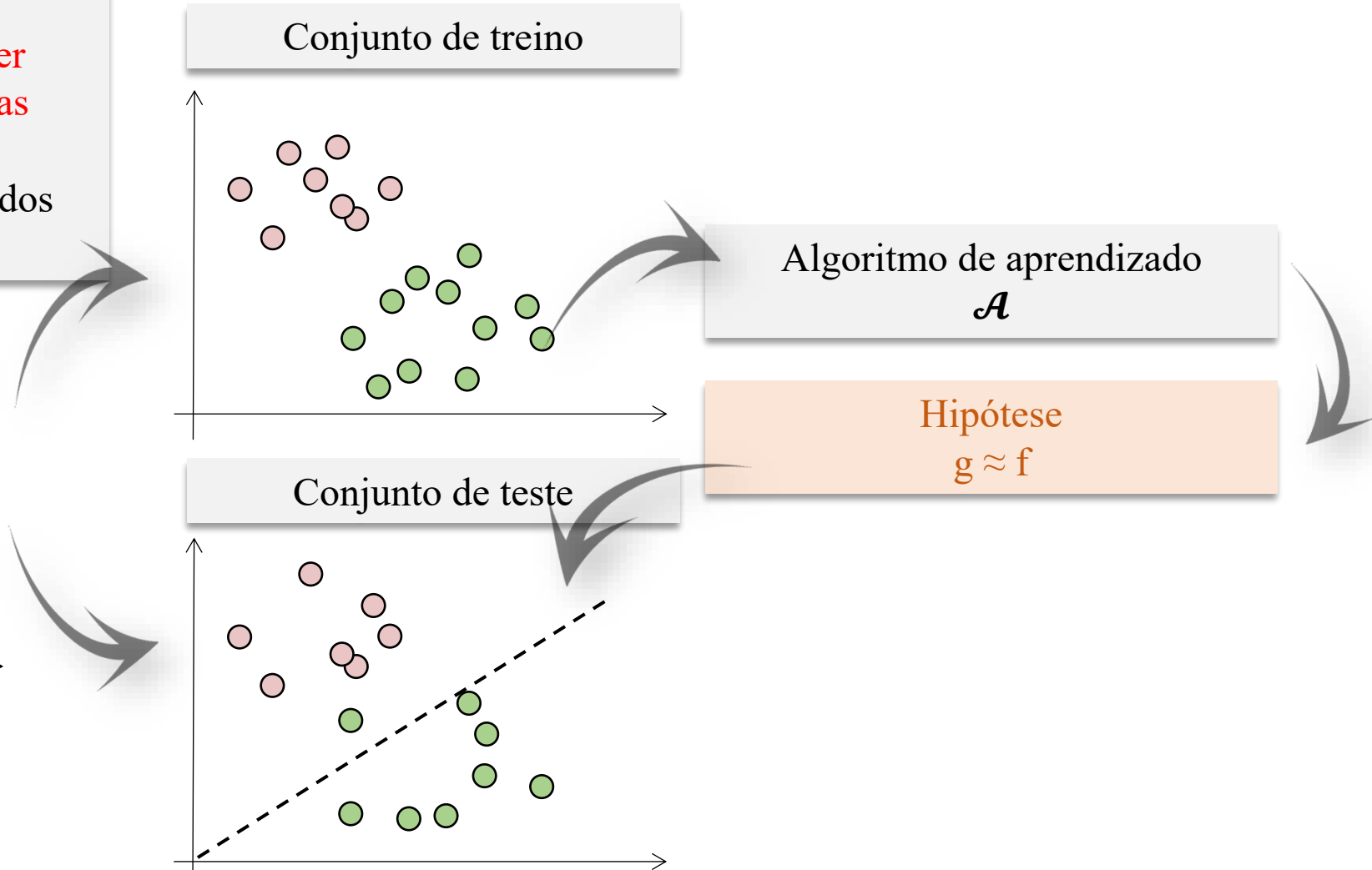
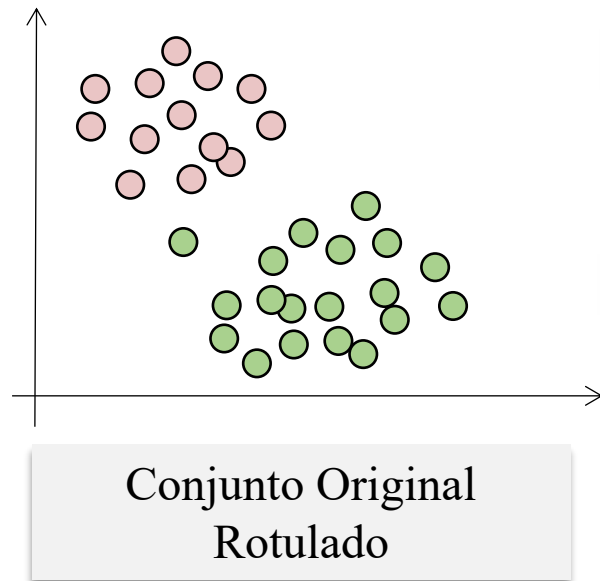
Algoritmo de aprendizado
 $\mathcal{A}$

Hipótese
 $g \approx f$

Conjunto de treino e teste

Como separar?

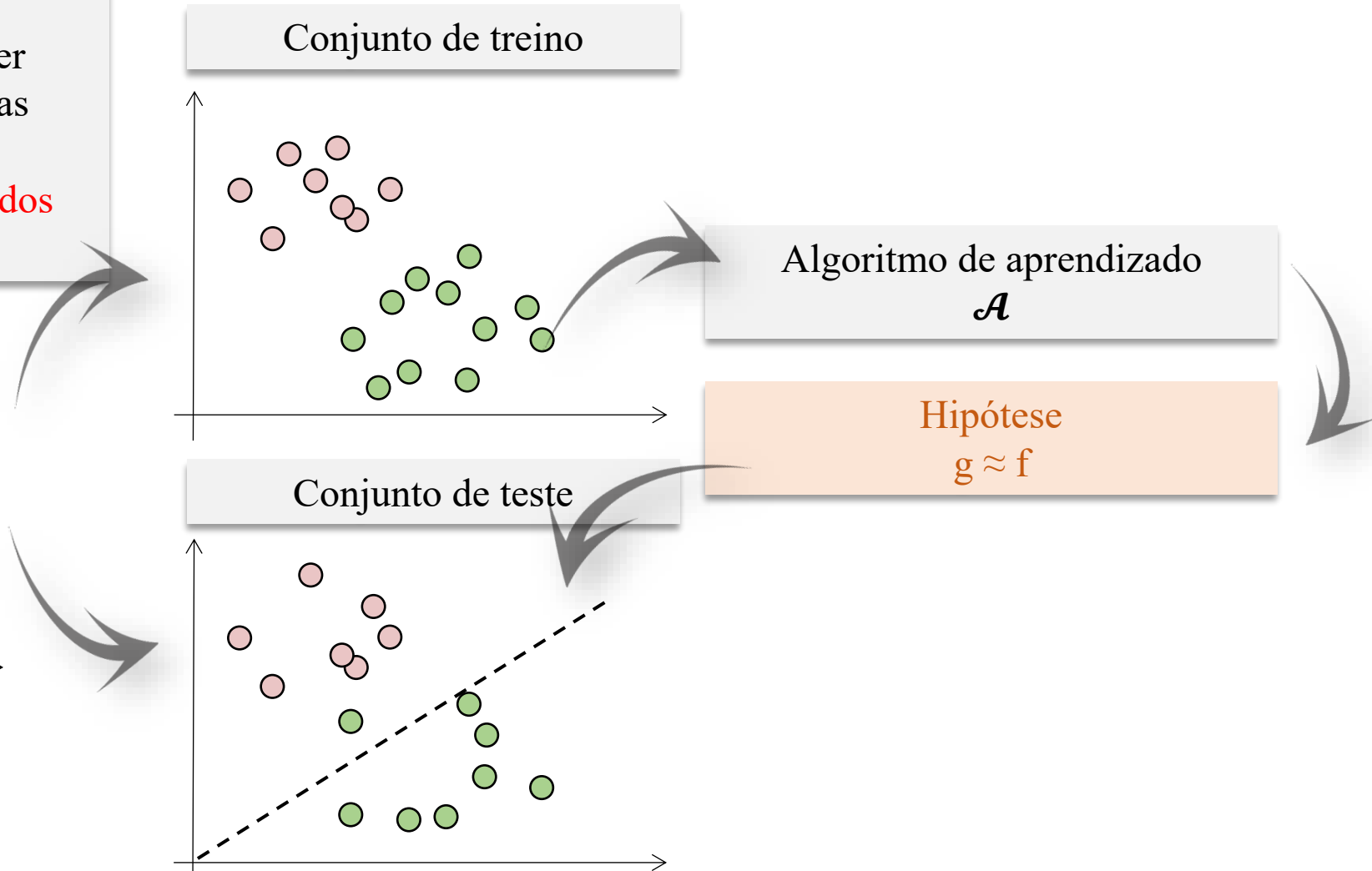
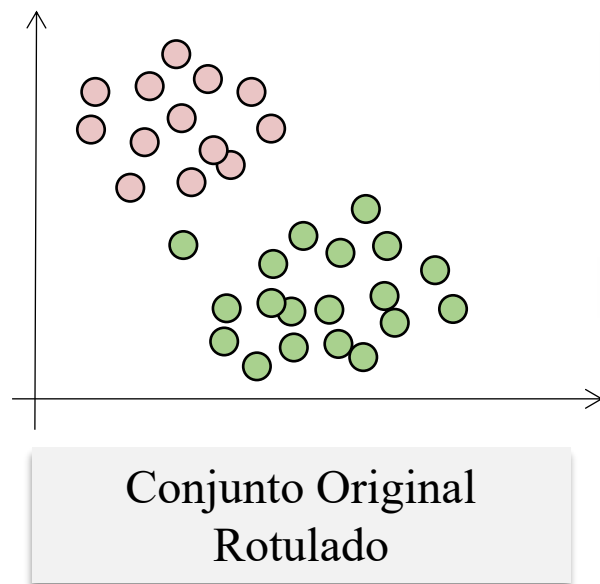
- O modelo não pode ver os dados de teste durante o treinamento.
- Caso isso ocorra, seria o equivalente a fazer uma prova sabendo de antemão quais são as questões e quais são as respostas.
- Ou seja, os testes devem ser feitos com dados que o modelo não viu anteriormente.



Conjunto de treino e teste

Como separar?

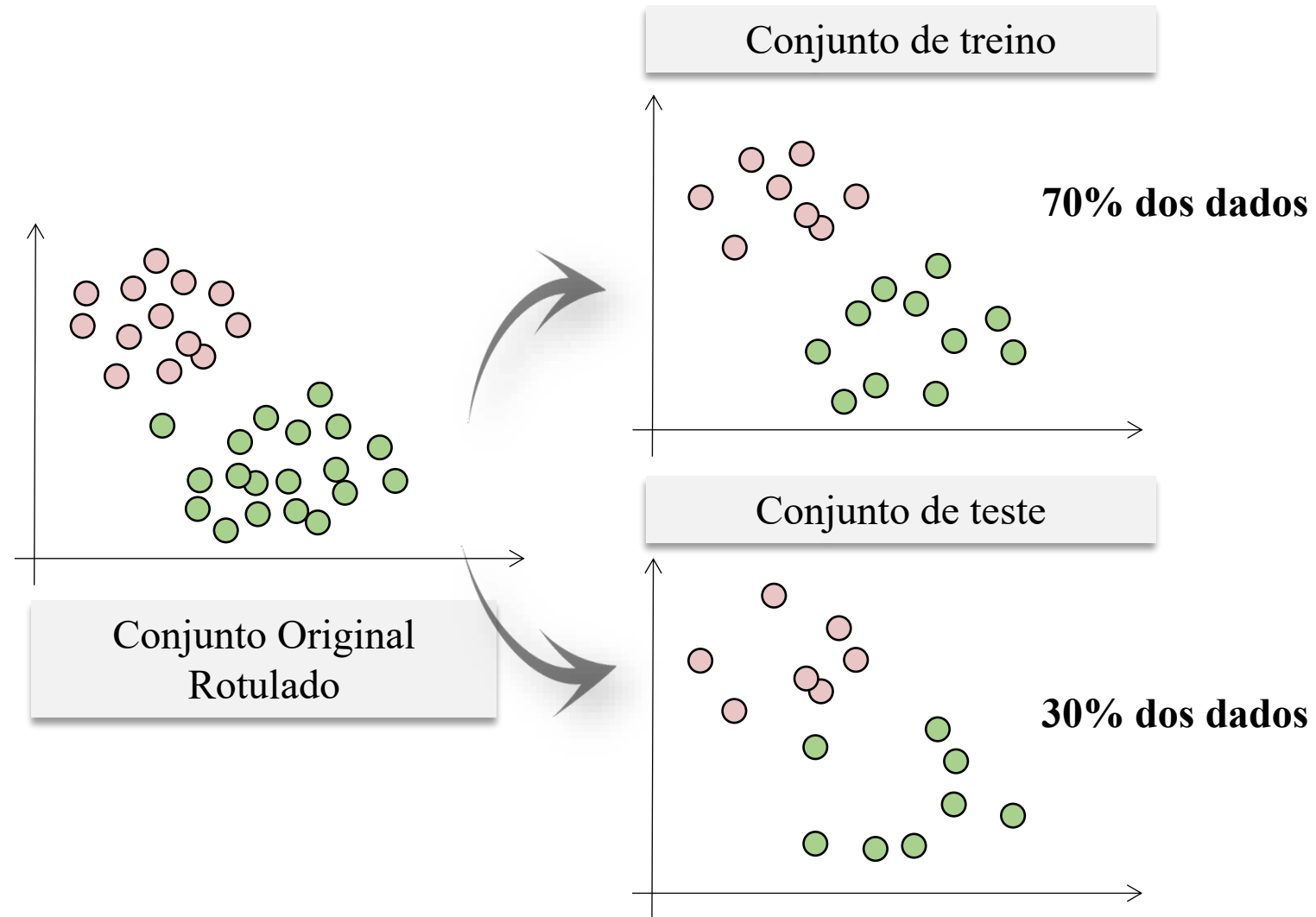
- O modelo não pode ver os dados de teste durante o treinamento.
- Caso isso ocorra, seria o equivalente a fazer uma prova sabendo de antemão quais são as questões e quais são as respostas.
- **Ou seja, os testes devem ser feitos com dados que o modelo não viu anteriormente.**



Conjunto de treino e teste

Hold-out

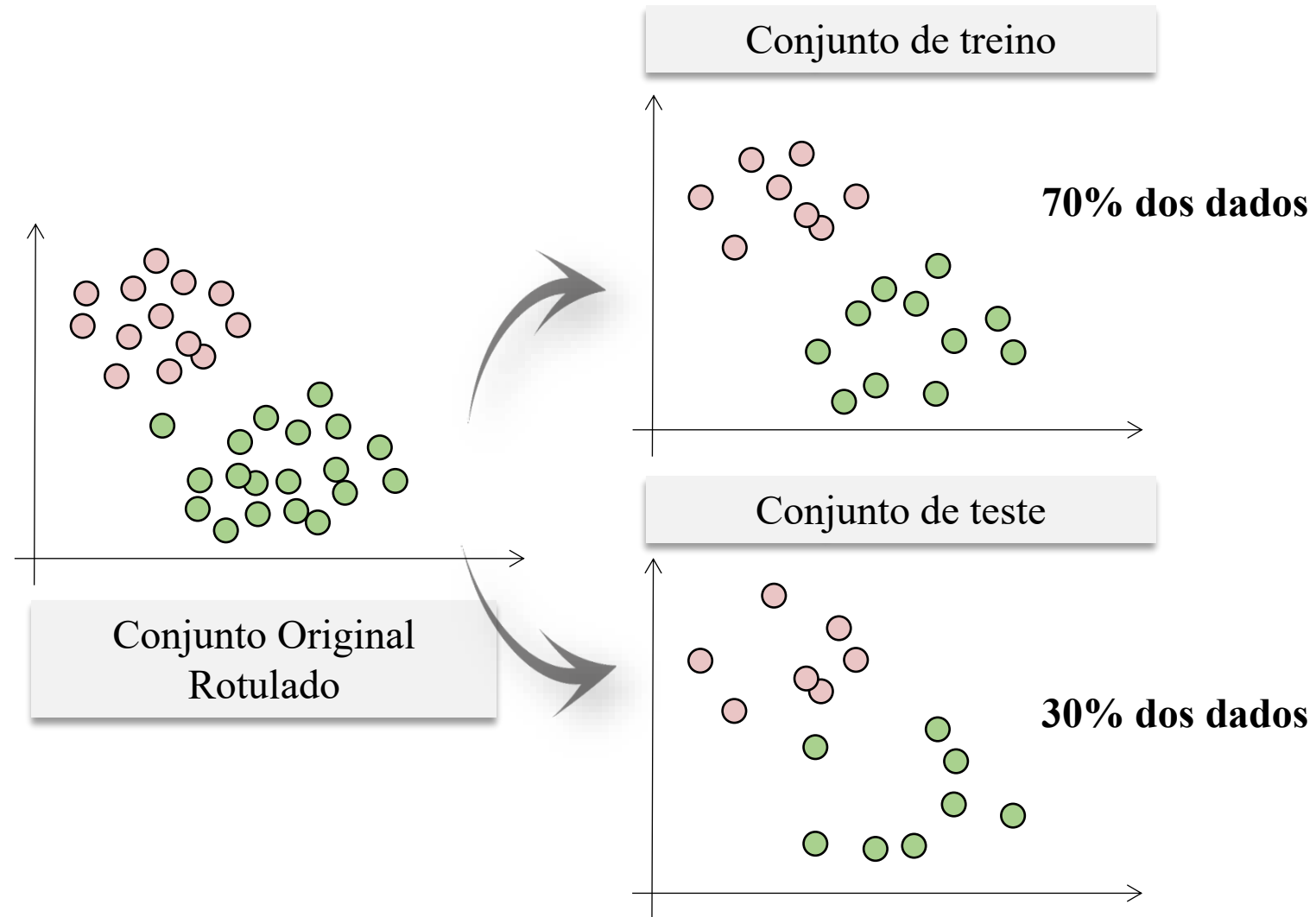
- O método mais simples de testar o modelo consiste em separar, de forma aleatória, uma parcela dos dados para testar o modelo, e utilizar o restante para treinamento.
- Isso é conhecido como *hold-out*.
- Geralmente, cerca de 30% dos dados são separados para teste.
- Essa técnica pode ser realizada várias vezes na mesma base de dados, para se obter a média do desempenho do classificador.
- Está sujeita à aleatoriedades do processo de treinamento e teste.
- Reduz a quantidade de amostras que podem ser usadas no treinamento.



Conjunto de treino e teste

Hold-out

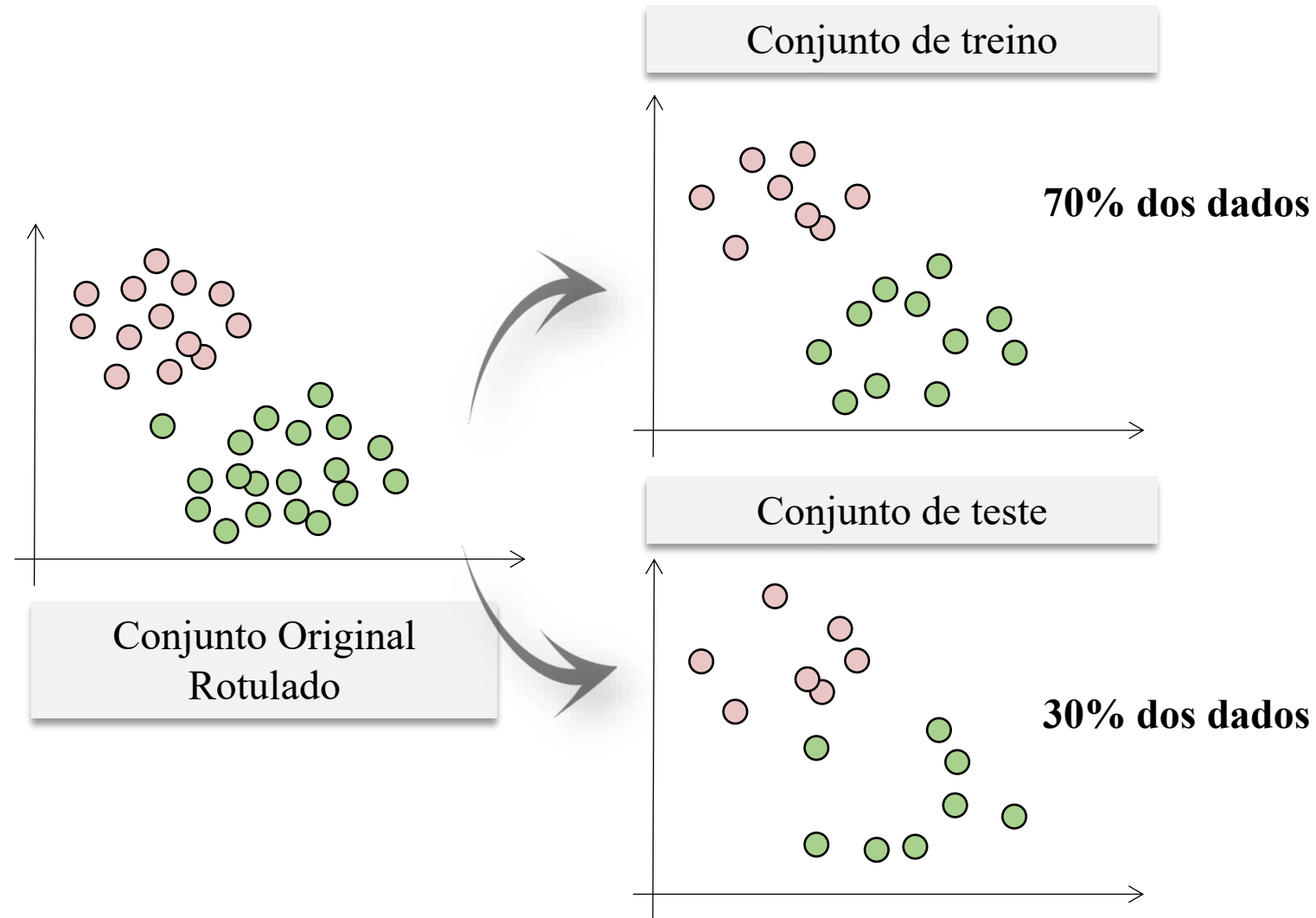
- O **método mais simples** de testar o modelo consiste em separar, de forma **aleatória**, uma parcela dos dados para testar o modelo, e utilizar o restante para treinamento.
- Isso é conhecido como *hold-out*.
- Geralmente, cerca de 30% dos dados são separados para teste.
- Essa técnica pode ser realizada várias vezes na mesma base de dados, para se obter a média do desempenho do classificador.
- Está sujeita à aleatoriedades do processo de treinamento e teste.
- Reduz a quantidade de amostras que podem ser usadas no treinamento.



Conjunto de treino e teste

Hold-out

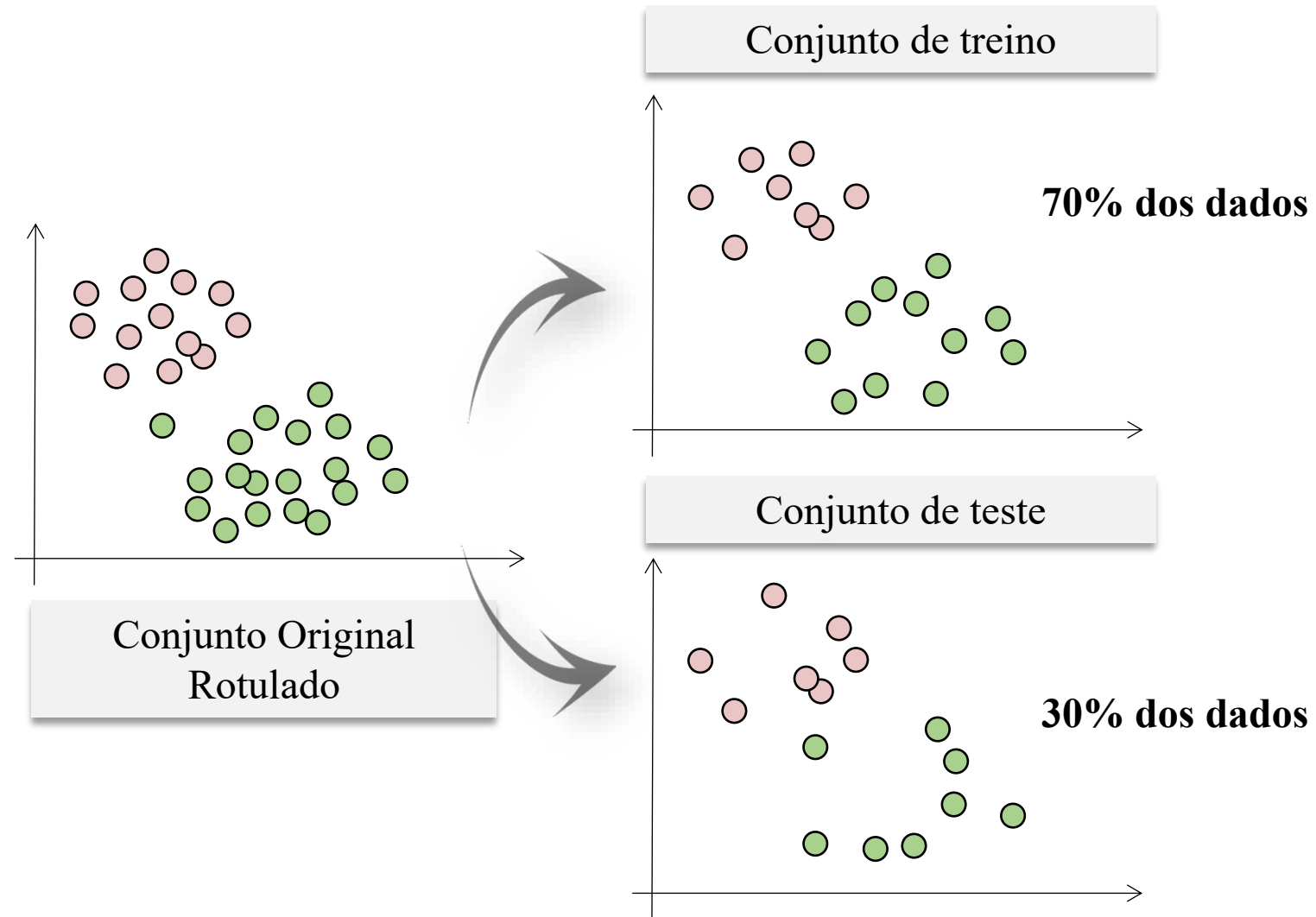
- O método mais simples de testar o modelo consiste em separar, de forma aleatória, uma parcela dos dados para testar o modelo, e utilizar o restante para treinamento.
- Isso é conhecido como *hold-out*.
- Geralmente, cerca de 30% dos dados são separados para teste.
- Essa técnica pode ser realizada várias vezes na mesma base de dados, para se obter a média do desempenho do classificador.
- Está sujeita à aleatoriedades do processo de treinamento e teste.
- Reduz a quantidade de amostras que podem ser usadas no treinamento.



Conjunto de treino e teste

Hold-out

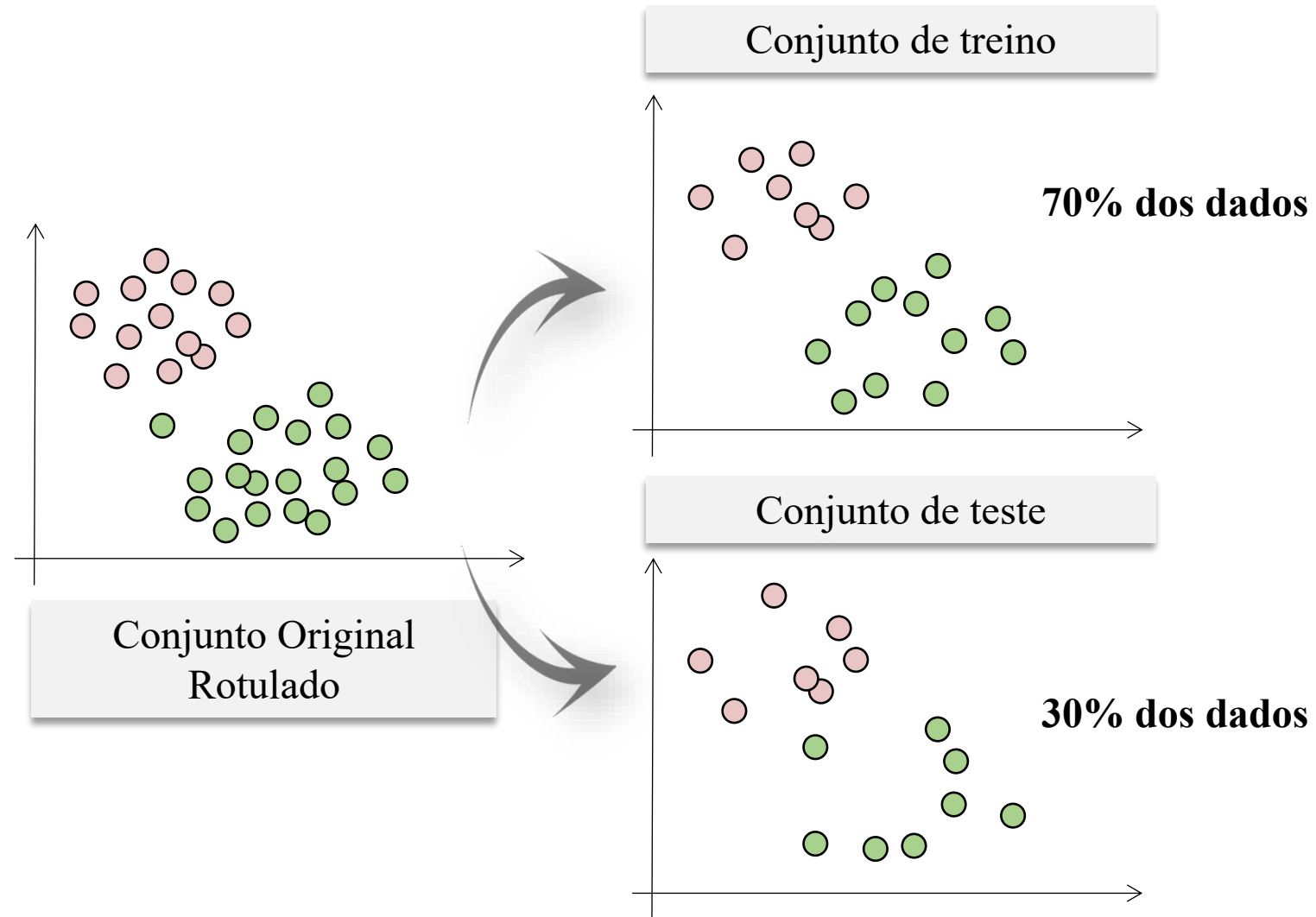
- O método mais simples de testar o modelo consiste em separar, de forma aleatória, uma parcela dos dados para testar o modelo, e utilizar o restante para treinamento.
- Isso é conhecido como *hold-out*.
- **Geralmente, cerca de 30% dos dados são separados para teste.**
- Essa técnica pode ser realizada várias vezes na mesma base de dados, para se obter a média do desempenho do classificador.
- Está sujeita à aleatoriedades do processo de treinamento e teste.
- Reduz a quantidade de amostras que podem ser usadas no treinamento.



Conjunto de treino e teste

Hold-out

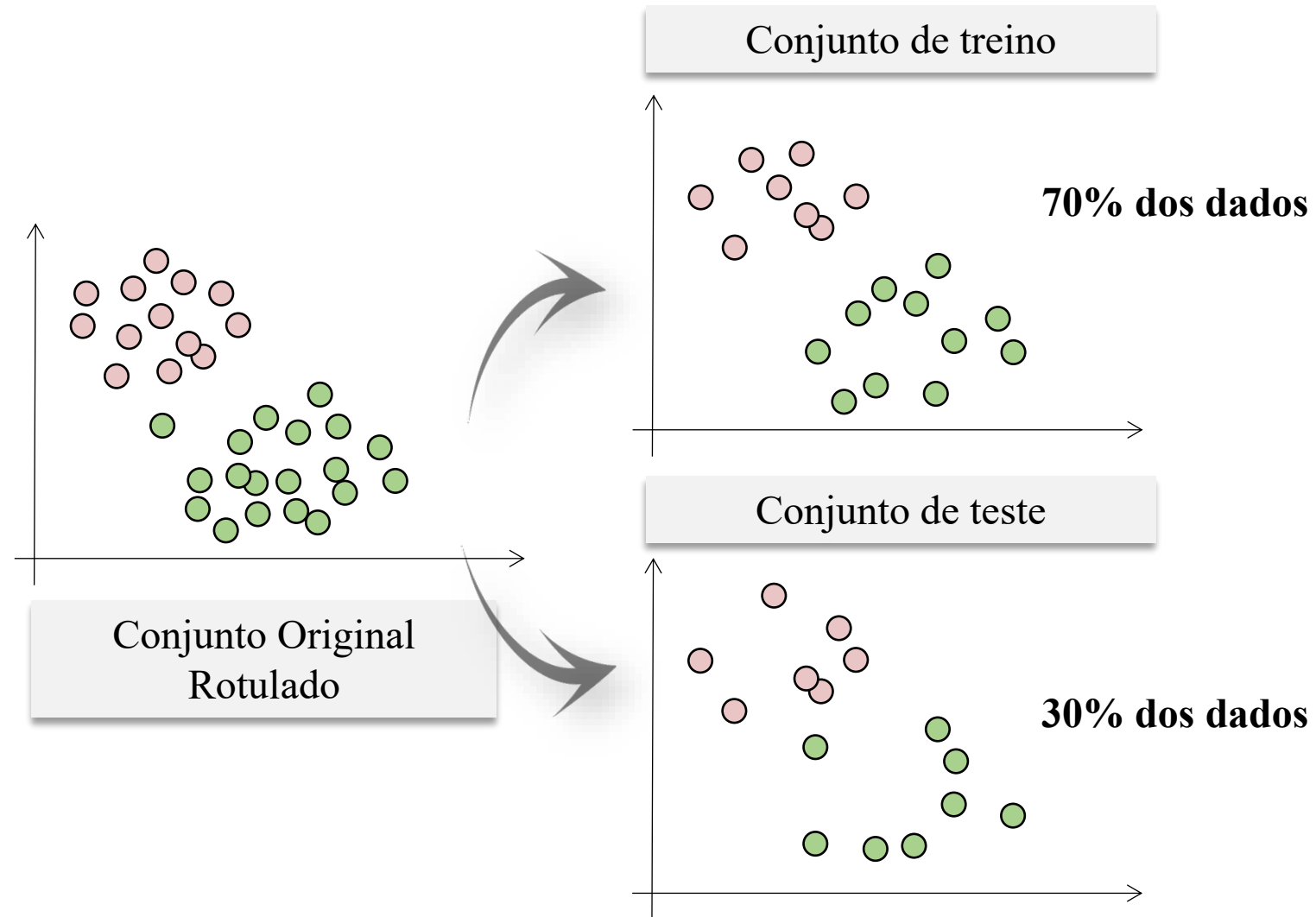
- O método mais simples de testar o modelo consiste em separar, de forma aleatória, uma parcela dos dados para testar o modelo, e utilizar o restante para treinamento.
- Isso é conhecido como *hold-out*.
- Geralmente, cerca de 30% dos dados são separados para teste.
- Essa técnica pode ser **realizada várias vezes** na mesma base de dados, para se obter a **média do desempenho do classificado**.
- Está sujeita à aleatoriedades do processo de treinamento e teste.
- Reduz a quantidade de amostras que podem ser usadas no treinamento.



Conjunto de treino e teste

Hold-out

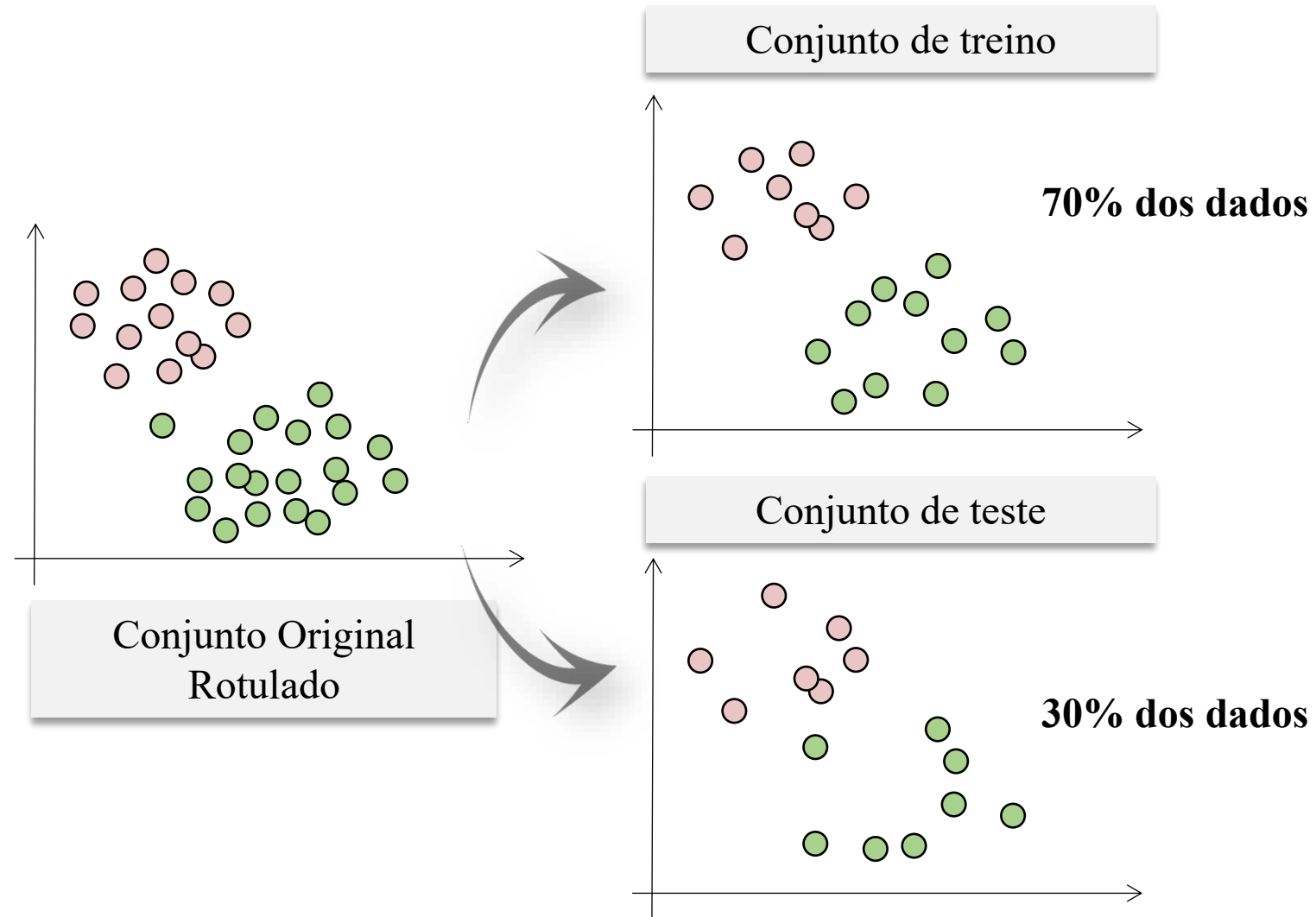
- O método mais simples de testar o modelo consiste em separar, de forma aleatória, uma parcela dos dados para testar o modelo, e utilizar o restante para treinamento.
- Isso é conhecido como *hold-out*.
- Geralmente, cerca de 30% dos dados são separados para teste.
- Essa técnica pode ser realizada várias vezes na mesma base de dados, para se obter a média do desempenho do classificador.
- **Está sujeita à aleatoriedades do processo de treinamento e teste.**
- Reduz a quantidade de amostras que podem ser usadas no treinamento.



Conjunto de treino e teste

Hold-out

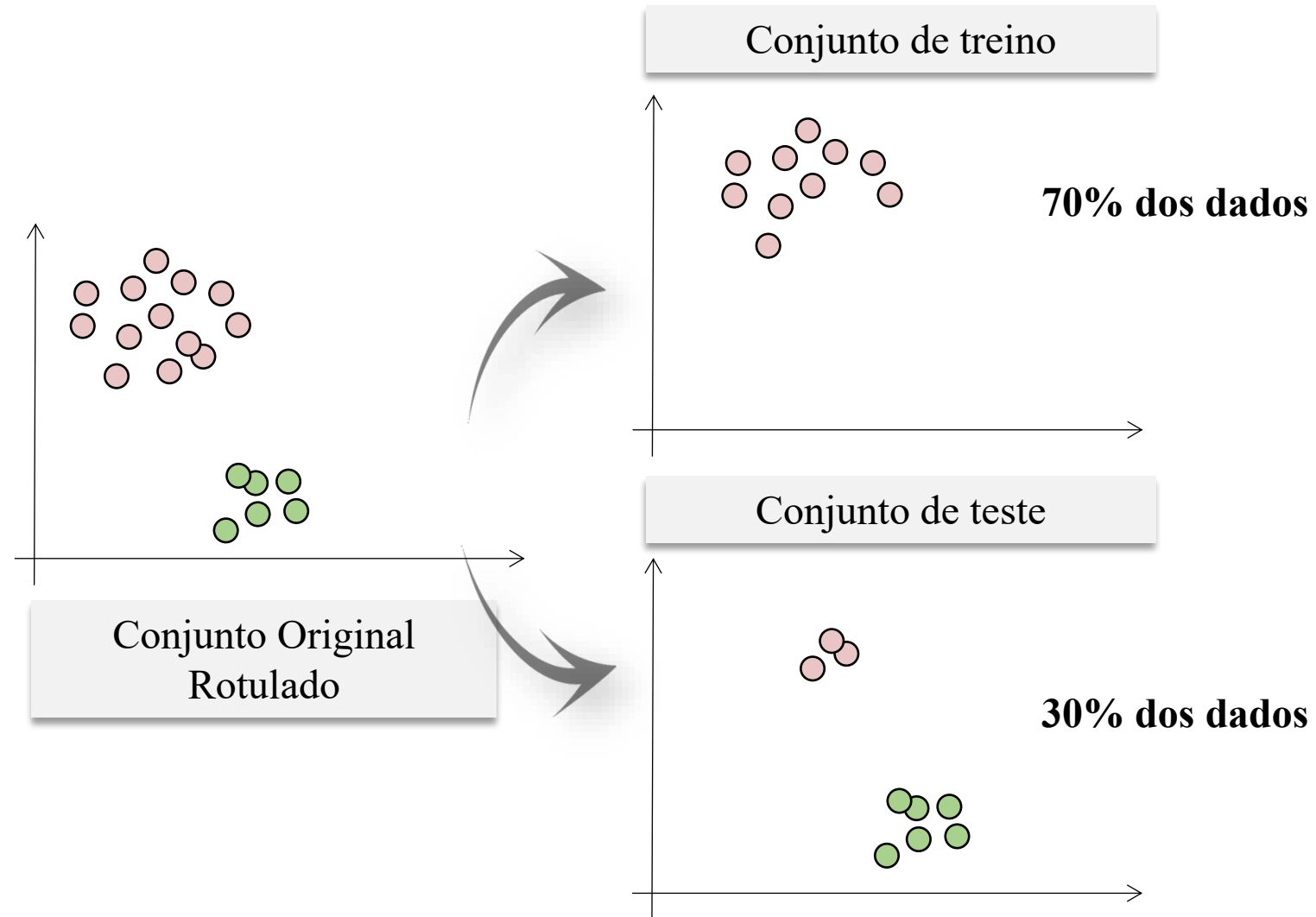
- O método mais simples de testar o modelo consiste em separar, de forma aleatória, uma parcela dos dados para testar o modelo, e utilizar o restante para treinamento.
- Isso é conhecido como *hold-out*.
- Geralmente, cerca de 30% dos dados são separados para teste.
- Essa técnica pode ser realizada várias vezes na mesma base de dados, para se obter a média do desempenho do classificador.
- Está sujeita à aleatoriedades do processo de treinamento e teste.
- **Reduz a quantidade de amostras que podem ser usadas no treinamento.**



Conjunto de treino e teste

Hold-out

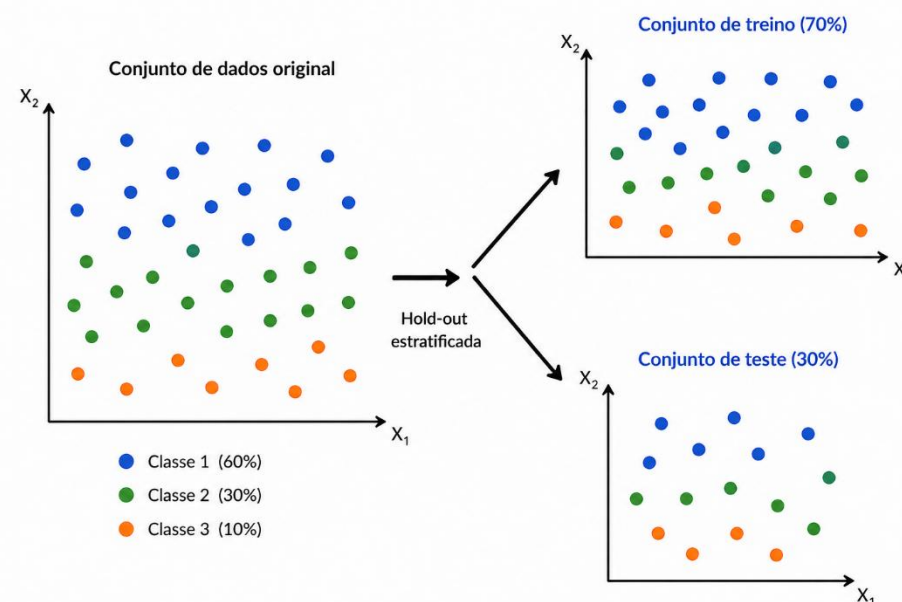
- **Problema!**
 - Algumas classes podem ficar sem representantes ou com poucos representantes (insuficiente para treinamento).
 - A distribuição do número de elementos em cada classe no conjunto original pode não estar representada no conjunto de treino e teste.



Conjunto de treino e teste

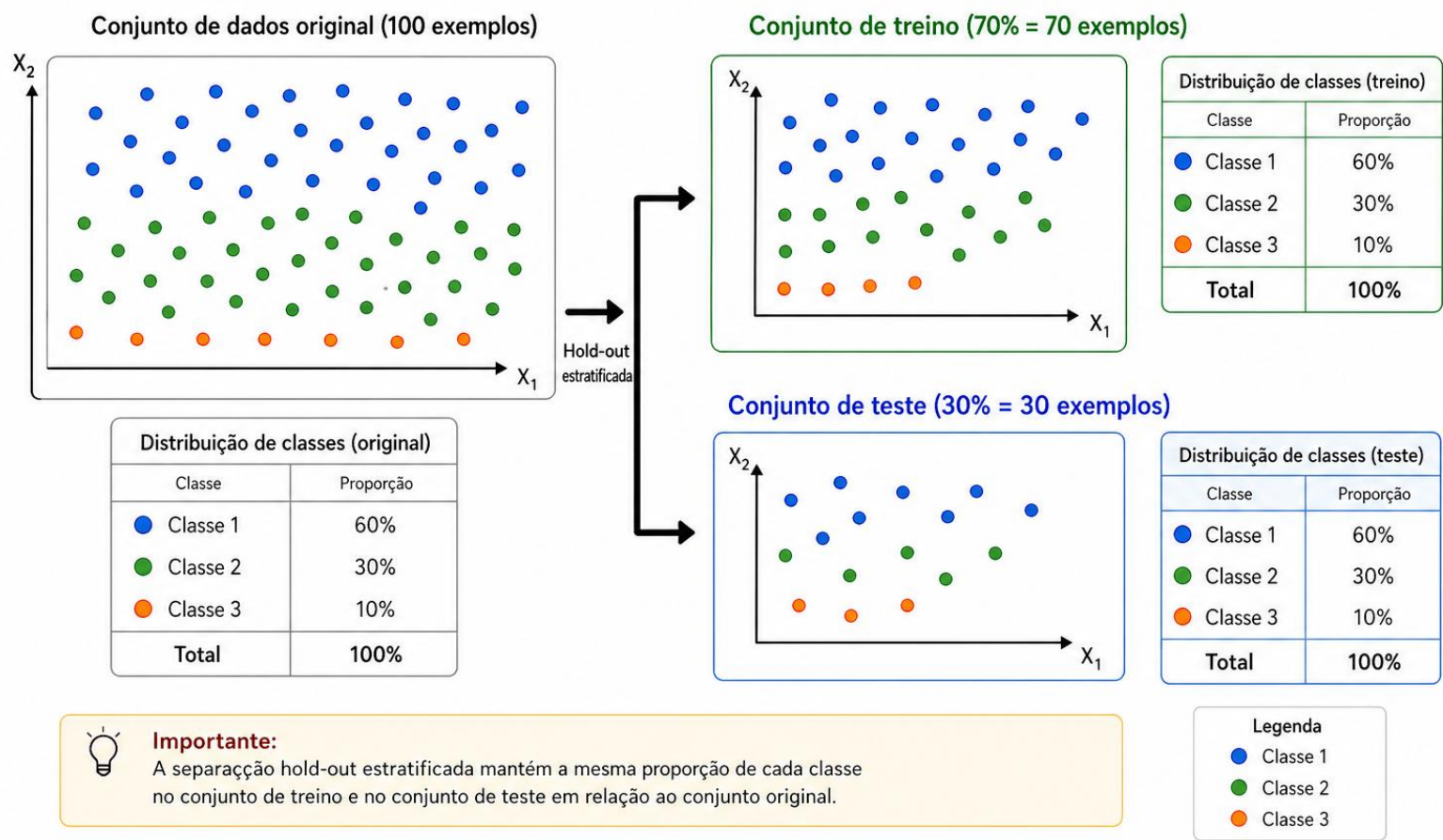
Separação estratificada

- Na separação estratificada, primeiro, identificamos a proporção de exemplos em cada classe, por exemplo:
 - Suponha uma base de dados com 60% de dados positivos e 40% de dados negativos;
 - Na separação do conjunto de treino e teste, independente da técnica, **ambos terão a mesma proporção de dados de cada classe**;
 - O objetivo é que tanto o conjunto de treino quanto o de teste sejam representativos em termos de proporção de classes.
 - Entretanto, **nem sempre conseguimos** selecionar **exatamente** as **distribuições original** das classes, porém, o ideal é que a separação se aproxime da distribuição original de classes.



Separação Hold-out estratificada

Divisão do conjunto de dados preservando a proporção das classes



Conjunto de treino e teste

Separação estratificada

ID	Comprimento Sépala	Largura Sépala	Classe
1	5.1	3.5	Setosa
2	4.9	3.0	Setosa
3	4.7	3.2	Setosa
4	5.0	3.4	Setosa
5	5.4	3.9	Setosa
6	7.0	3.2	Versicolor
7	6.4	3.2	Versicolor
8	6.9	3.1	Versicolor
9	6.3	3.3	Versicolor
10	5.8	2.7	Setosa

- Total:
 - Setosa = 6 amostras (60%);
 - Versicolor = 4 amostras (40%).
- Separação *Holdout* Estratificada (70% treino, 30% teste):
 - Cálculo da divisão por classe:
 - Setosa: $6 \times 70\% = 4.2 \rightarrow 4$ no treino, 2 no teste;
 - Versicolor: $4 \times 70\% = 2.8 \rightarrow 3$ no treino, 1 no teste.

- Conjunto total (10 instâncias)
 - Setosa $\rightarrow [1, 2, 3, 4, 5, 10]$ (6 instâncias);
 - Versicolor $\rightarrow [6, 7, 8, 9]$ (4 instâncias).
- *Holdout* estratificado (70% / 30%):
 - Treino:
 - Setosa $\rightarrow [1, 2, 3, 4]$;
 - Versicolor $\rightarrow [6, 7, 8]$.
 - Teste:
 - Setosa $\rightarrow [5, 10]$;
 - Versicolor $\rightarrow [9]$.

- Porque 70%/30% ou 80%/20%?
 - No passado — anos noventa e início dos anos dois mil — **dados eram extremamente escassos.**
 - Os conjuntos de dados clássicos (Iris, Wine, Sonar, Glass, Breast Cancer, etc.) tinham:
 - Dezenas;
 - Centenas;
 - ou, no máximo, poucos milhares de instâncias.
 - Naquela época, **cada amostra era valiosa.**
 - Se você separasse em cinquenta por cento e cinquenta por cento, perderia metade do dataset para treinar — e isso mataria a capacidade de modelos simples aprenderem.
 - Por isso surgiu a cultura de:
 - **Treino grande;**
 - **Teste pequeno;**
 - **Maximizar o uso de cada instância de treino.**

- Porque 70%/30% ou 80%/20%?
 - **No passado — anos noventa e início dos anos dois mil — dados eram extremamente escassos.**
 - Os conjuntos de dados clássicos (Iris, Wine, Sonar, Glass, Breast Cancer, etc.) tinham:
 - Dezenas;
 - Centenas;
 - ou, no máximo, poucos milhares de instâncias.
 - **Naquela época, cada amostra era valiosa.**
 - Se você separasse em cinquenta por cento e cinquenta por cento, perderia metade do dataset para treinar — e isso mataria a capacidade de modelos simples aprenderem.
 - **Por isso surgiu a cultura de:**
 - **Treino grande;**
 - **Teste pequeno;**
 - **Maximizar o uso de cada instância de treino.**

- **A explicação matemática por trás do 70%/30%:**
 - A divisão entre treino e teste envolve um **trade-off entre viés e variância**:
 - **Treino maior → menor viés do modelo**:
 - Com mais dados, o modelo aprende melhor e o erro de treino fica mais próximo do erro real.
 - **Teste maior → menor variância da estimativa**:
 - Com mais amostras no teste, a avaliação final é mais estável e menos sensível à sorte.
 - Setenta por cento e trinta por cento e oitenta por cento e vinte por cento são divisões clássicas porque, nos conjuntos pequenos:
 - uma parte pequena (vinte ou trinta por cento) já oferece um teste **minimamente estável**;
 - a parte maior (setenta ou oitenta por cento) maximiza o treino, reduzindo o viés;
 - Esse equilíbrio é matematicamente razoável.
 - Mas **isso só importa quando o conjunto de dados é pequeno ou moderado**.

- **A explicação matemática por trás do 70%/30%:**
 - A divisão entre treino e teste envolve um **trade-off entre viés e variância**:
 - **Treino maior → menor viés do modelo:**
 - Com mais dados, o modelo aprende melhor e o erro de treino fica mais próximo do erro real.
 - **Teste maior → menor variância da estimativa:**
 - Com mais amostras no teste, a avaliação final é mais estável e menos sensível à sorte.
 - Setenta por cento e trinta por cento e oitenta por cento e vinte por cento são divisões clássicas porque, nos conjuntos pequenos:
 - uma parte pequena (vinte ou trinta por cento) já oferece um teste **minimamente estável**;
 - a parte maior (setenta ou oitenta por cento) maximiza o treino, reduzindo o viés;
 - Esse equilíbrio é matematicamente razoável.
 - **Mas isso só importa quando o conjunto de dados é pequeno ou moderado.**

- **Então o que vale em *big data*?**
 - Em *big data*, a pergunta muda:
 - **Quanto posso colocar no teste sem atrapalhar o custo computacional?**
 - Em muitos sistemas, armazenar milhões de instâncias para teste:
 - aumenta o custo;
 - aumenta o tempo de avaliação;
 - exige mais memória;
 - exige mais infraestrutura.
 - Por isso, você pode ver **testes pequenos**, como cinco por cento, e mesmo assim a avaliação é ótima.
- Exemplo:
 - Alguns trabalhos, por exemplo, usam **noventa e cinco por cento para treino e/ou cinco por cento para teste**, e o resultado é perfeitamente estável, como dito em:
 - “With tens of millions of training examples, we reserve only 1–5% for testing.” - *Large-Scale Machine Learning Systems*, ACM Computing Surveys, 2017.
 - “In large-scale settings with millions of samples, validation and test sets often comprise a small fraction of data, typically around 1–5%.” - *Industrial-Scale Machine Learning: Challenges and Applications*, ACM Computing Surveys, 2022.
 - “For very large datasets, allocating even 1% for evaluation can result in millions of samples, which is more than enough for stable statistics.” - Hapke & Nelson, *Deploying Machine Learning Models*, O’Reilly, 2020.

- **Então o que vale em *big data*?**
 - Em *big data*, a pergunta muda:
 - **Quanto posso colocar no teste sem atrapalhar o custo computacional?**
 - Em muitos sistemas, armazenar milhões de instâncias para teste:
 - aumenta o custo;
 - aumenta o tempo de avaliação;
 - exige mais memória;
 - exige mais infraestrutura.
 - **Por isso, você pode ver testes pequenos, como cinco por cento, e mesmo assim a avaliação é ótima.**
- Exemplo:
 - **Alguns trabalhos, por exemplo, usam noventa e cinco por cento para treino e/ou cinco por cento para teste**, e o resultado é perfeitamente estável, como dito em:
 - “With tens of millions of training examples, we reserve only 1–5% for testing.” - *Large-Scale Machine Learning Systems*, ACM Computing Surveys, 2017.
 - “In large-scale settings with millions of samples, validation and test sets often comprise a small fraction of data, typically around 1–5%.” - *Industrial-Scale Machine Learning: Challenges and Applications*, ACM Computing Surveys, 2022.
 - “For very large datasets, allocating even 1% for evaluation can result in millions of samples, which is more than enough for stable statistics.” - Hapke & Nelson, *Deploying Machine Learning Models*, O’Reilly, 2020.

- **Qual é o problema matemático do *hold-out*?**
 - O problema é: **a estimativa depende da sorte.**
 - Se a divisão 80/20 ou 70/30 pegou:
 - exemplos fáceis no teste → o modelo parece melhor do que é;
 - exemplos difíceis no teste → parece pior do que é.
 - Matematicamente, isso significa:
 - **Alta variância na estimativa do erro;**
 - **A avaliação muda quando você troca o *random seed*;**
 - **A divisão única não é representativa da distribuição real;**
 - **Você está usando poucos dados para avaliar** (especialmente em datasets pequenos).

- **Qual é o problema matemático do *hold-out*?**
 - **O problema é: a estimativa depende da sorte.**
 - Se a divisão 80/20 ou 70/30 pegou:
 - **exemplos fáceis no teste** → o modelo parece melhor do que é;
 - **exemplos difíceis no teste** → parece pior do que é.
 - Matematicamente, isso significa:
 - **Alta variância na estimativa do erro;**
 - **A avaliação muda quando você troca o *random seed*;**
 - **A divisão única não é representativa da distribuição real;**
 - **Você está usando poucos dados para avaliar** (especialmente em datasets pequenos).

- **Como isso aparece na prática?**
 - Se você repetir o *hold-out* com diferentes seeds:
 - O modelo pode ter 85%, 89%, 84%, 90% de desempenho.
 - Ou seja:
 - **O resultado não é confiável;**
 - **Não existe estabilidade estatística.**
 - E o mais grave:
 - Às vezes você acha que um modelo é melhor do que o outro só por sorte da divisão.

- **Como isso aparece na prática?**
 - Se você repetir o *hold-out* com diferentes seeds:
 - O modelo pode ter 85%, 89%, 84%, 90% de desempenho.
 - Ou seja:
 - **O resultado não é confiável;**
 - **Não existe estabilidade estatística.**
 - E o mais grave:
 - Às vezes você acha que um modelo é melhor do que o outro só por sorte da divisão.

- *Cross-validation* responde exatamente duas perguntas:
 - E se eu treinar e testar em **várias divisões diferentes** e depois combinar as avaliações utilizando média e desvio padrão?
 - Como eu posso fazer com que o meu algoritmo treine e teste com todos os dados do meu conjunto?
- Matematicamente, isso reduz drasticamente:
 - **Variância** da estimativa;
 - **Dependência da sorte**;
 - **Tendência causada por uma única divisão ruim.**
- E aumenta:
 - **Representatividade da avaliação**;
 - **Utilização total do conjunto de dados**;
 - **Confiabilidade na comparação entre modelos.**

- *Cross-validation* responde exatamente duas perguntas:
 - E se eu treinar e testar em **várias divisões diferentes** e depois combinar as avaliações utilizando média e desvio padrão?
 - Como eu posso fazer com que o meu algoritmo treine e teste com todos os dados do meu conjunto?
- Matematicamente, isso reduz drasticamente:
 - **Variância** da estimativa;
 - **Dependência da sorte**;
 - **Tendência causada por uma única divisão ruim.**
- E aumenta:
 - **Representatividade da avaliação**;
 - **Utilização total do conjunto de dados**;
 - **Confiabilidade na comparação entre modelos.**

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.
- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos *10-fold cross-validation*.
- Temos o seguinte processo:
 1. Dividimos o conjunto em k grupos:
 2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam um único grupo, utilizado para o treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste, salvamos o valor da métrica e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
 3. Analisamos as métricas em cada uma das iterações.

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.

- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos 10-fold cross-validation.

- Temos o seguinte processo:

1. Dividimos o conjunto em k grupos:
2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam o conjunto de treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
3. Analisamos as métricas em cada uma das iterações.

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Fold 1	Fold 2	Fold 3	Fold 4	Fold 5

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.
- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos *10-fold cross-validation*.
- Temos o seguinte processo:
 1. Dividimos o conjunto em k grupos:
 2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam um único grupo, utilizado para o treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste, salvamos o valor da métrica e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
 3. Analisamos as métricas em cada uma das iterações.

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.
- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos *10-fold cross-validation*.
- Temos o seguinte processo:
 1. **Dividimos o conjunto em k grupos:**
 2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam um único grupo, utilizado para o treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste, salvamos o valor da métrica e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
 3. Analisamos as métricas em cada uma das iterações.

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.
- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos *10-fold cross-validation*.
- Temos o seguinte processo:
 1. Dividimos o conjunto em k grupos:
 2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam um único grupo, utilizado para o treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste, salvamos o valor da métrica e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
 3. Analisamos as métricas em cada uma das iterações.

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.
- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos *10-fold cross-validation*.
- Temos o seguinte processo:
 1. Dividimos o conjunto em k grupos:
 2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam um único grupo, utilizado para o treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste, salvamos o valor da métrica e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
 3. Analisamos as métricas em cada uma das iterações.

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.
- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos *10-fold cross-validation*.
- Temos o seguinte processo:
 1. Dividimos o conjunto em k grupos:
 2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam um único grupo, utilizado para o treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste, salvamos o valor da métrica e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
 3. Analisamos as métricas em cada uma das iterações.

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.
- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos *10-fold cross-validation*.
- Temos o seguinte processo:
 1. Dividimos o conjunto em k grupos:
 2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam um único grupo, utilizado para o treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste, salvamos o valor da métrica e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
 3. Analisamos as métricas em cada uma das iterações.

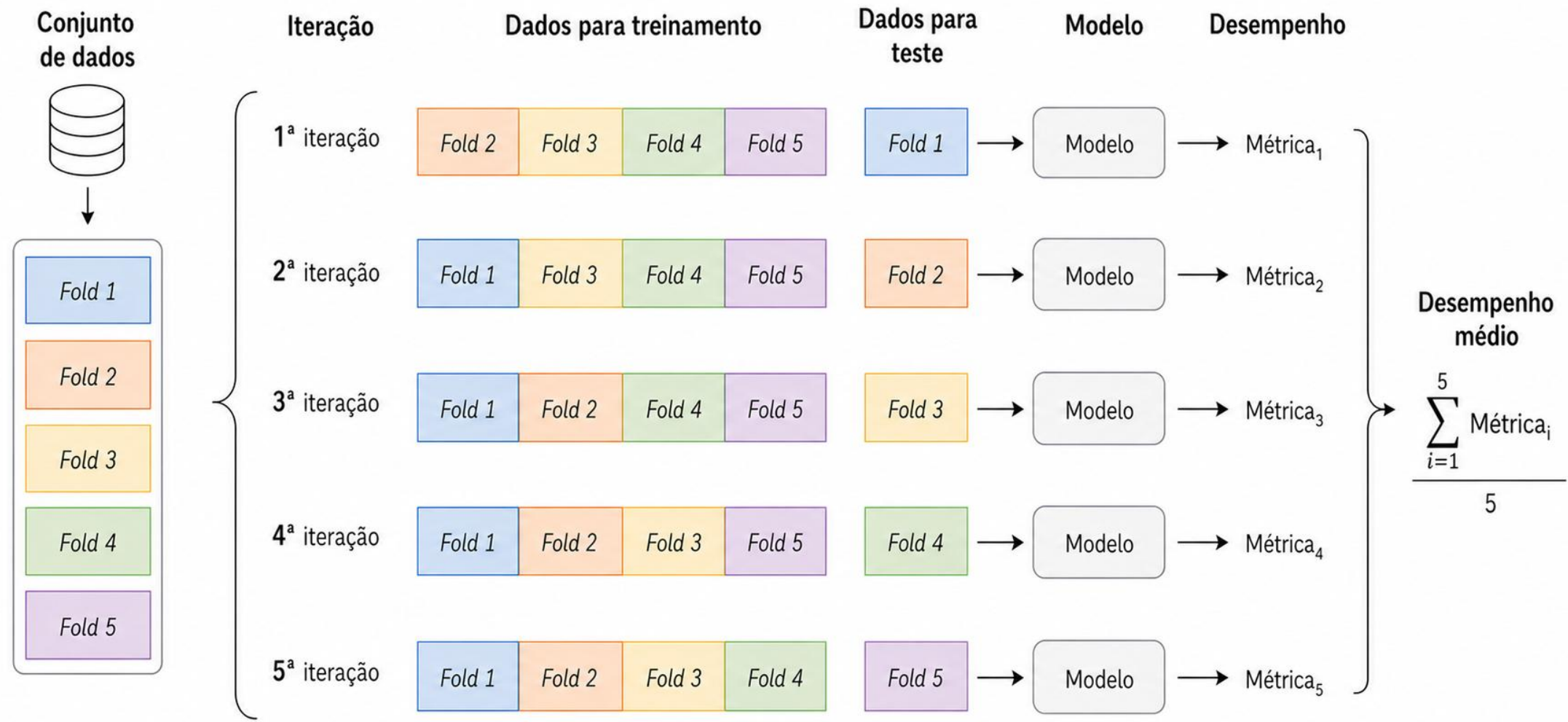
Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Na validação cruzada, o conjunto de dados é dividido aleatoriamente em “ k ” grupos.
- Usamos o número “ k ” para fazer referência ao teste
 - Por exemplo: $k=10$ temos *10-fold cross-validation*.
- Temos o seguinte processo:
 1. Dividimos o conjunto em k grupos:
 2. Fazemos uma iteração para cada grupo (k iterações):
 - A. O grupo selecionado é separado para teste, enquanto os demais se tornam um único grupo, utilizado para o treinamento;
 - B. Treinamos um modelo com os dados de treinamento, testamos com os dados de teste, salvamos o valor da métrica e descartamos o modelo;
 - C. O próximo grupo é selecionado e voltamos ao passo A;
 3. **Analizamos as métricas em cada uma das iterações.**

Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)



Conjunto de treino e teste

Validação cruzada (*k-fold cross-validation*)

- Instâncias = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].
- A cada iteração, uma instância diferente será usada como teste e as outras 9 como treino.

<i>Fold</i>	Teste	Treino
1	[1]	[2, 3, 4, 5, 6, 7, 8, 9, 10]
2	[2]	[1, 3, 4, 5, 6, 7, 8, 9, 10]
3	[3]	[1, 2, 4, 5, 6, 7, 8, 9, 10]
4	[4]	[1, 2, 3, 5, 6, 7, 8, 9, 10]
5	[5]	[1, 2, 3, 4, 6, 7, 8, 9, 10]
6	[6]	[1, 2, 3, 4, 5, 7, 8, 9, 10]
7	[7]	[1, 2, 3, 4, 5, 6, 8, 9, 10]
8	[8]	[1, 2, 3, 4, 5, 6, 7, 9, 10]
9	[9]	[1, 2, 3, 4, 5, 6, 7, 8, 10]
10	[10]	[1, 2, 3, 4, 5, 6, 7, 8, 9]

- *K-Fold* com $k = 10$:
 - A cada iteração, uma instância diferente será usada como teste e as outras 9 como treino;
 - Cada instância será usada exatamente uma vez como teste, e nove vezes como treino.

Matriz de confusão

Avaliação

Matriz de confusão

- Representação do erro por classe.
- A matriz de confusão ajuda a avaliar o desempenho do modelo de classificação no aprendizado de máquina **comparando os valores previstos com os valores reais de um conjunto de dados.**

		Classe predita	
		Positivo	Negativo
Classe Verdadeira	Positivo	VP	FN
	Negativo	FP	VN

<https://www.ibm.com/br-pt/topics/confusion-matrix>

Avaliação

Matriz de confusão

- Dada uma amostra de 13 fotos, 8 de gatos e 5 de cachorros, onde os gatos pertencem à classe 1 e os cães pertencem à classe 0:
 - Dados reais = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0].
- Suponha que um classificador que distingue entre cachorros e gatos seja treinado. Das 13 fotos examinadas, o classificador faz 8 previsões precisas e erra 5, sendo que 3 gatos são erroneamente previstos como cachorros (primeiras 3 previsões) e 2 cachorros são erroneamente previstos como gatos (últimas 2 previstos):
 - Predição = [0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1].
- Com esses dois conjuntos rotulados (reais e previsões), podemos criar uma matriz de confusão que resumirá os resultados do teste do classificador:

														Classe predita	
														Gato	Cachorro
Real	Predito	1	1	1	1	1	1	1	1	0	0	0	0	0	
		0	0	0	1	1	1	1	1	0	0	0	1	1	

Classe Verdadeira	Gato	VP = 5	FN = 3
	Cachorro	FP = 2	VN = 3

https://pt.wikipedia.org/wiki/Matriz_de_confus%C3%A3o

Avaliação

Matriz de confusão

- Dada uma amostra de 13 fotos, 8 de gatos e 5 de cachorros, onde os gatos pertencem à classe 1 e os cães pertencem à classe 0:
 - Dados reais = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0].
- Suponha que um classificador que distingue entre cachorros e gatos seja treinado. 8 previsões precisas e erra 5, sendo que 3 gatos são erroneamente previstos como cachorros e 2 cachorros são erroneamente previstos como gatos (últimas 2 previstos):
 - Predição = [0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1].
- Com esses dois conjuntos rotulados (reais e previsões), podemos criar uma matriz de confusão que resumirá os resultados do teste do classificador:

É fácil inspecionar visualmente a tabela em busca de erros de previsão, pois eles serão representados por valores fora da diagonal.

Real	1	1	1	1	1	1	1	0	0	0	0	0
Predito	0	0	0	1	1	1	1	0	0	0	1	1

		Classe predita	
		Gato	Cachorro
Classe Verdadeira	Gato	VP = 5	FN = 3
	Cachorro	FP = 2	VN = 3

https://pt.wikipedia.org/wiki/Matriz_de_confus%C3%A3o

Avaliação

Matriz de confusão

- É claro que esse modelo é para um problema de classificação binária.
- A **matriz de confusão** também **pode visualizar resultados para problemas de classificação multiclass**.
- Por exemplo:
 - Imagine que estamos desenvolvendo um modelo de classificação de espécies como parte de um programa de conservação da vida marinha;
 - O modelo prevê as espécies de peixes. Uma matriz de confusão para um problema de classificação multiclass poderia ter a seguinte aparência:

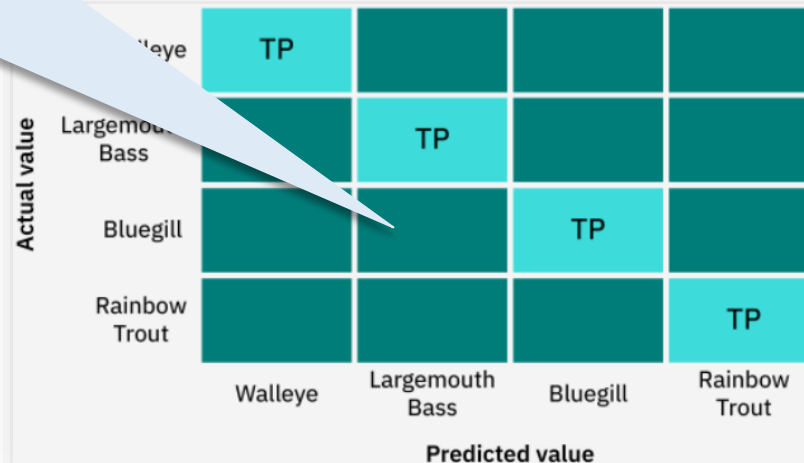
Actual value	Walleye	TP			
	Largemouth Bass		TP		
	Bluegill			TP	
	Rainbow Trout				TP
		Walleye	Largemouth Bass	Bluegill	Rainbow Trout
		Predicted value			

<https://www.ibm.com/br-pt/topics/confusion-matrix>

Avaliação

Matriz de confusão

- É claro que esse modelo é para um problema de classificação binária.
 - **Resultados para problemas de classificação multiclass.**
 - Todas as células da **diagonal** indicam:
 - **Verdadeiros positivos** previstos.
 - As **outras células** fornecem quantidades para:
 - **Falsos positivos;**
 - **Falsos negativos;** e
 - **Verdadeiros negativos.**
- Modelo de classificação de espécies como parte de um programa de conservação
- matriz de confusão para um problema de classificação multiclass poderia



	Walleye	Largemouth Bass	Bluegill	Rainbow Trout
Actual value	Walleye	Largemouth Bass	Bluegill	Rainbow Trout
	TP			
		TP		
			TP	
				TP
	Walleye	Largemouth Bass	Bluegill	Rainbow Trout
	Predicted value			

<https://www.ibm.com/br-pt/topics/confusion-matrix>

Avaliação

Matriz de confusão

- Em problema **multiclasse**, quando você quer analisar **VP, FP, FN e VN** para uma classe específica, você faz exatamente a ideia de **“uma classe vs. o resto”**:
 - A classe escolhida vira a **classe positiva**;
 - As outras duas viram, juntas, a **classe negativa**.
- Ou seja, em **multiclasse**, para analisar uma classe específica, fazemos:
- No exemplo abaixo, para analisar a classe **A**:
 - **Positiva** = A;
 - **Negativa** = {B, C}.
- Assim, transformamos o problema **multiclasse** em um problema **binário temporário**.

- Primeiro montamos a matriz de confusão e já conseguimos observar algumas coisas. Entretanto, não conseguimos calcular VP, FP, VN e FN.

		Classe predita		
		A	B	C
Classe Verdadeira	A	2	1	0
	B	0	2	1
	C	1	1	2

Avaliação

Matriz de confusão

- Em problema **multiclasse**, quando você quer analisar **VP, FP, FN e VN** para uma classe específica, você faz exatamente a ideia de **“uma classe vs. o resto”**:
 - A classe escolhida vira a **classe positiva**;
 - As outras duas viram, juntas, a **classe negativa**.
- Ou seja, em **multiclasse**, para analisar uma classe específica, fazemos:
- No exemplo abaixo, para analisar a classe **A**:
 - **Positiva** = A;
 - **Negativa** = {B, C}.
- Assim, transformamos o problema **multiclasse** em um problema **binário temporário**.

- Depois fixamos uma classe e as outras são definidas como negativas, assim conseguimos calcular VP, FP, VN e FN.

$VP = 2$

$FP = 1$

$FN = 1$

$VN = 6$

		Classe predita		
		A	B	C
Classe Verdadeira	A	VP=2	FN=1	FN=0
	B	FP=0	VN=2	VN=1
	C	FP=1	VN=1	VN=2

Avaliação

Matriz de confusão

- Em problema **multiclasse**, quando você quer analisar **VP, FP, FN e VN** para uma classe específica, você faz exatamente a ideia de **“uma classe vs. o resto”**:
 - A classe escolhida vira a **classe positiva**;
 - As outras duas viram, juntas, a **classe negativa**.
- Ou seja, em **multiclasse**, para analisar uma classe específica, fazemos:
- No exemplo abaixo, para analisar a classe **A**:
 - **Positiva** = A;
 - **Negativa** = {B, C}.
- Assim, transformamos o problema **multiclasse** em um problema **binário temporário**.

- Depois fixamos uma classe e as outras são definidas como negativas, assim conseguimos calcular VP, FP, VN e FN.

$VP = 2$

$FP = 1$

$FN = 1$

$VN = 6$

		Classe predita		
		A	B	C
Classe Verdadeira	A	VP=2	FN=1	FN=0
	B	FP=0	VN=2	VN=1
	C	FP=1	VN=1	VN=2

- Isso está confuso.

- Em problema **multiclasse**, quando você quer analisar **VP, FP, FN e VN** para uma classe específica, você faz exatamente a ideia de **“uma classe vs. o resto”**:
 - A classe escolhida vira a **classe positiva**;
 - As outras duas viram, juntas, a **classe negativa**.
- Ou seja, em **multiclasse**, para analisar uma classe específica, fazemos:
- No exemplo abaixo, para analisar a classe **A**:
 - **Positiva** = A;
 - **Negativa** = {B, C}.
- Assim, transformamos o problema **multiclasse** em um problema **binário temporário**.

• Por fim, melhoramos a matriz para não gerar confusão.

$VP = 2$

$FP = 1$

$FN = 1$

$VN = 6$

		Classe predita	
		A	Não A {B, C}
Classe Verdadeira	A	VP=2	FN=1
	Não A {B, C}	FP=1	VN=6

Acurácia e erro

- Representação do erro por classe em problema de diagnóstico de risco a uma determinada doença:
 - Paciente com alto risco +;
 - Paciente com baixo risco -.

		Classe predita	
		+	-
Classe Verdadeira	+	VP	FN
	-	FP	VN

LORENA, Ana Carolina; GAMA, João; FACELI, Katti. Inteligência Artificial: Uma abordagem de aprendizado de máquina. Grupo Gen-LTC, 2011.

- **Acurácia:**
 - Taxa de acerto da classificação, ou seja, dos itens retornados, qual foi a porcentagem de acerto.
- **Erro:**
 - Taxa de erro da classificação, ou seja, dos itens retornados, qual foi a porcentagem de erro.

		Classe predita	
		+	-
Classe Verdadeira	+	VP	FN
	-	FP	VN

err(f) (points to FN)

acc(f) (points to VP)

$$n = VP + FP + VN + FN$$

$$acc(f) = \frac{VP + VN}{n} = 1 - err(f)$$

$$err(f) = \frac{FP + FN}{n}$$

LORENA, Ana Carolina; GAMA, João; FACELI, Katti. Inteligência Artificial: Uma abordagem de aprendizado de máquina. Grupo Gen-LTC, 2011.

Avaliação

Métricas: Exemplo

- Dada uma amostra de 20 fotos, 10 de macacos e 10 de sapos, onde os macacos pertencem à classe 0 e os sapos pertencem à classe 1:
 - Dados reais = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1].
- Suponha que um classificador que distingue entre macacos e sapos e gatos seja treinado e obtenha o seguinte resultado:
 - Predição = [0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1].
- Com esses dois conjuntos rotulados (reais e previsões), podemos criar uma matriz de confusão que resumirá os resultados do teste do classificador:

Real	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1			
Predito	0	1	0	1	0	0	0	0	0	0	1	1	0	1	1	0	1	0	1	1	Classe predita	
																					Macaco	Sapo
																					?	?
																					?	?

Avaliação

Métricas: Exemplo

- Dada uma amostra de 20 fotos, 10 de macacos e 10 de sapos, onde os macacos pertencem à classe 0 e os sapos pertencem à classe 1:
 - Dados reais = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1].
- Suponha que um classificador que distingue entre macacos e sapos e gatos seja treinado e obtenha o seguinte resultado:
 - Predição = [0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1].
- Com esses dois conjuntos rotulados (reais e previsões), podemos criar uma matriz de confusão que resumirá os resultados do teste do classificador:

Real	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
Predito	0	1	0	1	0	0	0	0	0	0	1	1	0	1	1	0	1	0	1

		Classe predita	
		Macaco	Sapo
Classe Verdadeira	Macaco	8	2
	Sapo	3	7

LORENA, Ana Carolina; GAMA, João; FACELI, Katti. Inteligência Artificial: Uma abordagem de aprendizado de máquina. Grupo Gen-LTC, 2011.

- Agora, podemos computar as medidas de qualidade.

$$acc(f) = \frac{VP + VN}{n} = ?$$

$$err(f) = \frac{FP + FN}{n} = ?$$

		Classe predita	
		Macaco	Sapo
Classe Verdadeira	Macaco	8	2
	Sapo	3	7

- Agora, podemos computar as medidas de qualidade.

$$acc(f) = \frac{VP + VN}{n} = \frac{8 + 7}{8 + 7 + 2 + 3} = 0,75$$

$$err(f) = \frac{FP + FN}{n} = \frac{2 + 3}{8 + 7 + 2 + 3} = 0,25$$

		Classe predita	
		Macaco	Sapo
Classe Verdadeira	Macaco	8	2
	Sapo	3	7

Avaliação

Métricas: Exemplo

- Vamos criar um exemplo ilustrativo para dois classificadores que avaliam pacientes com e sem Câncer.
- Teremos:
 - Classe Positiva (P): Pacientes com Câncer;
 - Classe Negativa (N): Pacientes sem Câncer.
- Vamos simular os dados para dois classificadores:

		<u>Classificador A</u>	
		Classe predita	
		+	-
Classe Verdadeira	+	20	30
	-	5	45

		<u>Classificador B</u>	
		Classe predita	
		+	-
Classe Verdadeira	+	45	5
	-	35	15

LORENA, Ana Carolina; GAMA, João; FACELI, Katti. Inteligência Artificial: Uma abordagem de aprendizado de máquina. Grupo Gen-LTC, 2011.

Classificador A

		Classe predita	
		+	-
Classe Verdadeira	+	20	30
	-	5	45

$$acc(f) = \frac{VP + VN}{n} = \frac{20 + 45}{20 + 30 + 5 + 45} = 0,65$$

$$err(f) = \frac{FP + FN}{n} = \frac{5 + 30}{20 + 30 + 5 + 45} = 0,35$$

Classificador B

		Classe predita	
		+	-
Classe Verdadeira	+	45	5
	-	35	15

$$acc(f) = \frac{VP + VN}{n} = \frac{15 + 45}{20 + 30 + 5 + 45} = 0,60$$

$$err(f) = \frac{FP + FN}{n} = \frac{5 + 35}{20 + 30 + 5 + 45} = 0,40$$

Avaliação

Métricas: Exemplo

Classificador A

		Classe predita	
		+	-
Classe Verdadeira	+	20	30
	-	5	45

$$acc(f) = \frac{VP + VN}{n} = \frac{20 + 45}{20 + 30 + 5 + 45} = 0,65$$

$$err(f) = \frac{FP + FN}{n} = \frac{5 + 30}{20 + 30 + 5 + 45} = 0,35$$

Classificador B

		Classe predita	
		+	-
Classe Verdadeira	+	45	5
	-	35	15

$$acc(f) = \frac{VP + VN}{n} = \frac{15 + 45}{20 + 30 + 5 + 45} = 0,60$$

$$err(f) = \frac{FP + FN}{n} = \frac{5 + 35}{20 + 30 + 5 + 45} = 0,40$$

- Qual classificador é melhor?

Avaliação

Métricas: Exemplo

Classificador A

		Classe predita	
		+	-
Classe Verdadeira	+	20	30
	-	5	45

$$acc(f) = \frac{VP + VN}{n} = \frac{20 + 45}{20 + 30 + 5 + 45} = 0,65$$

$$err(f) = \frac{FP + FN}{n} = \frac{5 + 30}{20 + 30 + 5 + 45} = 0,35$$

Classificador B

		Classe predita	
		+	-
Classe Verdadeira	+	45	5
	-	35	15

$$acc(f) = \frac{VP + VN}{n} = \frac{15 + 45}{20 + 30 + 5 + 45} = 0,60$$

$$err(f) = \frac{FP + FN}{n} = \frac{5 + 35}{20 + 30 + 5 + 45} = 0,40$$

- Qual classificador é melhor?
 - De acordo com a *acc* e *err*, o Classificador A é melhor.

Classificador A

Classe verdadeira

Classe predita

+

-

Classe Verdadeira

+

-

20

5

Classificador B

Classe predita

+

-

45

5

35

15



$$acc(f) = \frac{VP + VN}{n} = \frac{20 + 45}{20 + 30 + 5 + 45}$$

$$err(f) = \frac{FP + FN}{n} = \frac{5 + 30}{20 + 30 + 5 + 45}$$

$$acc(f) = \frac{15 + 45}{20 + 30 + 5 + 45} = 0,60$$

$$err(f) = \frac{5 + 35}{20 + 30 + 5 + 45} = 0,40$$

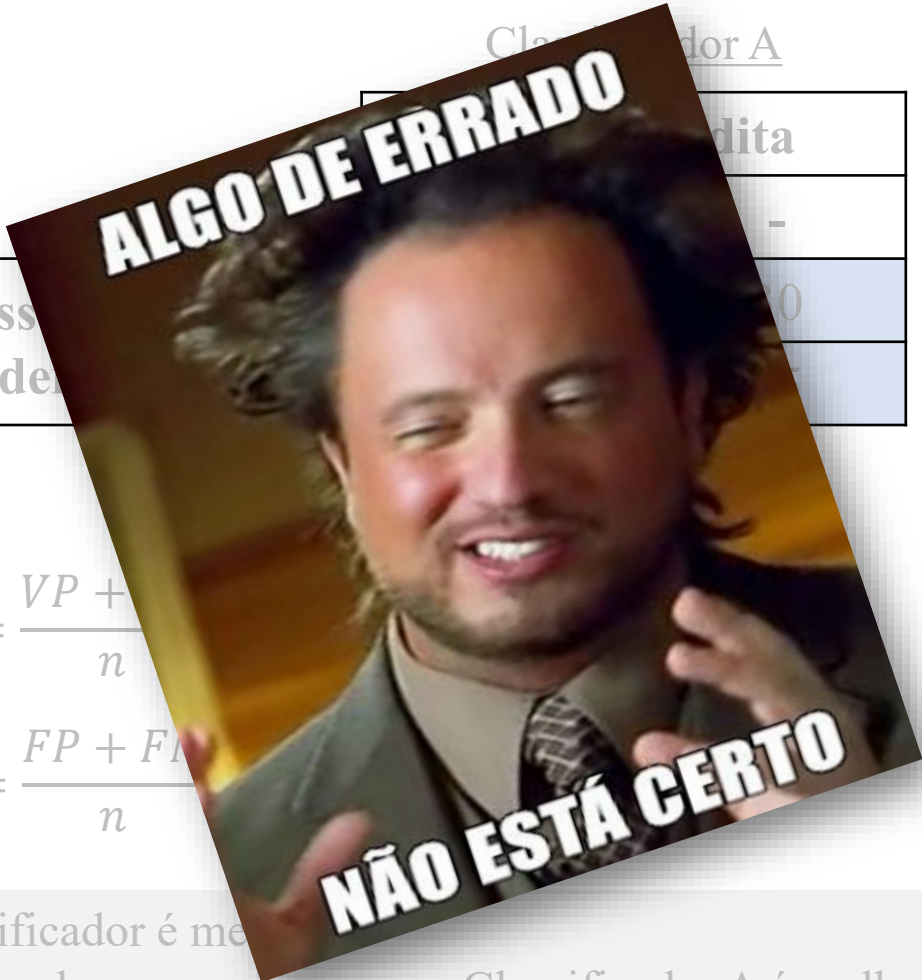
- Qual classificador é melhor?
 - De acordo com a *acc* e *err*, o Classificador A é melhor.

Avaliação

Métricas: Exemplo

Classificador A

Classe Verdadeira	Classe predita
0	0
1	1



$$acc(f) = \frac{VP + VN}{n}$$

$$err(f) = \frac{FP + FN}{n}$$

Classificador B

Classe Verdadeira	Classe predita
0	0
1	1



$$acc(f) = \frac{VP + VN}{n}$$

$$err(f) = \frac{FP + FN}{n} = \frac{5 + 35}{20 + 30 + 5 + 45} = 0,40$$

- Qual classificador é melhor?
 - De acordo com a *acc* e *err*, o Classificador A é melhor.

Precisão, Revocação e F1-Score

Avaliação

Métricas: Precisão e Revocação

- **Precisão:**
 - Dentre os preditos como +, proporção de corretos.
 - Entre todos os exemplos que o modelo previu como positivos, quantos realmente são positivos?
 - **Serve para: Evitar falsos alarmes.**
 - Útil quando erros de falsos positivos são custosos (ex: indústria 4.0, na qual a curadoria de equipamento tem alto custo).
- **Revocação:**
 - Taxa de acerto na classe +.
 - Entre todos os exemplos que realmente são positivos, quantos o modelo conseguiu encontrar?
 - **Serve para: Evitar deixar casos positivos passarem.**
 - Útil quando erros de falsos negativos são graves (ex: identificar doenças).

		Classe predita	
		+	-
Classe Verdadeira	+	VP	FN
	-	FP	VN

$prec(f)$ (vertical arrow pointing to VP)

$revoc(f)$ (horizontal arrow pointing to FN)

$$prec(f) = \frac{VP}{VP + FP}$$

$$revoc(f) = \frac{VP}{VP + FN}$$

- ***F1-score***:
 - Média harmônica entre precisão e revocação.

$$F_1(f) = 2 \times \frac{prec(f) \times revoc(f)}{prec(f) + revoc(f)}$$

LORENA, Ana Carolina; GAMA, João; FACELI, Katti. Inteligência Artificial: Uma abordagem de aprendizado de máquina. Grupo Gen-LTC, 2011.

Avaliação

Métricas: Exemplo

- Dada uma amostra de 20 fotos, 10 de macacos e 10 de sapos, onde os macacos pertencem à classe 0 e os sapos pertencem à classe 1:
 - Dados reais = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1].
- Suponha que um classificador que distingue entre macacos e sapos e gatos seja treinado e obtenha o seguinte resultado:
 - Predição = [0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1].
- Com esses dois conjuntos rotulados (reais e previsões), podemos criar uma matriz de confusão que resumirá os resultados do teste do classificador:

Real	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1			
Predito	0	1	0	1	0	0	0	0	0	0	1	1	0	1	1	0	1	0	1	1	Classe predita	
																					Macaco	Sapo

LORENA, Ana Carolina; GAMA, João; FACELI, Katti. Inteligência Artificial: Uma abordagem de aprendizado de máquina. Grupo Gen-LTC, 2011.

Avaliação

Métricas: Exemplo

- Dada uma amostra de 20 fotos, 10 de macacos e 10 de sapos, onde os macacos pertencem à classe 0 e os sapos pertencem à classe 1:
 - Dados reais = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1].
- Suponha que um classificador que distingue entre macacos e sapos e gatos seja treinado e obtenha o seguinte resultado:
 - Predição = [0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1].
- Com esses dois conjuntos rotulados (reais e previsões), podemos criar uma matriz de confusão que resumirá os resultados do teste do classificador:

Real	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
Predito	0	1	0	1	0	0	0	0	0	0	1	1	0	1	1	0	1	0	1

		Classe predita	
		Macaco	Sapo
Classe Verdadeira	Macaco	8	2
	Sapo	3	7

LORENA, Ana Carolina; GAMA, João; FACELI, Katti. Inteligência Artificial: Uma abordagem de aprendizado de máquina. Grupo Gen-LTC, 2011.

- Agora, podemos computar as medidas de qualidade.

$$acc(f) = \frac{VP + VN}{n} = ?$$

$$err(f) = \frac{FP + FN}{n} = ?$$

$$prec(f) = \frac{VP}{VP + FP} = ?$$

$$revoc(f) = \frac{VP}{VP + FN} = ?$$

$$F_1(f) = 2 \times \frac{prec(f) \times revoc(f)}{prec(f) + revoc(f)} = ?$$

		Classe predita	
		Macaco	Sapo
Classe Verdadeira	Macaco	8	2
	Sapo	3	7

- Agora, podemos computar as medidas de qualidade.

$$acc(f) = \frac{VP + VN}{n} = \frac{8 + 7}{8 + 7 + 2 + 3} = 0,75$$

$$err(f) = \frac{FP + FN}{n} = \frac{2 + 3}{8 + 7 + 2 + 3} = 0,25$$

$$prec(f) = \frac{VP}{VP + FP} = \frac{8}{8 + 3} = 0,73$$

$$revoc(f) = \frac{VP}{VP + FN} = \frac{8}{8 + 2} = 0,80$$

$$F_1(f) = 2 \times \frac{prec(f) \times revoc(f)}{prec(f) + revoc(f)} = 2 \times \frac{0,73 \times 0,80}{0,73 + 0,80} = 0,76$$

		Classe predita	
		Macaco	Sapo
Classe Verdadeira	Macaco	8	2
	Sapo	3	7

Avaliação

Métricas: Exemplo

- Vamos criar um exemplo ilustrativo para dois classificadores que avaliam pacientes com e sem Câncer.
- Teremos:
 - Classe Positiva (P): Pacientes com Câncer;
 - Classe Negativa (N): Pacientes sem Câncer.
- Vamos simular os dados para dois classificadores:

		<u>Classificador A</u>	
		Classe predita	
		+	-
Classe Verdadeira	+	20	30
	-	5	45

		<u>Classificador B</u>	
		Classe predita	
		+	-
Classe Verdadeira	+	45	5
	-	35	15

LORENA, Ana Carolina; GAMA, João; FACELI, Katti. Inteligência Artificial: Uma abordagem de aprendizado de máquina. Grupo Gen-LTC, 2011.

Avaliação

Métricas: Exemplo

Classificador A

		Classe predita	
		+	-
Classe Verdadeira	+	20	30
	-	5	45

$$prec(f) = \frac{VP}{VP + FP} = \frac{20}{20 + 5} = 0,8$$

$$revoc(f) = \frac{VP}{VP + FN} = \frac{20}{20 + 30} = 0,4$$

$$F_1(f) = 2 \times \frac{prec(f) \times revoc(f)}{prec(f) + revoc(f)} = 2 \times \frac{0,32}{1,20} = 0,53$$

Classificador B

		Classe predita	
		+	-
Classe Verdadeira	+	45	5
	-	35	15

$$prec(f) = \frac{VP}{VP + FP} = \frac{45}{45 + 35} = 0,5625$$

$$revoc(f) = \frac{VP}{VP + FN} = \frac{45}{45 + 5} = 0,90$$

$$F_1(f) = 2 \times \frac{prec(f) \times revoc(f)}{prec(f) + revoc(f)} = 2 \times \frac{0,50625}{1,4625} = 0,692$$

- Interpretação:
 - **Classificador A** é mais **conservador**: só diagnostica quando tem certeza → menos falsos positivos, mas perde muitos pacientes com câncer (FN).
 - **Classificador B** é mais **sensível**: identifica quase todos com câncer → poucos FN, mas muitos falsos positivos (FP).
- Em um cenário médico, o **Classificador B** pode ser **mais útil**, pois é mais arriscado perder um paciente com câncer (FN) do que gerar alarmes falsos.

Métrica	Classificador A	Classificador B
Precisão	0.800	0.5625
Revocação	0.400	0.9000
F1-score	0.533	0.6920

- Vamos analisar mais profundamente o caso da detecção de spam, comparando precisão e revocação nesse contexto.
- Problema: Detecção de *Spam*: Você tem um sistema que analisa e-mails e decide se cada um deles é *spam* ou *não-spam*:
 1. **Falso Positivo (FP):**
 - Um e-mail **legítimo** é classificado como *spam*;
 - Isso é ruim porque o usuário pode **nunca ver** um e-mail importante.
 2. **Falso Negativo (FN):**
 - Um e-mail *spam* é classificado como *não-spam*;
 - Isso é irritante, mas o usuário ainda pode **ver e apagar** esse e-mail.

- Comparando **Precisão** e **Revocação** nesse caso:
 - Alta Precisão:
 - Significa que, **quando o sistema diz que é *spam*, provavelmente é mesmo;**
 - Ou seja, a caixa de spam não terá quase nenhum e-mail legítimo;
 - **Bom para evitar falsos positivos.**
 - Baixa Precisão:
 - A caixa de *spam* pode conter muitos e-mails legítimos;
 - O usuário pode perder mensagens importantes.
 - Alta Revocação:
 - Significa que o sistema consegue detectar **quase todos os *spams*;**
 - Pouco spam passa para a caixa de entrada;
 - **Bom para evitar falsos negativos.**
 - Baixa Revocação:
 - Alguns *spams* chegam na caixa de entrada, o que pode incomodar.

- Por que preferimos alta precisão nesse caso?
 - Porque, em geral:
 - **O custo de perder um e-mail importante (falso positivo) é maior** do que o de ver um *spam* na caixa de entrada (falso negativo);
 - Usuários preferem apagar um *spam* manualmente do que **perder uma proposta de emprego, uma resposta de um cliente ou um e-mail da universidade.**

- Por que preferimos alta precisão nesse caso?
 - Porque, em geral:
 - **O custo de perder um e-mail importante (falso positivo) é maior** do que o de ver um spam na caixa de entrada (falso negativo);
 - Usuários preferem apagar um spam manualmente do que **perder uma proposta de emprego, uma resposta de um cliente ou um e-mail da universidade.**
- Conclusão:
 - Na maioria dos cenários de detecção de spam, **alta precisão** é preferida porque:
 - Os **falsos positivos são mais prejudiciais** (você pode perder algo importante).
 - Os **falsos negativos são menos graves** (você apenas deleta um spam manualmente).

- Vamos fazer uma análise detalhada, nos mesmos moldes, para **um exemplo em que a revocação é mais importante** que a precisão.
- **Problema: Detecção de câncer em exames médicos (triagem inicial):**
 - Imagine um sistema automático que analisa exames médicos (como mamografias) para identificar **possíveis casos de câncer**.
 - Objetivo:
 - Identificar todos os **pacientes com câncer**, mesmo que isso signifique errar em alguns diagnósticos.
 - Erros possíveis:
 - **Falso Negativo (FN):**
 - Um paciente com câncer é classificado como saudável;
 - Muito grave: o paciente **não recebe tratamento** a tempo, o que pode levar à progressão da doença ou até à morte
 - **Falso Positivo (FP):**
 - Um paciente saudável é classificado como suspeito de câncer;
 - Menos grave: o paciente **faz exames adicionais**, talvez se preocupe à toa, mas o erro pode ser corrigido mais adiante.

- Comparando **Precisão** e **Revocação** nesse caso:
 - Alta Revocação:
 - Quase todos os pacientes com câncer são detectados;
 - Ótimo para **não deixar nenhum caso passar**.
 - Baixa Revocação:
 - Muitos casos reais não são detectados;
 - Isso pode **custar vidas**.
 - Alta Precisão:
 - A maioria dos pacientes classificados como doentes realmente tem câncer;
 - Menos alarmes falsos, menos exames desnecessários.
 - Baixa Precisão:
 - Muitos pacientes sem câncer são classificados como suspeitos;
 - Isso gera **falsos alarmes**, mas pode ser controlado com exames complementares.

- Por que preferimos alta revocação nesse caso?
 - **O custo de um falso negativo é altíssimo:** risco à vida;
 - **O custo de um falso positivo é aceitável:** mais exames e preocupações, mas **não impede o tratamento de quem precisa.**
 - No estágio de **triagem inicial**, o sistema não precisa dar o diagnóstico final, apenas alertar: “esse exame pode ter algo suspeito”.
- Conclusão:
 - Na triagem de câncer, **alta revocação é mais importante** porque:
 - Precisamos **garantir que ninguém com câncer passe despercebido;**
 - É melhor investigar alguns pacientes saudáveis do que **ignorar um caso verdadeiro.**

Avaliação

Métricas: Exemplo

Aspecto	Detecção de Spam	Triagem de Câncer	Reconhecimento Facial em Acesso Restrito
Objetivo do sistema	Identificar e-mails indesejados	Detectar possíveis casos de câncer	Permitir acesso apenas a pessoas autorizadas
Falso Positivo (FP)	E-mail legítimo marcado como spam	Pessoa saudável marcada como doente	Pessoa não autorizada recebe acesso
Falso Negativo (FN)	Spam passa como e-mail normal	Pessoa com câncer não é detectada	Pessoa autorizada é barrada indevidamente
Impacto do Falso Positivo	Perda de e-mail importante	Exames extras e preocupação desnecessária	Risco de segurança (invasão)
Impacto do Falso Negativo	Spam visível, mas pode ser apagado	Doença avança sem tratamento	Transtorno ao barrar alguém legítimo
Erro mais grave	Falso positivo	Falso negativo	Ambos são sérios
Métrica mais importante	Precisão	Revocação	Ambas: equilíbrio (F1-score)
Justificativa	Evitar bloquear e-mails bons	Capturar todos os possíveis casos	Sistema deve ser seguro e justo
Aceita falsos positivos?	Poucos	Sim, desde que não perca casos reais	Não (não pode deixar estranhos entrarem)
Aceita falsos negativos?	Sim, se não forem muitos	Não, risco à vida	Não (não pode impedir acesso legítimo)
Situação	Métrica Prioritária	Por quê?	
Detecção de Spam	Precisão	Evitar perder e-mails importantes.	
Triagem de Câncer	Revocação	Evitar deixar pacientes doentes sem diagnóstico.	
Controle de Acesso com Reconhecimento	F1-score (Equilíbrio)	Precisão e revocação são igualmente críticas.	

- Precisão, revocação e *F1-score* são medidas calculadas por classe, ou seja, elas avaliam o desempenho do modelo em identificar corretamente uma classe específica.
- Cada métrica é calculada em relação a uma classe de interesse.
- Por exemplo, o conjunto de dados Iris possui três classes: Setosa, Virginica e Versicolor:
 - Se você estiver analisando a classe Setosa como a positiva, tudo é interpretado do ponto de vista de Setosa:
 - VP = acertos como Setosa;
 - FP = outras classes que o modelo classificou erroneamente como Setosa;
 - FN = Setosas que o modelo não reconheceu;
 - Note que para essas medidas nem olhamos o VN.
 - Se quiser avaliar o desempenho para a classe Versicolor, você repete o processo considerando essa classe como a nova positiva.

Avaliação

Métricas: Macro e Ponderada ou *Weighted*

- Para avaliar o desempenho global, existem duas abordagens comuns:
 1. Macro média (*macro average*):
 - Calcula a métrica separadamente para cada classe;
 - Depois tira a média simples;
 - Serve para dar peso igual a todas as classes, mesmo que elas tenham tamanhos diferentes.

$$P_{macro} = \frac{P_1 + P_2}{2}$$

2. Média ponderada (*weighted average*):
 - Também calcula as métricas separadamente para cada classe;
 - Mas a média final é ponderada pelo número de amostras em cada classe;
 - Serve para quando você quer levar em conta o desequilíbrio entre classes.

$$P_{weighted} = \frac{n_1 P_1 + n_2 P_2}{n_1 + n_2}$$

Avaliação

Métricas: Macro e Ponderada ou *Weighted*

- Exemplo com duas classes.
- Suponha:
 - Classe A (80 amostras);
 - Classe B (20 amostras).
- O modelo tem os seguintes resultados:

Classe	Precisão	Revocação	F1-score
A	0.90	0.85	0.875
B	0.60	0.50	0.545

Macro média do F1-score:

$$f_{1_{macro}} = \frac{0,875 + 0,545}{2} = 0,71$$

Weighted média do F1-score:

$$f_{1_{weighted}} = \frac{(80 \times 0,875) + (20 \times 0,545)}{100} = 0,829$$

- Resumo:
 - Sim, você **calcula as métricas** para cada classe **separadamente**;
 - Depois, escolhe como agregá-las: média simples (macro), ponderada (*weighted*), ou até usa só a da classe principal, se for o caso;
 - O F1-score macro é útil quando todas as classes são igualmente importantes, e o *weighted* é melhor se as classes estiverem desbalanceadas.

Avaliação

Métricas: Macro e Ponderada ou *Weighted*

- Vamos montar um exemplo passo a passo com uma matriz de confusão real para duas classes: Positiva (P) e Negativa (N).

		Classe predita	
		+	-
Classe Verdadeira	+	40	10
	-	20	30

- Métricas para a classe Positiva (P):
 - VP = 40, FP = 20, FN = 10.

$$P_+ = \frac{VP}{VP + FP} = \frac{40}{40 + 20} = 0,6667$$

$$R_+ = \frac{VP}{VP + FN} = \frac{10}{40 + 10} = 0,8$$

$$F_{1+} = 2 \times \frac{prec(f) \times revoc(f)}{prec(f) + revoc(f)} = 2 \times \frac{0,533}{1,4667} = 0,727$$

- Métricas para a classe Negativa (N):
 - VP = 30, FP = 10, FN = 20.

$$P_- = \frac{VP}{VP + FP} = \frac{30}{30 + 10} = 0,75$$

$$R_- = \frac{VP}{VP + FN} = \frac{30}{30 + 20} = 0,6$$

$$F_{1-} = 2 \times \frac{prec(f) \times revoc(f)}{prec(f) + revoc(f)} = 2 \times \frac{0,45}{1,35} = 0,667$$

- Médias finais Macro:
 - Precisão: $(0.6667 + 0.75) / 2 = 0.708$
 - Revocação: $(0.8 + 0.6) / 2 = 0.7$
 - F1-score: $(0.727 + 0.667) / 2 = 0.697$
- *Weighted* (baseado em tamanho das classes):
 - Total real da classe P: 50;
 - Total real da classe N: 50;
 - Total = 100;
 - Como os tamanhos são iguais, média ponderada = macro média, neste exemplo.

Avaliação

K-fold cross validation e a matriz de confusão

- Mas como avaliamos isso quando utilizamos o *K-fold cross validation*?
- Para avaliar as métricas:
 - **Basta fazer o processo de avaliação das métrica em cada *fold* e ao final tirar a média;**
 - As métricas são calculadas em cada *fold* separadamente;
 - Depois disso:
 - **Fazemos a média das métricas.**
- Para avaliar visualmente as matrizes:
 - Agora é diferente:
 - Para cada *fold* construímos a matriz de confusão e ao final geramos uma matriz de confusão que é **composta pela soma das outras matrizes de confusão;**
 - Essa matriz é chamada de **matriz de confusão agregada ou global.**

Avaliação

K-fold cross validation estratificado

- Vamos usar $k=3$ e estratificado.
- Total:
 - Sim = 6 amostras (60%);
 - Não = 4 amostras (40%).

Paciente	Tosse	Febre	Dor de Cabeça	Doente
1	Sim	Não	Sim	Sim
2	Não	Não	Sim	Não
3	Sim	Não	Não	Não
4	Sim	Sim	Sim	Sim
5	Não	Sim	Sim	Sim
6	Sim	Não	Não	Sim
7	Não	Não	Não	Não
8	Não	Sim	Não	Sim
9	Não	Não	Não	Não
10	Sim	Sim	Sim	Sim

Fold 1

Fold 2

Fold 1

Fold 2

Fold 1

Fold 3

Fold 3

Fold 2

Fold 3

Fold 3

- *Fold 1*:
 - Sim = 2 amostras (67%);
 - Não = 1 amostras (33%).
- *Fold 2*:
 - Sim = 2 amostras (67%);
 - Não = 1 amostras (33%).
- *Fold 3*:
 - Sim = 2 amostras (50%);
 - Não = 2 amostras (50%).
- Essa é a melhor distribuição estratificada, uma vez que não é possível gerar 3 *folds* de mesmo tamanho e com exatamente 60% de amostras da classe “Sim” e 40% de amostras da classe “Não”.
- Nesse caso teremos 3 avaliações de treino e teste gerando 3 matrizes de confusão.
- Ao final geramos uma matriz de confusão final contendo o somatório das 3 matrizes.

Avaliação

K-fold cross validation estratificado

- Treino: *Fold 1* e *Fold 2*.
- Teste: *Fold 3*.

		Classe predita	
		+	-
Classe Verdadeira	+	2	0
	-	1	0

- Treino: *Fold 2* e *Fold 3*.
- Teste: *Fold 1*.

		Classe predita	
		+	-
Classe Verdadeira	+	1	1
	-	0	1

- Treino: *Fold 1* e *Fold 3*.
- Teste: *Fold 2*.

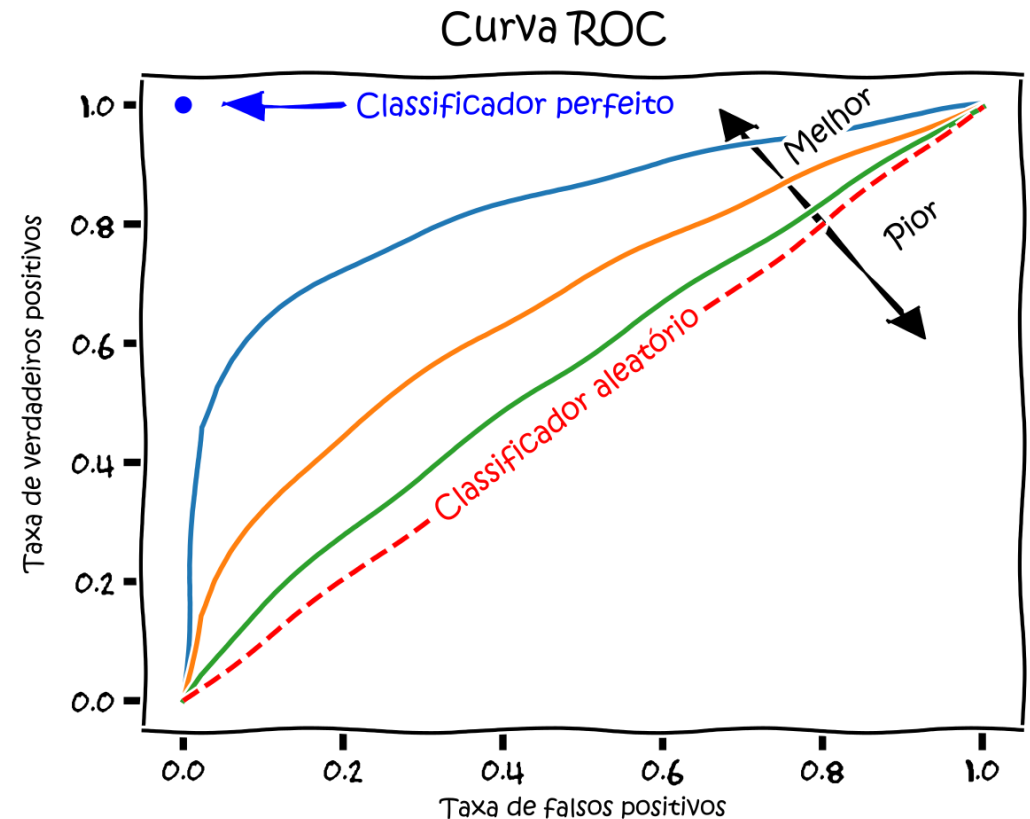
		Classe predita	
		+	-
Classe Verdadeira	+	1	1
	-	0	2

- Matriz global.
- Somatórios das 3 anteriores.

		Classe predita	
		+	-
Classe Verdadeira	+	4	2
	-	1	3

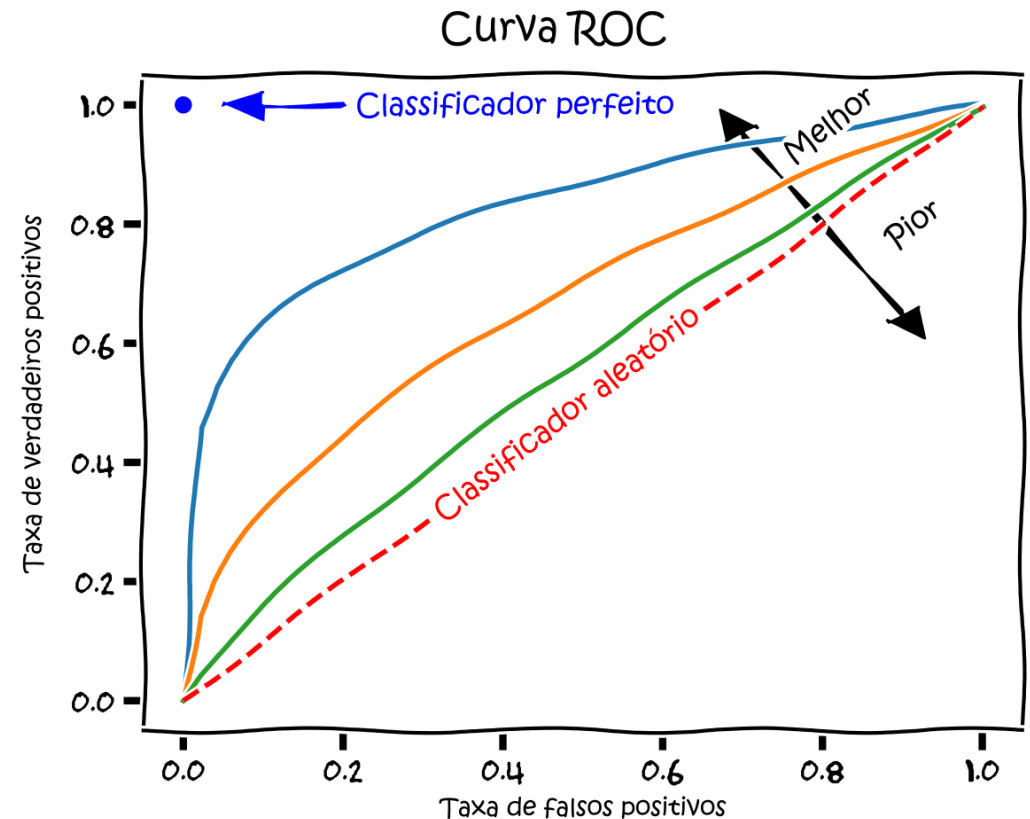
Curva ROC-AUC

- AUC-ROC é um acrônimo para “*Area Under the Curve of the Receiver Characteristic Operator*” e é uma das métricas mais importantes para avaliação de modelos de classificação.
- Através dela, podemos especular sobre a performance do modelo em relação ao seu grau de separabilidade, isso é, o quão bem o modelo é capaz de distinguir as classes.
- Mostra relação entre custo (TFP) e benefício (TVP).
- No eixo X (horizontal), temos a taxa de falsos positivos.
- No eixo Y (vertical), temos a taxa de verdadeiros positivos.



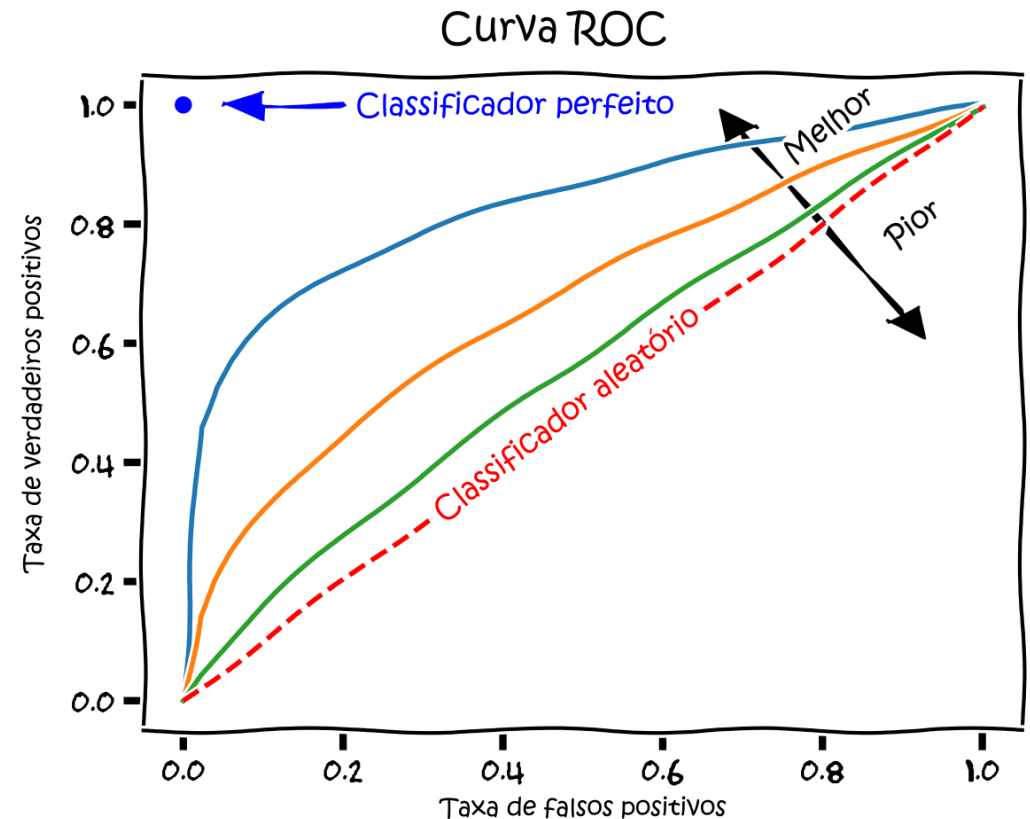
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Oper%C3%A7%C3%A3o_do_Receptor

- AUC-ROC é um acrônimo para “*Area Under the Curve of the Receiver Characteristic Operator*” e é uma das métricas mais importantes para avaliação de modelos de classificação.
- Através dela, podemos especular sobre a performance do modelo em relação ao seu grau de separabilidade, isso é, o quão bem o modelo é capaz de distinguir as classes.
- Mostra relação entre custo (TFP) e benefício (TVP).
- No eixo X (horizontal), temos a taxa de falsos positivos.
- No eixo Y (vertical), temos a taxa de verdadeiros positivos.



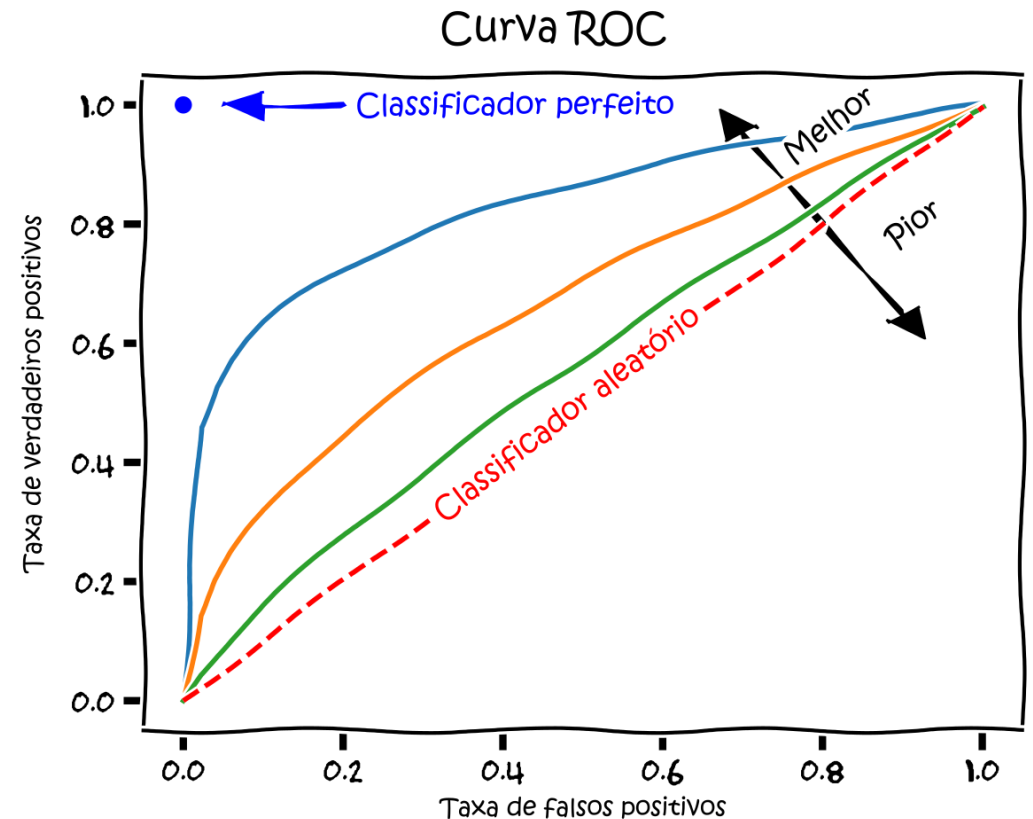
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- AUC-ROC é um acrônimo para “*Area Under the Curve of the Receiver Characteristic Operator*” e é uma das métricas mais importantes para avaliação de modelos de classificação.
- Através dela, podemos especular sobre a performance do modelo em relação ao seu grau de separabilidade, isso é, o quão bem o modelo é capaz de distinguir as classes.
- Mostra relação entre custo (TFP) e benefício (TVP).
- No eixo X (horizontal), temos a taxa de falsos positivos.
- No eixo Y (vertical), temos a taxa de verdadeiros positivos.



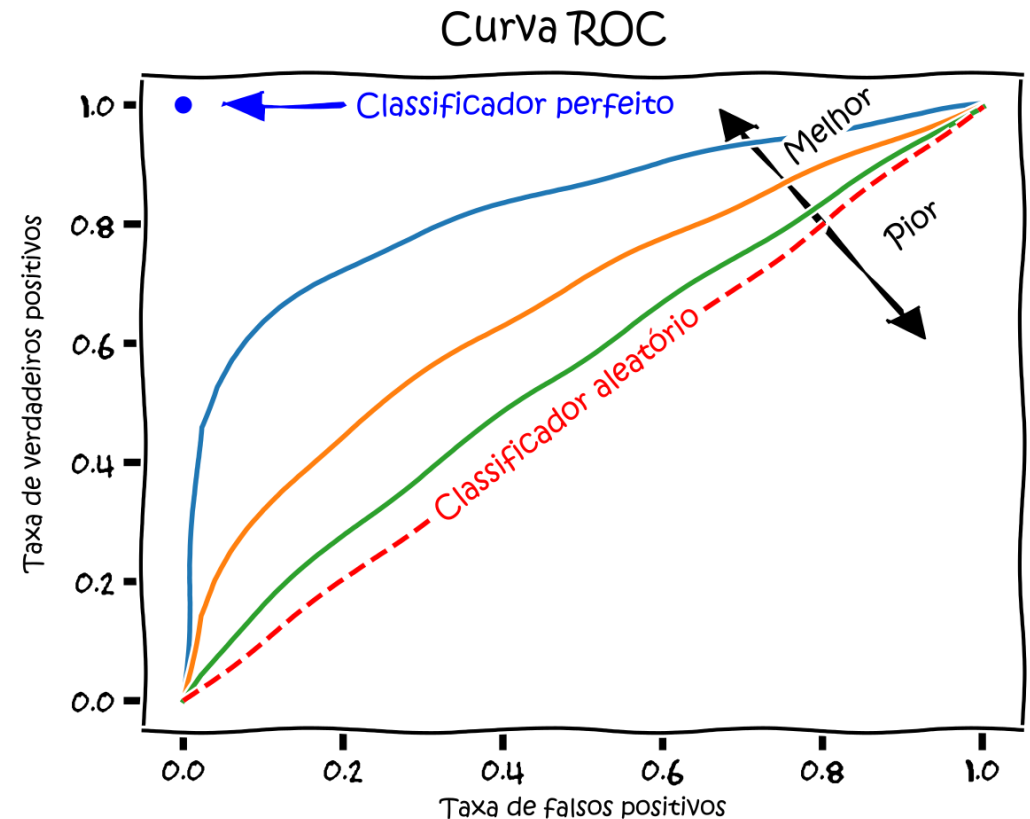
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Oper%C3%A7%C3%A3o_do_Receptor

- AUC-ROC é um acrônimo para “*Area Under the Curve of the Receiver Characteristic Operator*” e é uma das métricas mais importantes para avaliação de modelos de classificação.
- Através dela, podemos especular sobre a performance do modelo em relação ao seu grau de separabilidade, isso é, o quão bem o modelo é capaz de distinguir as classes.
- **Mostra relação entre custo (TFP) e benefício (TVP).**
- No eixo X (horizontal), temos a taxa de falsos positivos.
- No eixo Y (vertical), temos a taxa de verdadeiros positivos.



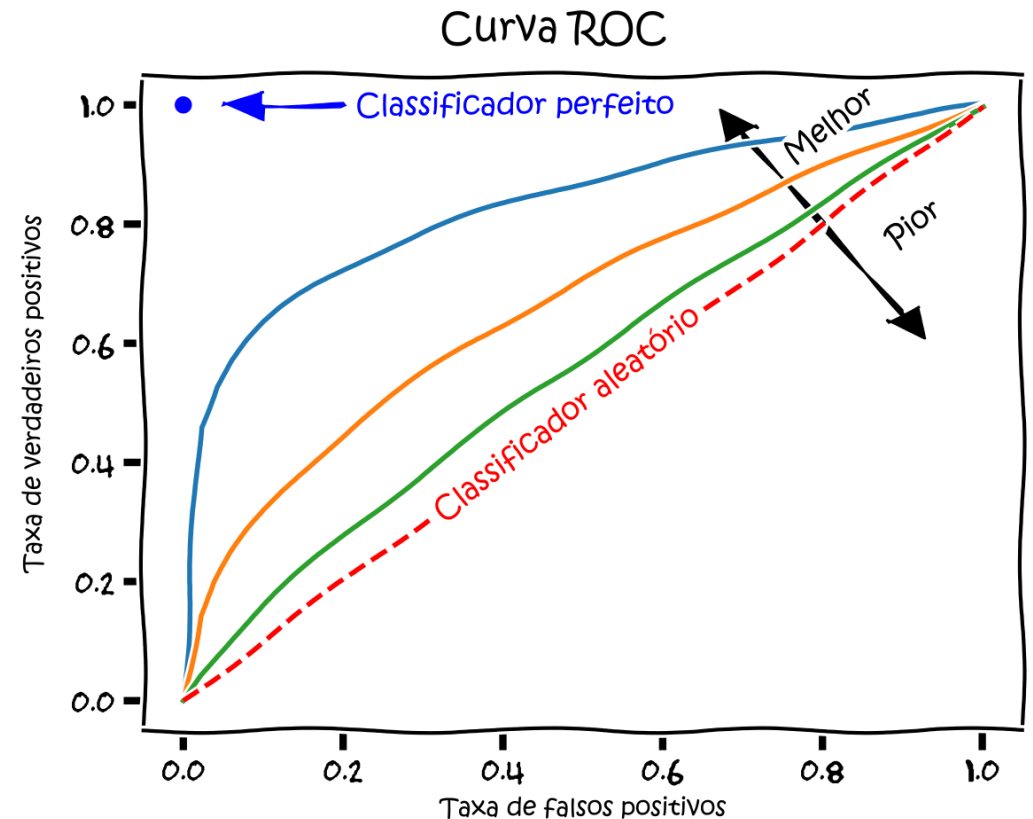
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- AUC-ROC é um acrônimo para “*Area Under the Curve of the Receiver Characteristic Operator*” e é uma das métricas mais importantes para avaliação de modelos de classificação.
- Através dela, podemos especular sobre a performance do modelo em relação ao seu grau de separabilidade, isso é, o quão bem o modelo é capaz de distinguir as classes.
- Mostra relação entre custo (TFP) e benefício (TVP).
- No eixo X (horizontal), temos a taxa de falsos positivos.
- No eixo Y (vertical), temos a taxa de verdadeiros positivos.



https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- AUC-ROC é um acrônimo para “*Area Under the Curve of the Receiver Characteristic Operator*” e é uma das métricas mais importantes para avaliação de modelos de classificação.
- Através dela, podemos especular sobre a performance do modelo em relação ao seu grau de separabilidade, isso é, o quão bem o modelo é capaz de distinguir as classes.
- Mostra relação entre custo (TFP) e benefício (TVP).
- No eixo X (horizontal), temos a taxa de falsos positivos.
- No eixo Y (vertical), temos a taxa de verdadeiros positivos.



https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Oper%C3%A7%C3%A3o_do_Receptor

- **TVP:**
 - Taxa de verdadeiros positivos;
 - Identificados como + corretamente em razão de todos que são +.
- **TFP:**
 - Taxa de falsos positivos;
 - Alarme falso;
 - Identificados como + erroneamente em razão de todos os -.

		Classe predita	
		+	-
Classe Verdadeira	+	VP	FN
	-	FP	VN

→ TVP

→ TFP

$$TVP = \frac{VP}{VP + FN}$$

$$TFP = \frac{FP}{FP + VN}$$

<https://cienciaenegocios.com/curva-roc-e-auc-em-machine-learning/>

Avaliação

Curva AUC-ROC

- **TVP:**
 - Taxa de verdadeiros positivos;
 - Identificados como + corretamente em razão de todos que são +.
- **TFP:**
 - Taxa de falsos positivos;
 - Alarme falso;
 - Identificados como + erroneamente em razão de todos os -.

Ou seja, a curva AUC-ROC é calculada para uma classe de interesse (positiva).

		Classe predita	
		+	-
Classe Verdadeira	+	VP	FN
	-	FP	VN

→ TVP

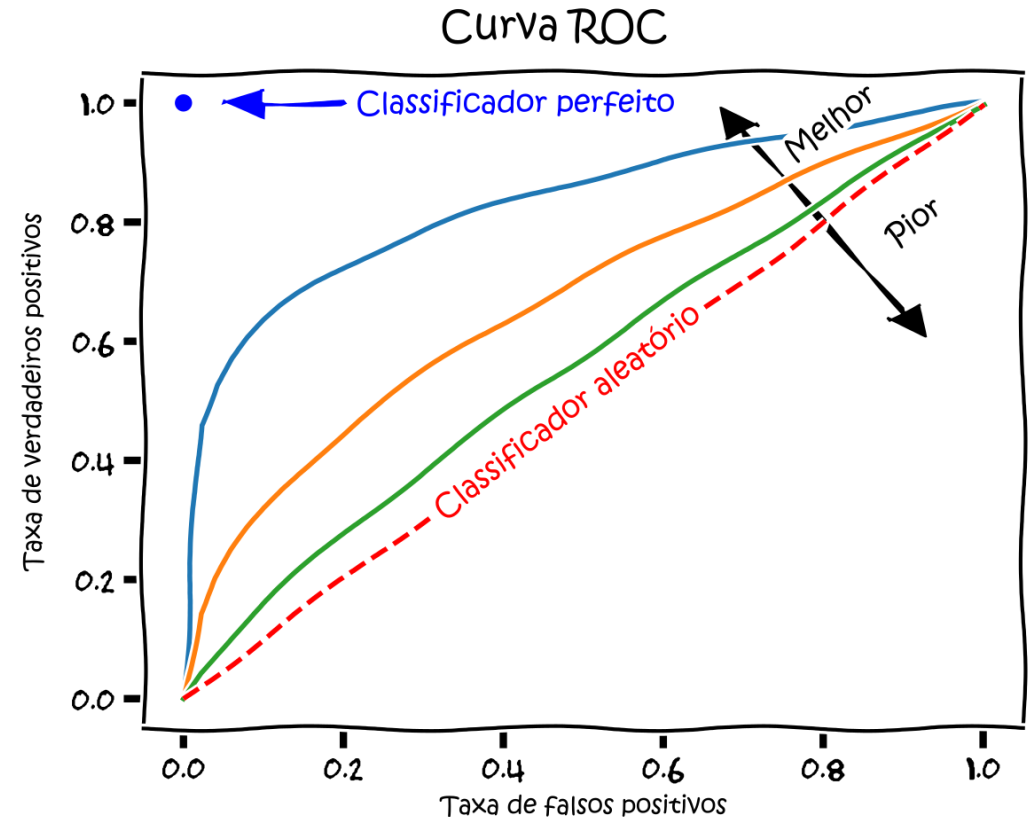
→ TFP

$$TVP = \frac{VP}{VP + FN}$$

$$TFP = \frac{FP}{FP + VN}$$

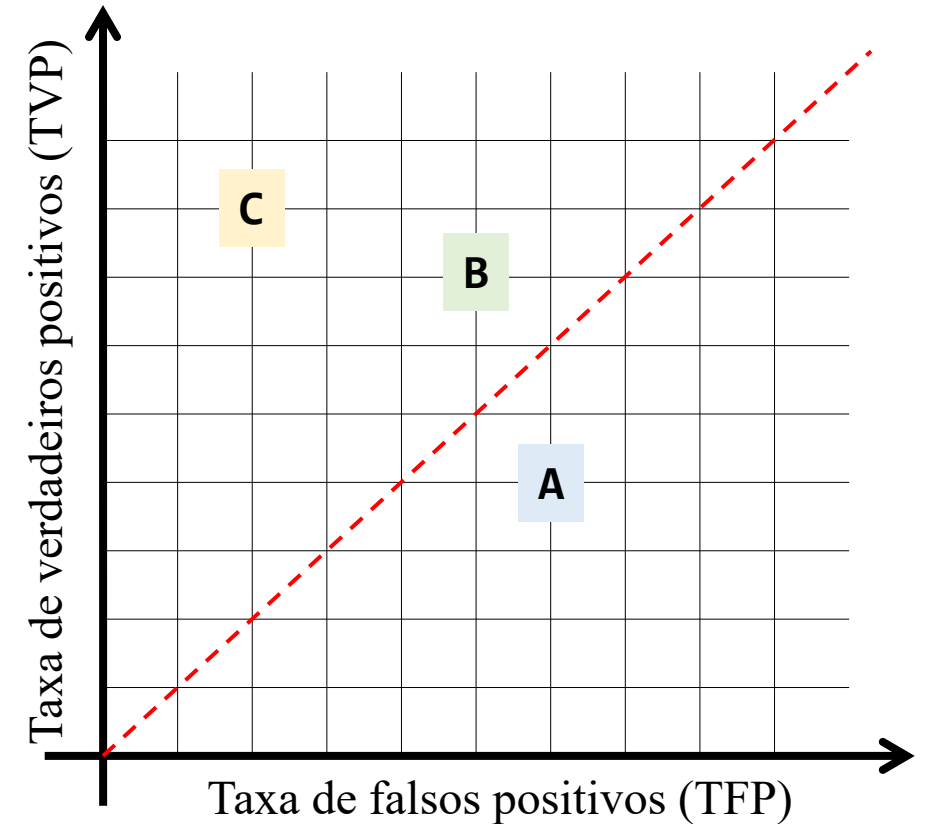
<https://cienciaenegocios.com/curva-roc-e-auc-em-machine-learning/>

- Um AUC-ROC igual a 1 significa que o modelo classifica todos os positivos e negativos corretamente (não há falso positivos nem falso negativos) e igual a 0.5 significa que ele não sabe identificar, isto é, tem a performance de previsões aleatórias. Com AUC ROC de 0 o modelo está invertendo as previsões, classificando 1 como 0 e 0 como 1.



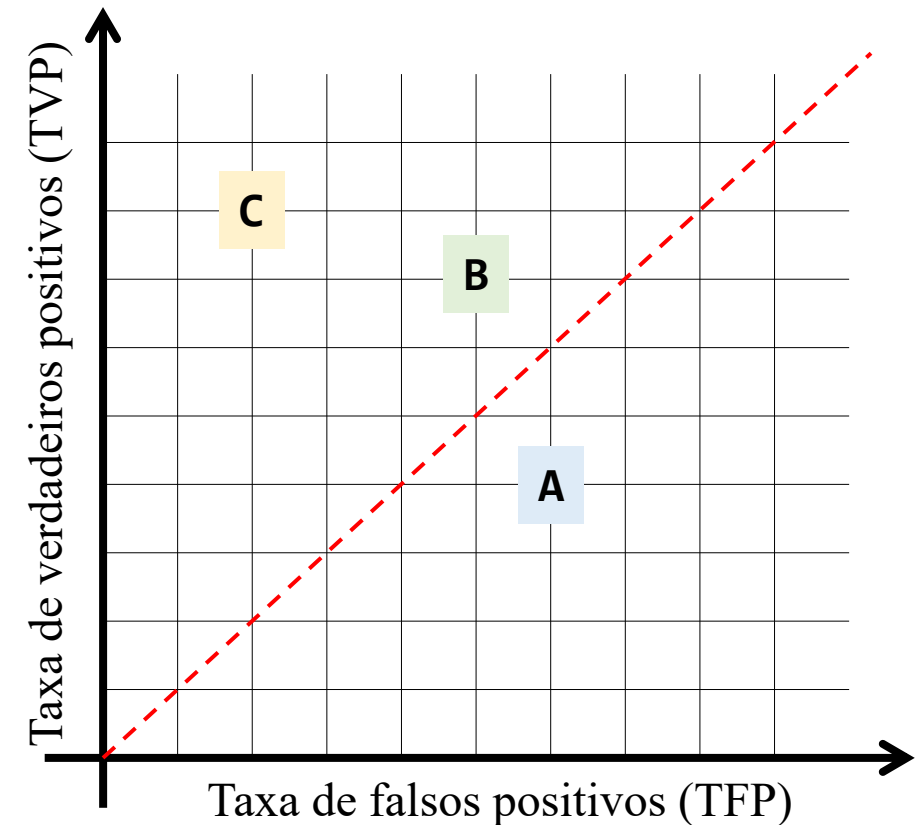
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Oper%C3%A7%C3%A3o_do_Receptor

- Colocar no gráfico ROC os 3 classificadores abaixo:
 - Classificador A:
 - TFP: 0,6;
 - TVP: 0,4.
 - Classificador B:
 - TFP: 0,5;
 - TVP: 0,7.
 - Classificador C:
 - TFP: 0,2;
 - TVP: 0,8.



https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

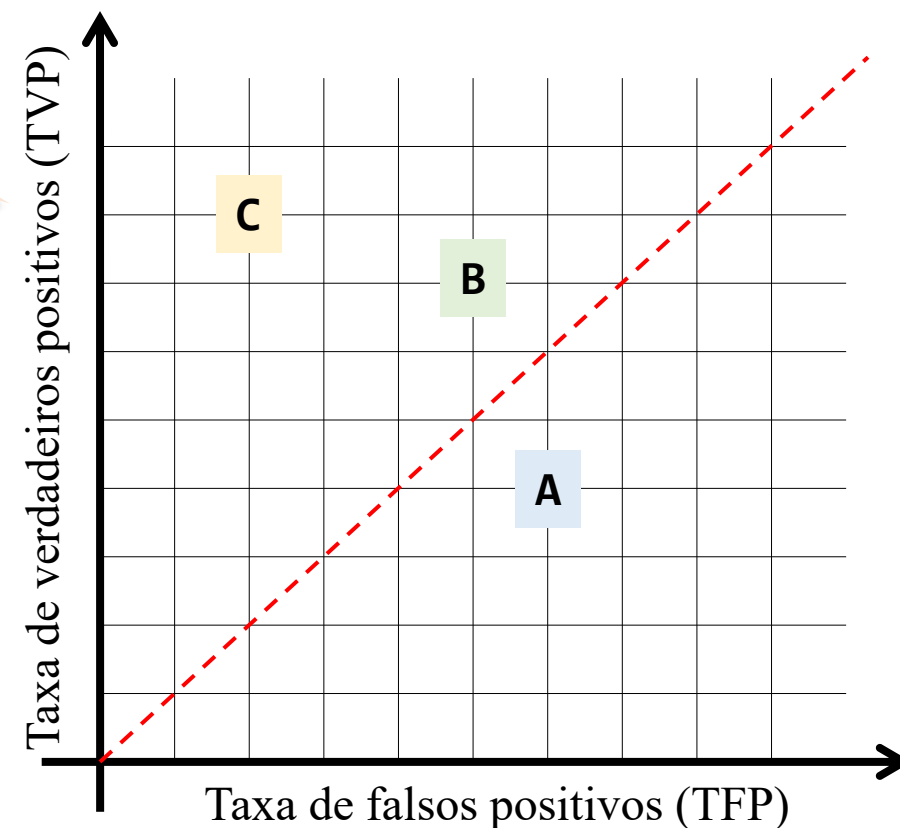
- A maioria dos conjuntos de dados da vida real é **desequilibrada**, em que uma classe é desproporcionalmente maior que a outra, por exemplo, **análise de sentimentos**, **detecção de fraudes e detecção de câncer**. Nesses cenários, métricas como a precisão fornecem uma avaliação enganosa do desempenho do modelo, pois são tendenciosas em relação à classe majoritária. **Nesses cenários, métricas como AUC-ROC são mais confiáveis.**



https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

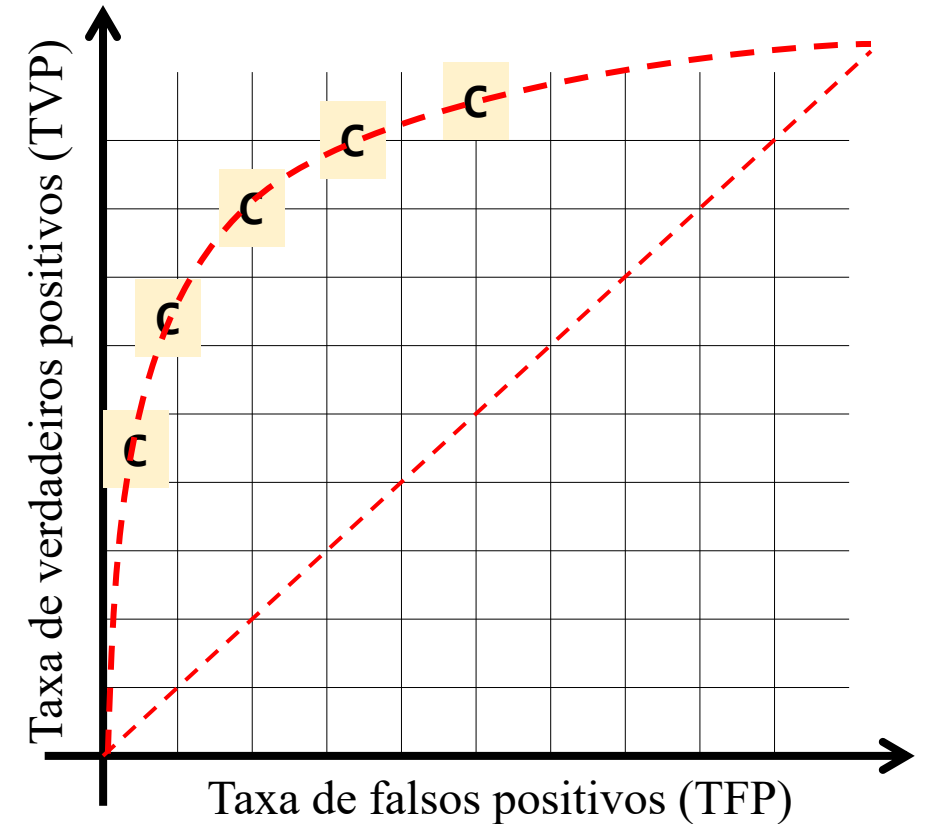
Mas professor, isso não é uma curva é um ponto!

- A maioria dos conjuntos de dados da vida real é **desequilibrada**, em que uma classe é desproporcionalmente maior que a outra, por exemplo, **análise de sentimentos**, **detecção de fraudes e detecção de câncer**. Nesses cenários, métricas como a precisão fornecem uma avaliação enganosa do desempenho do modelo, pois são tendenciosas em relação à classe majoritária. **Nesses cenários, métricas como AUC-ROC são mais confiáveis.**



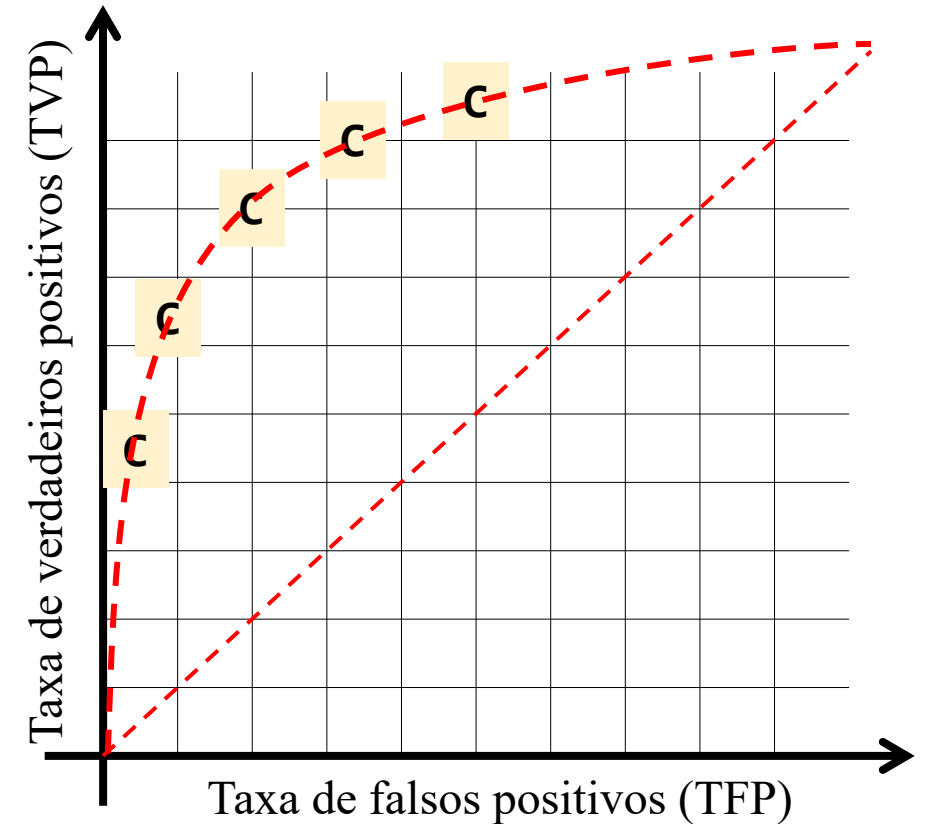
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- Como executar o algoritmo inúmeros vezes?
 - Para criar a curva ROC, você precisa plotar os valores de TVP em relação aos valores de TFP em diferentes limites de decisão.
 - Você pode perguntar: o que significam valores “diferentes” de TVP e TFP?
 - Na verdade, calculamos os valores para uma determinada matriz de confusão em um determinado limite de decisão. Mas para um modelo de classificação probabilística, esses valores de TPR e FPR não são imutáveis.



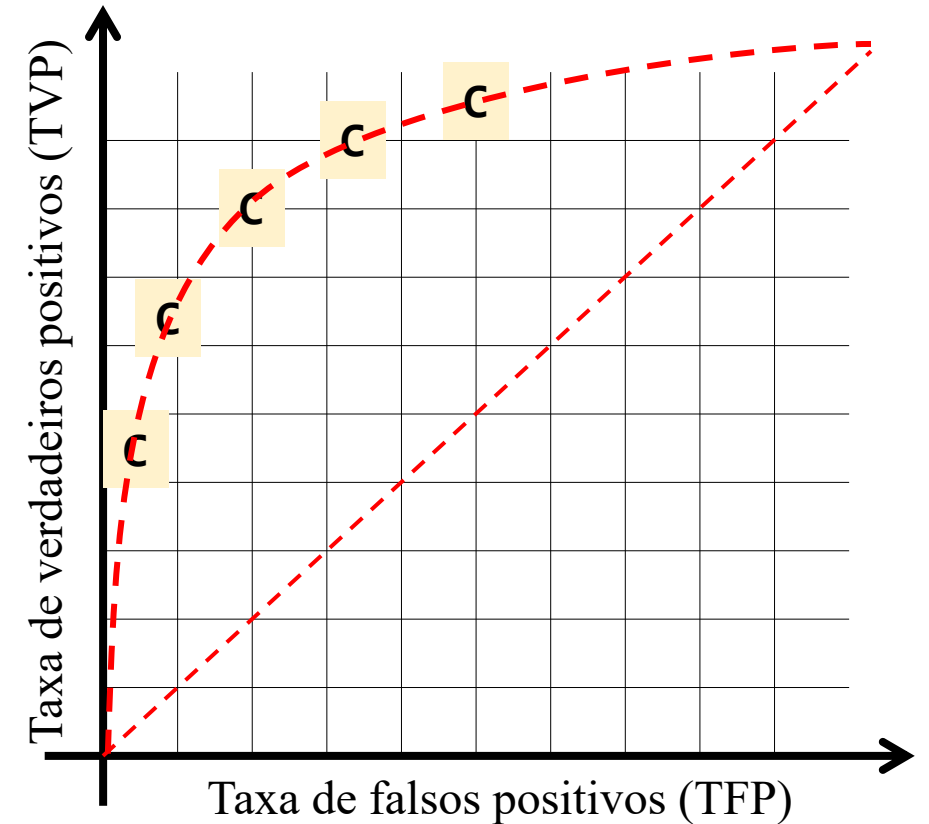
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- Como executar o algoritmo inúmeros vezes?
 - Para criar a curva ROC, você precisa plotar os valores de TVP em relação aos valores de TFP em diferentes limites de decisão.
 - Você pode perguntar: o que significam valores “diferentes” de TVP e TFP?
 - Na verdade, calculamos os valores para uma determinada matriz de confusão em um determinado limite de decisão. Mas para um modelo de classificação probabilística, esses valores de TPR e FPR não são imutáveis.



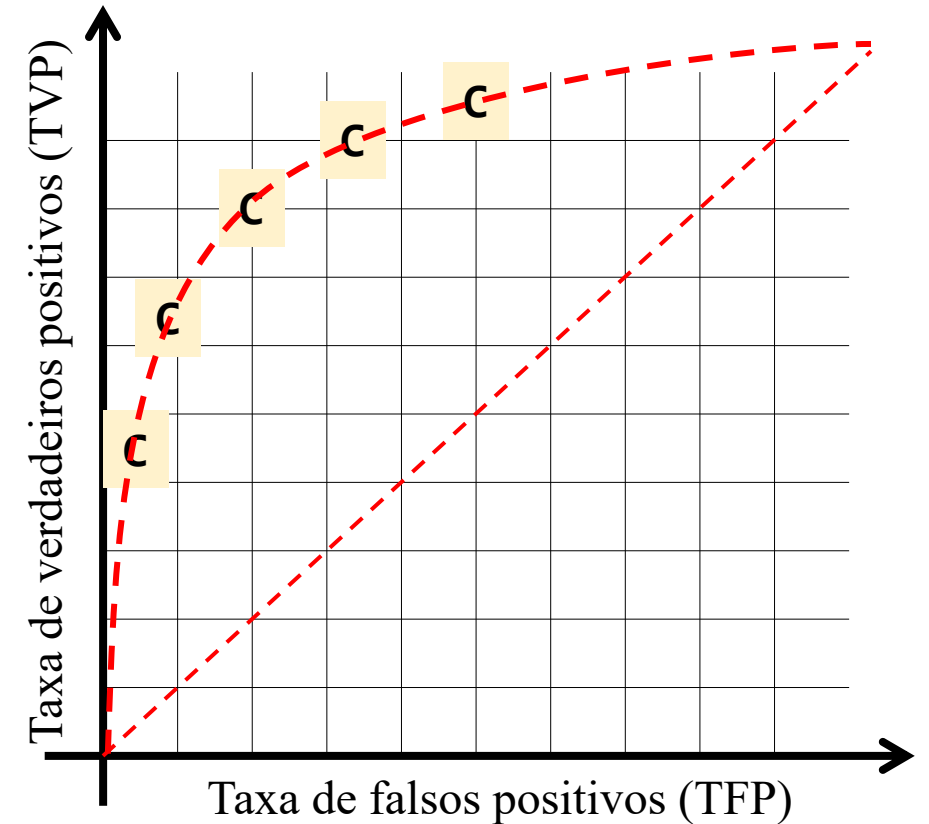
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- Como executar o algoritmo inúmeros vezes?
 - Para criar a curva ROC, você precisa **plotar os valores de TVP em relação aos valores de TFP em diferentes limites de decisão**.
 - Você pode perguntar: o que significam valores “diferentes” de TVP e TFP?
 - Na verdade, calculamos os valores para uma determinada matriz de confusão em um determinado limite de decisão. Mas para um modelo de classificação probabilística, esses valores de TPR e FPR não são imutáveis.



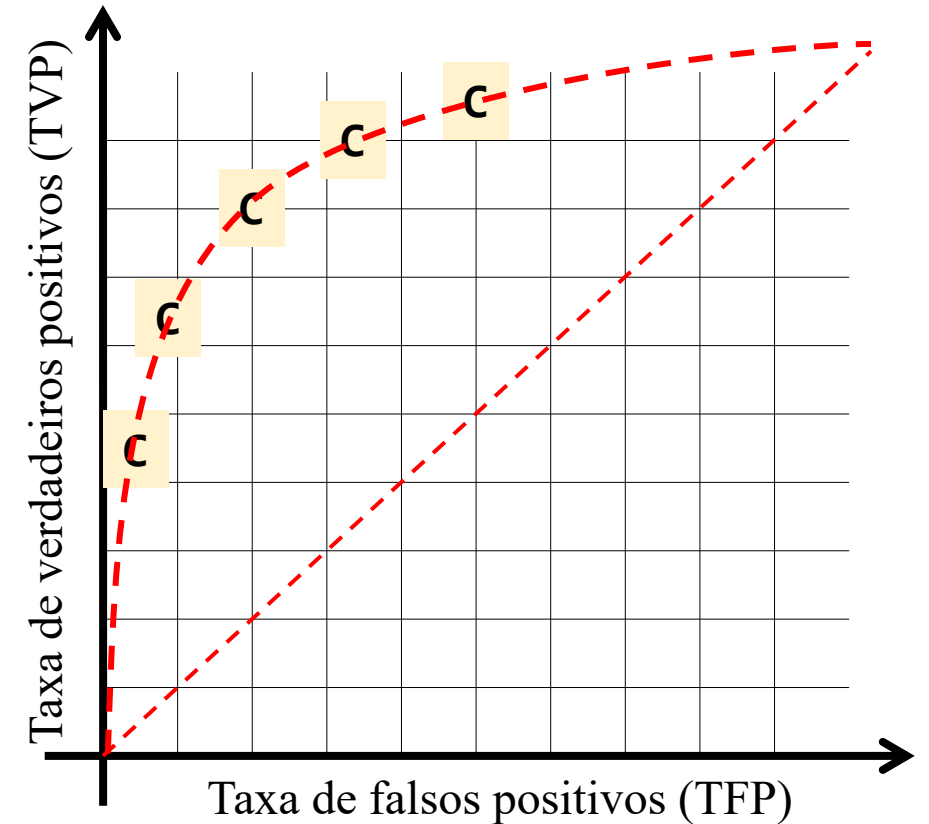
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- Como executar o algoritmo inúmeros vezes?
 - Para criar a curva ROC, você precisa plotar os valores de TVP em relação aos valores de TFP em diferentes limites de decisão.
 - **Você pode perguntar: o que significam valores “diferentes” de TVP e TFP?**
 - Na verdade, calculamos os valores para uma determinada matriz de confusão em um determinado limite de decisão. Mas para um modelo de classificação probabilística, esses valores de TPR e FPR não são imutáveis.



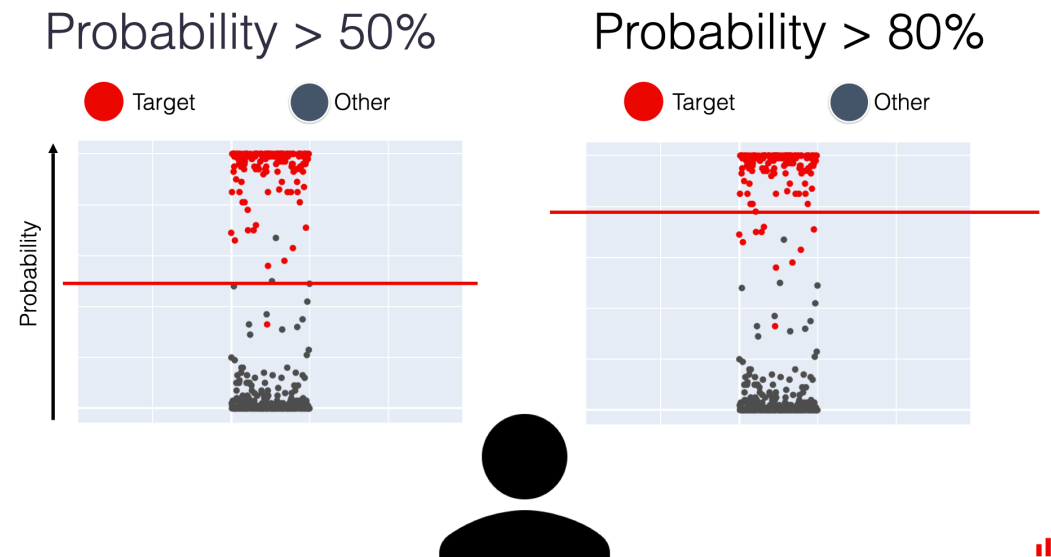
https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- Como executar o algoritmo inúmeros vezes?
 - Para criar a curva ROC, você precisa plotar os valores de TVP em relação aos valores de TFP em diferentes limites de decisão.
 - Você pode perguntar: o que significam valores “diferentes” de TVP e TFP?
 - Na verdade, calculamos os valores para uma determinada matriz de confusão em um determinado limite de decisão. Mas para um **modelo de classificação probabilística**, esses valores de TPR e FPR não são imutáveis.



https://pt.wikipedia.org/wiki/Caracter%C3%ADstica_de_Operac%C3%A7%C3%A3o_do_Receptor

- Você pode **variar o limite de decisão** que define como converter as previsões do modelo em rótulos. Isso, por sua vez, pode alterar o número de erros cometidos pelo modelo.
- Observe que **muitos classificadores discretos podem ser convertidos em um classificador de pontuação** “olhando dentro” de suas estatísticas de instância. Por exemplo, uma árvore de decisão determina a classe de um nó folha a partir da proporção de instâncias no nó.



<https://www.evidentlyai.com/classification-metrics/explain-roc-curve>

- Portanto, o que são “limites de decisão”?
 - Quando um modelo de classificação binária estima a **probabilidade** de uma instância pertencer à classe positiva (por exemplo, “doente” ou “*spam*”), ele precisa depois **decidir** se vai rotular essa instância como positiva ou negativa.
 - Para isso, usamos um **limite de decisão**.
 - Por padrão, esse limite costuma ser **0,5**, ou seja:
 - Se a probabilidade for $\geq 0,5$, classificamos como classe **positiva**.
 - Se for $< 0,5$, classificamos como **classe negativa**.
 - **Porém, esse limite pode ser ajustado!** E ao variá-lo de 0 até 1, podemos observar como mudam:
 - **A Taxa de Verdadeiros Positivos (TVP ou TPR):** quantos positivos foram corretamente classificados.
 - **A Taxa de Falsos Positivos (TFP ou FPR):** quantos negativos foram incorretamente classificados como positivos.
 - A **curva ROC** mostra justamente como TVP e TFP variam à medida que **mudamos esse limite de decisão**.

- Exemplo Ilustrativo com k -NN:
 - O k -NN tradicional **não fornece uma probabilidade diretamente**, mas podemos derivar uma “pontuação de confiança” com base na **proporção de vizinhos** que pertencem à classe positiva.
 - Vamos a um exemplo prático:
 - Suponha que temos um conjunto de **5 amostras** de teste e que usamos $k = 3$. O modelo retorna, para cada amostra, a **proporção de vizinhos da classe positiva (classe 1)**:

Amostra	Proporção de vizinhos positivos	Classe verdadeira
A1	1.0	1
A2	0.66	1
A3	0.33	0
A4	0.66	0
A5	0.0	0

- Agora, vamos calcular TVP e TFP para diferentes limites: 1.0, 0.66, 0.33, 0.0.

- Limite = 1.0:
 - Classificamos como 1 apenas quem tiver score ≥ 1.0 ;
 - TVP (TPR): 1 verdadeiro positivo / 2 positivos = 0.5;
 - TFP (FPR): 0 falsos positivos / 3 negativos = 0.0;
 - Ponto da curva: (0.0, 0.5).

Amostra	Score	Predição
A1	1.0	1
A2	0.66	0
A3	0.33	0
A4	0.66	0
A5	0.0	0

- Limite = 0.66:
 - Classificamos como 1 quem tiver score ≥ 0.66 ;
 - TP: 2;
 - FP: 1 (A4);
 - TVP: $2 / 2 = 1.0$;
 - TFP: $1 / 3 \approx 0.33$;
 - Ponto: (0.33, 1.0).

Amostra	Score	Predição
A1	1.0	1
A2	0.66	1
A3	0.33	0
A4	0.66	1
A5	0.0	0

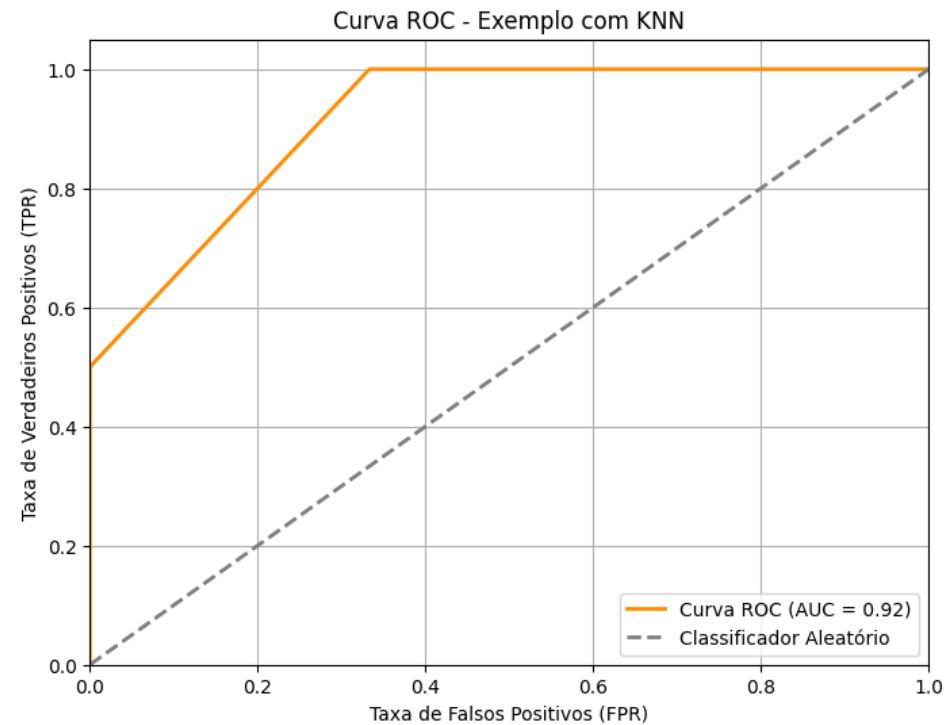
- Limite = 0.33:
 - Classificamos como 1 quem tiver score ≥ 0.33 ;
 - TP: 2;
 - FP: 2 (A3, A4);
 - TPR: $2 / 2 = 1.0$;
 - FPR: $2 / 3 \approx 0.66$;
 - Ponto: (0.66, 1.0).

Amostra	Score	Predição
A1	1.0	1
A2	0.66	1
A3	0.33	1
A4	0.66	1
A5	0.0	0

- Limite = 0.00:
 - Classificamos tudo como positivo;
 - TP = 2;
 - FP = 3;
 - TVP = 1.0;
 - TFP = 1.0;
 - Ponto: (1.0, 1.0).

Amostra	Score	Predição
A1	1.0	1
A2	0.66	1
A3	0.33	1
A4	0.66	1
A5	0.0	1

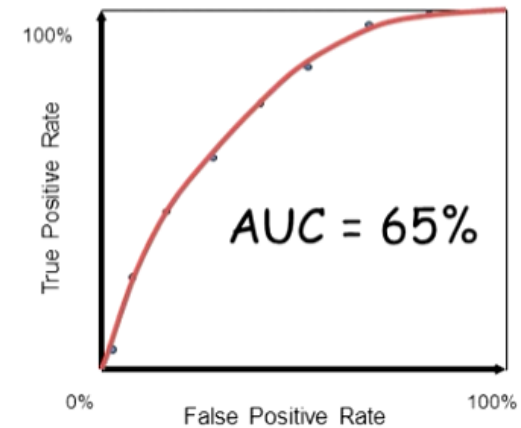
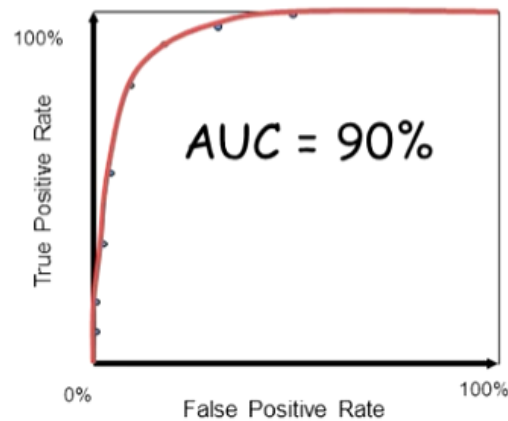
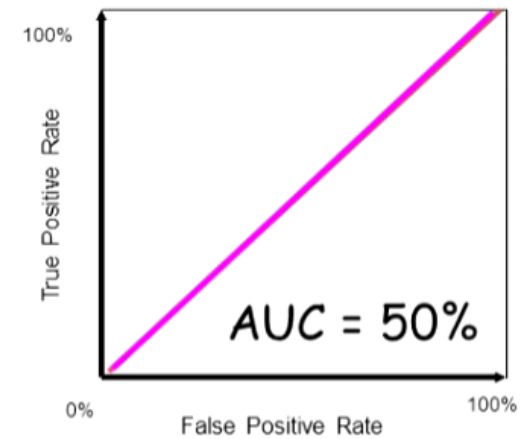
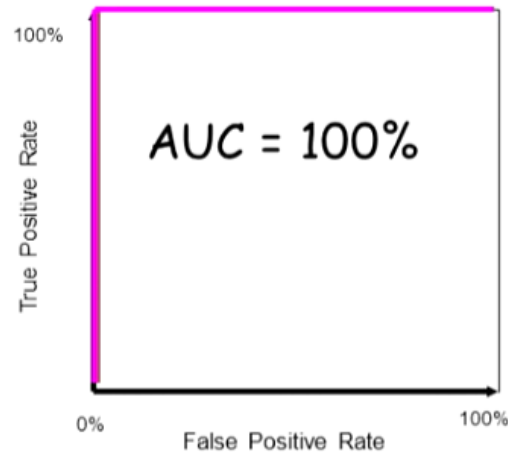
- $AUC = 0.9167$.



Avaliação

Curva AUC-ROC

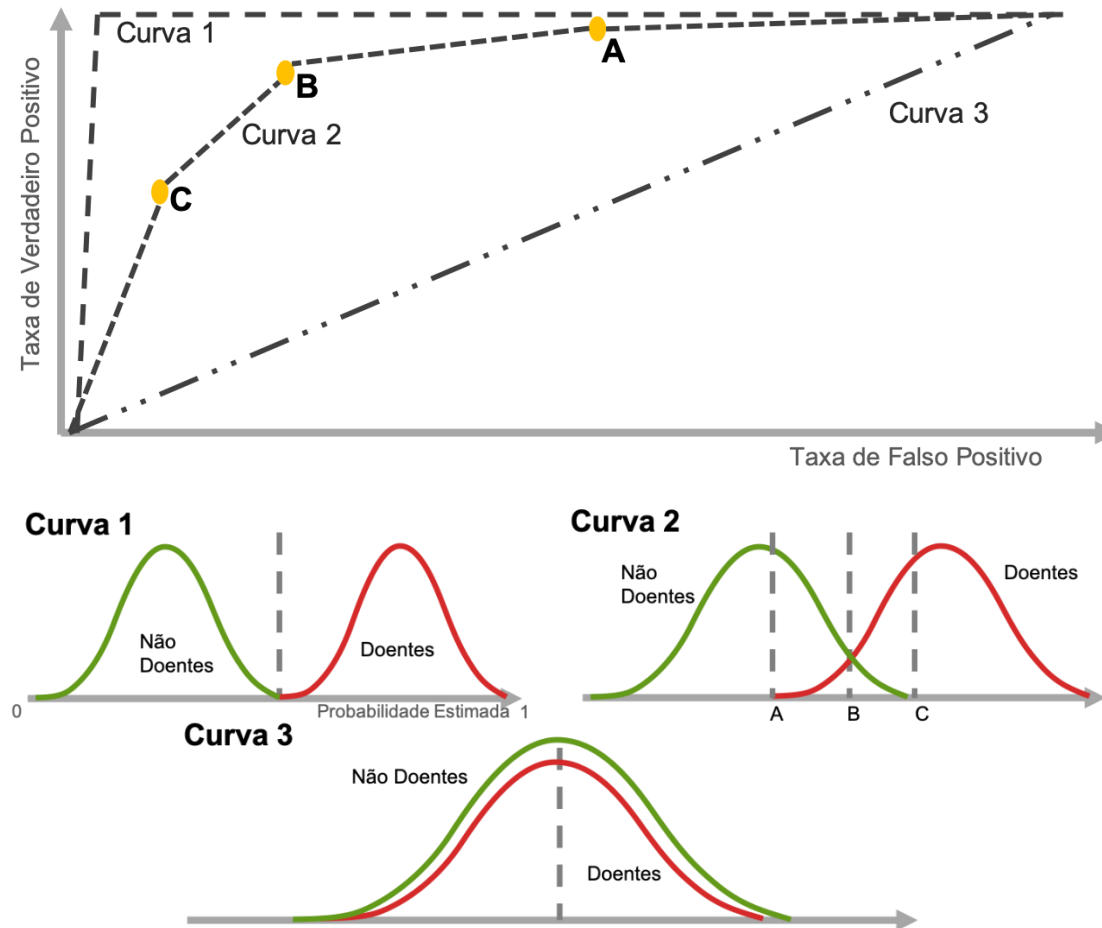
O AUC-ROC é um valor escalar único que facilita a comparação de vários modelos.



<https://www.youtube.com/watch?v=v29IPKrY4yM>

Avaliação

Curva AUC-ROC



<https://cienciaenegocios.com/curva-roc-e-auc-em-machine-learning/>

Qual modelo subir em produção?

- **Primeiro: para que serve mesmo o k -fold?**
 - O k -fold não serve para escolher o melhor modelo;
 - Os modelos dos *folds* são subótimos, pois usam apenas $k-1/k$ dos dados;
 - Ele serve para estimar o desempenho esperado de um modelo treinado com os dados disponíveis;
 - Em cada *fold* você treina com uma parte dos dados e testa na outra;
 - Isso **simula conjuntos diferentes** de treino e teste, produzindo uma estimativa mais robusta do quão bem o seu algoritmo generaliza.
 - Mas cada modelo de cada *fold* foi treinado com **uma fração menor dos dados** (por exemplo, com nove décimos);
 - Então, **nenhum desses modelos é o melhor que você pode treinar.**

- Qual modelo vai para produção?
 - Regra padrão da prática profissional:
 - **Treine um novo modelo usando *todos os dados de treino*** (sem o conjunto de teste final), usando os hiperparâmetros que foram validados no *cross-validation*.
 - Intuição
 - Se os modelos dos *folds* tiveram bom desempenho na média, um modelo treinado com **todos os dados** tende a:
 - Ter menor variância,
 - Generalizar melhor,
 - Incorporar mais padrões,
 - Reduzir *overfitting* em relação aos modelos de cada fold isoladamente.
 - Por isso ele é quase sempre o modelo final.
 - Isso é consenso?
 - Sim.

- **Scikit-learn Documentation — Model Selection**
 - *Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al.*
Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. doze, p. duzentos e oitenta e dois–duzentos e oitenta e seis, dois mil e onze.
 - Documentação oficial:
https://scikit-learn.org/stable/modules/cross_validation.html
 - **Trecho relevante (na documentação oficial):**
 - Na seção “**Grid Search**” e na descrição do **cross_val_score**:
 - *“Note that after choosing the best hyperparameters via cross-validation, the estimator is typically retrained on the full training set”.*
(Scikit-learn User Guide, Model Selection, Cross-Validation)
 - É exatamente a orientação padrão recomendada pela biblioteca.

- **Bishop (2006) — Pattern Recognition and Machine Learning**
 - *Bishop, C. M.*
Pattern Recognition and Machine Learning.
New York: Springer, dois mil e seis. ISBN 0387310738.
 - **Capítulo 1: *Introduction*:**
 - Discussão sobre generalização, treinamento e validação.
 - **Capítulo 7: *Model Selection and Bayesian Inference*:**
 - Seções 7.1 a 7.5 tratam de validação cruzada, evidência e seleção de modelos.
 - **Trechos relevantes (paráfrase):**
 - Bishop explica que:
 - A validação cruzada é usada para **estimar** o erro de generalização;
 - Após determinar os hiperparâmetros que maximizam o desempenho esperado, **o modelo final deve ser treinado com o conjunto completo de dados disponíveis para obter o melhor estimador possível;**
 - No tópico **7.10 *Model Comparison and Cross-Validation***, ele reforça que CV é uma ferramenta para seleção de modelos, **não** para produção de um modelo final diretamente.

- **Hastie, Tibshirani & Friedman (2009) — The Elements of Statistical Learning**
 - *Hastie, T., Tibshirani, R., & Friedman, J.*
The Elements of Statistical Learning: Data Mining, Inference, and Prediction.
2. ed. New York: Springer, dois mil e nove. ISBN 0387848576.
Disponível gratuitamente em:
<https://hastie.su.domains/ElemStatLearn/>
 - **Localização no livro:**
 - **Capítulo 7: *Model Assessment and Selection*:**
 - **Seção 7.10 — *Cross-Validation*:**
 - **Trechos relevantes (paráfrase):**
 - Na seção 7.10, os autores afirmam que:
 - A validação cruzada é uma técnica para **estimar o erro de predição**, especialmente quando os dados são limitados;
 - O objetivo do CV é **avaliar** modelos e **ajustar hiperparâmetros**, não selecionar um dos modelos treinados dentro do CV para uso final;
 - Depois que o modelo e seus hiperparâmetros são escolhidos, ***“the final model is typically trained on the entire dataset”***.

- **Paleyes, A., Urma, R.-G., & Lawrence, N. D. (2021).**
Challenges in Deploying Machine Learning: A Survey of Case Studies.
ACM Computing Surveys, 55(6), 1–29.
- Link: <https://dl.acm.org/doi/10.1145/3533378>.
- **Conteúdo relevante:**
 - Embora o artigo seja sobre desafios de *deploy*, ele descreve a prática comum em pipelines reais:
 - “*Model selection is performed on validation splits, while the final production model is retrained on the full training data to maximize generalization*”.

- **Outras técnicas para gerar o modelo final:**
 - Existem sim alternativas mais sofisticadas:
 - **Ensemble dos modelos dos folds:**
 - Você pode pegar os **10 modelos** treinados no *k-fold* e fazer:
 - média das probabilidades (*soft voting*),
 - votação majoritária (*hard voting*).
 - **Prós:**
 - Reduz variância;
 - Pode superar a performance de cada modelo individual.
 - **Contras:**
 - Mais pesado na produção;
 - Mais difícil de explicar;
 - Essa abordagem é comum em competições do *Kaggle*.

- Existem técnicas modernas que identificam *folds* “ruins” ou instáveis e usam apenas os *folds* consistentes para **orientar** o modelo final:
 - Isso existe e é um campo real de pesquisa;
 - Mas é importante entender **como** isso é feito porque *não se treina o modelo final com os dados de apenas alguns folds, e sim usa-se a análise dos folds bons para guiar que modelo treinar com todos os dados.*
- **O MAIS IMPORTANTE:**
 - Você **não treina o modelo final** usando apenas os dados de alguns *folds* bons;
 - Você treina um modelo “final” que:
 - Usa **todos os dados**
 - Mas “carrega” hiperparâmetros aprendidos usando apenas *folds* estáveis.
 - Isso melhora:
 - Robustez;
 - Generalização;
 - Reprodutibilidade;
 - Detecção de problemas reais de distribuição.

1. Técnicas de estabilidade de validação (*Model Stability / Fold Stability*)

- Essas técnicas analisam se os *folds* apresentam **variância muito alta** entre si;
- Quando um *fold* tem erro muito discrepante, ele pode ser:
 - Descartado;
 - Suavizado;
 - Ou tratado como “*outlier*”.
- Exemplos de métodos:
 - *Cross-Validation Influence Functions*;
 - *Fold Influence Diagnostics*;
 - *Trimmed Cross-Validation* (TCV);
 - *Robust Cross-Validation* (RCV).
- Referência moderna:
 - **Yu & Kumbier (2020)** — *Veridical Data Science* (PNAS 2020).
Eles descrevem como verificar se um *fold* gera resultados instáveis ou implausíveis devido a problemas nos dados.

2. *Trimmed Cross-Validation* (TCV)

- Descartar os *folds* ruins/*outliers* e calcular a estimativa de generalização só com os “bons”.
- **Como funciona:**
 - Calcula o erro em cada *fold*;
 - Detecta *outliers* (típico: valores acima de z-score 2 ou 3).
 - Remove esses *folds*.
 - Faz a média apenas dos *folds* estáveis.
- **Por que existe?**
 - *Folds* podem estar “contaminados” por:
 - Distribuição muito diferente;
 - Erros de particionamento;
 - Rótulos errados;
 - Baixa representatividade.
- **Referência:**
 - **G. Nadeau & Y. Bengio (2000)** — *Inference for the Generalization Error*.
Mostra que folds com comportamento muito discrepante podem distorcer a estimativa de erro.

3. *Repeated Cross-Validation + Selection of Stable Folds:*

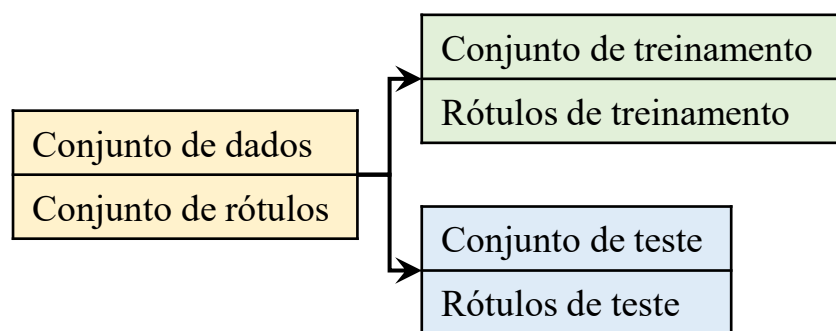
- Uma técnica mais moderna é:
 - Executar ***k-fold*** várias vezes (ex.: 10x10-fold);
 - Observar a distribuição dos erros;
 - Selecionar apenas as execuções estáveis;
 - Essa técnica é usada em:
 - Neurociência (análise EEG/MEG);
 - Bioinformática;
 - Previsão de séries temporais curtas.
- **Referência:**
 - **Varma & Simon (2021)** — *Bias in Cross-Validation*.
Mostra como repetição e seleção de instâncias estáveis melhoram a estimativa de generalização.

4. *Nested Cross-Validation* com detecção de *folds* problemáticos

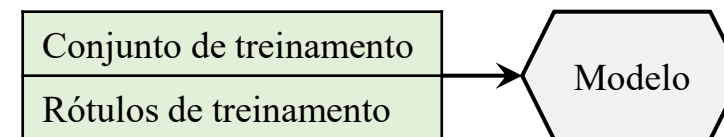
- No contexto de nested CV:
 - O *inner loop* é o tuning;
 - O *outer loop* é a estimativa.
- O *outer loop* pode sinalizar *folds* problemáticos e removê-los.
- **Referência:**
 - **Probst, Wright & Boulesteix (2019)** — *Hyperparameter Tuning: A Practical Guide*.
Eles discutem folds “problemáticos” e como descartá-los.

Fluxograma

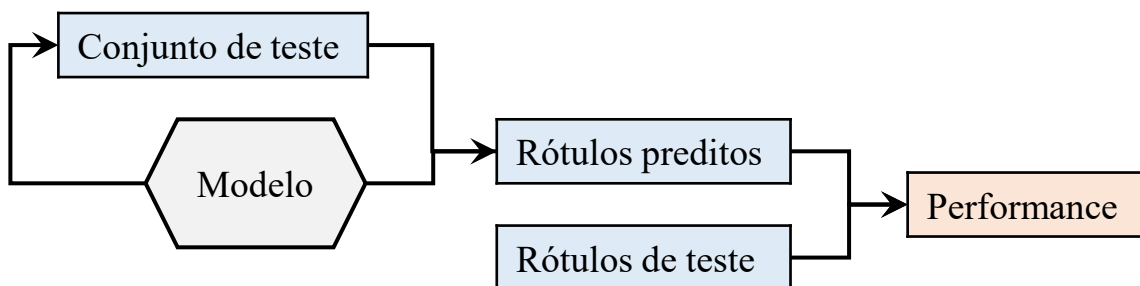
1. Divida o conjunto de dados em dois (de preferência 70–30%; no entanto, a porcentagem de divisão pode variar e deve ser aleatória).



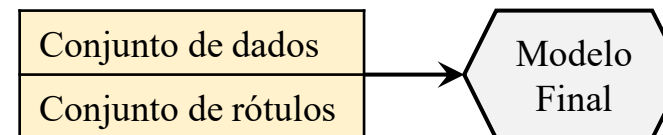
2. Agora, treinamos o modelo no conjunto de dados de treinamento selecionando um conjunto fixo de hiperparâmetros durante o treinamento do modelo.



3. Use o conjunto de dados de teste para avaliar o modelo.



4. Use todo o conjunto de dados para treinar o modelo final para que ele possa ser melhor generalizado em conjuntos de dados futuros.



Análise visual

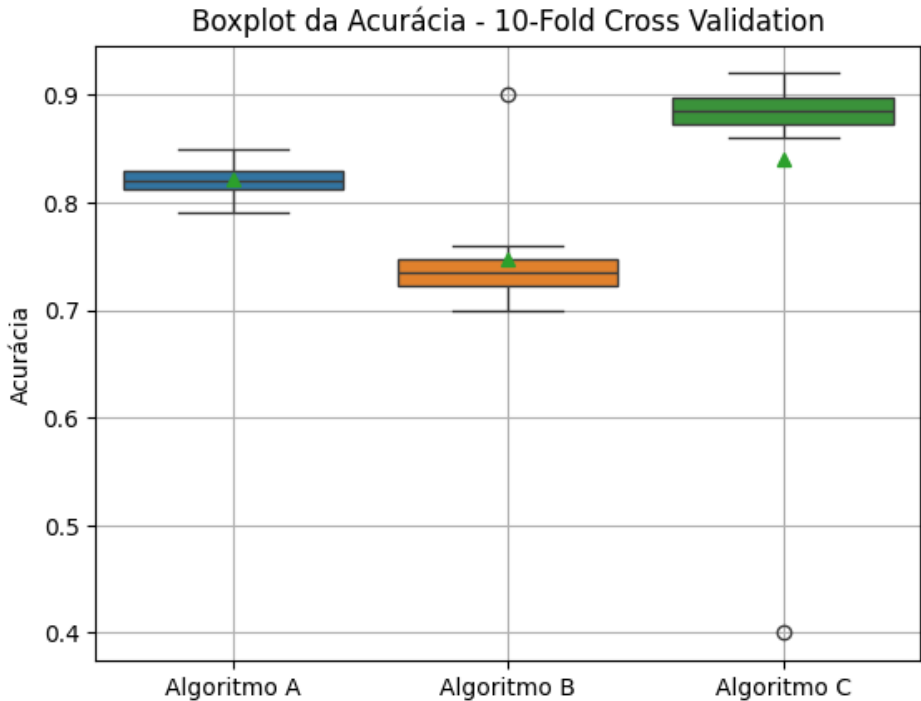
Avaliação

Visualização dos *folds*

- Vamos simular um exemplo simples de validação cruzada com 10 *folds*, comparando três algoritmos com base na acurácia. Também incluiremos outliers intencionais para análise visual e explicação.

<i>Fold</i>	Algoritmo A	Algoritmo B	Algoritmo C
1	0.82	0.75	0.90
2	0.83	0.74	0.89
3	0.85	0.73	0.91
4	0.80	0.70	0.88
5	0.82	0.74	0.87
6	0.84	0.76	0.89
7	0.81	0.71	0.86
8	0.79	0.90	0.88
9	0.83	0.72	0.92
10	0.82	0.73	0.40

Algoritmo	Média	Desvio Padrão	Outliers
Algoritmo A	0.821	0.017	[]
Algoritmo B	0.758	0.060	[0.90]
Algoritmo C	0.79	0.162	[0.40]

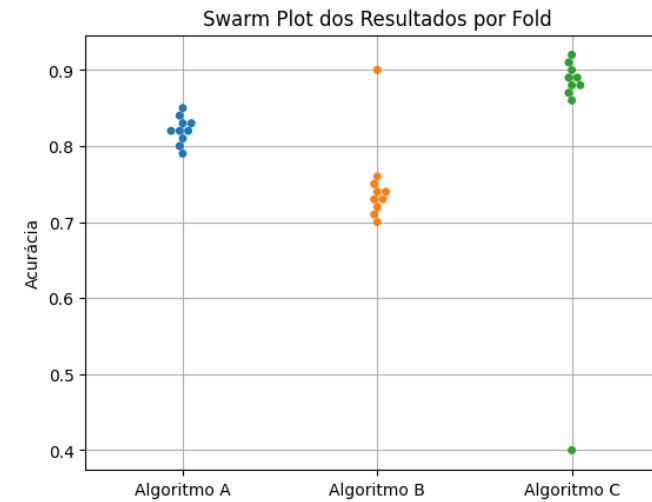
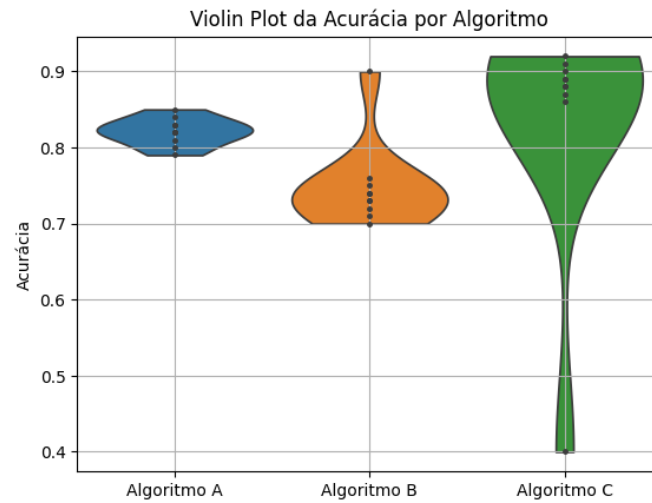
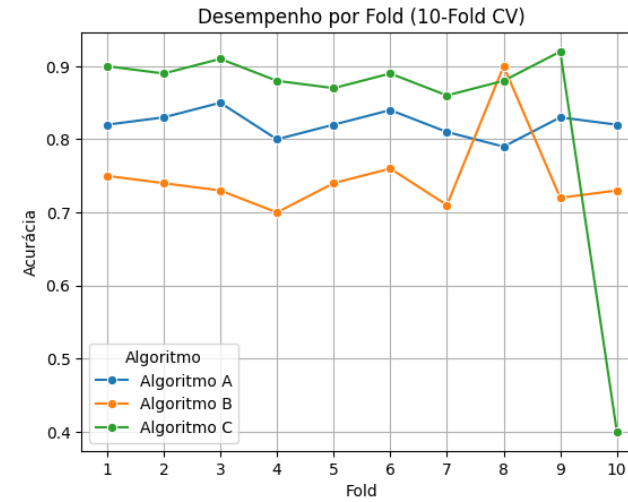
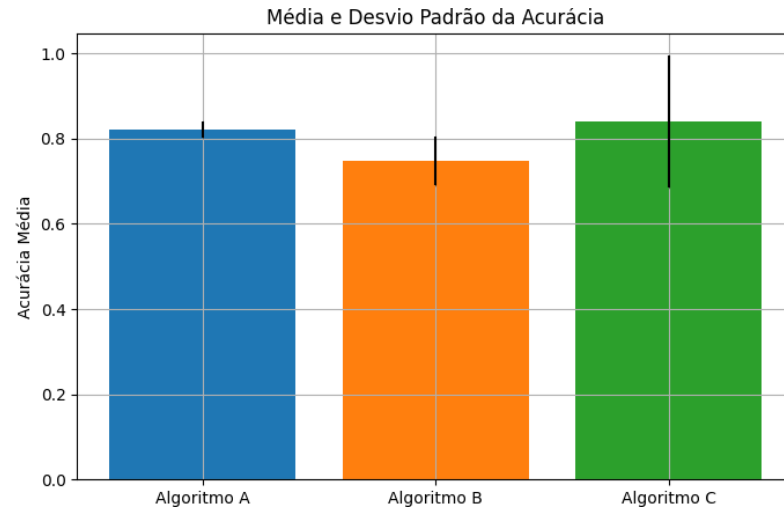


- Análise dos Resultados:
 - Algoritmo A: Desempenho estável com pequena variação (entre 0.79 e 0.85);
 - Algoritmo B: Quase todo o desempenho está entre 0.70 e 0.76, mas no *fold* 8 houve uma acurácia muito alta (0.90) → outlier positivo;
 - Algoritmo C: Consistentemente bom, mas no *fold* 10 teve uma queda brusca (0.40) → outlier negativo.
- Por que essas variações podem acontecer?
 - ***Fold* com distribuição de classes diferente:**
 - Mesmo com estratificação, pode haver variações na proporção de classes que tornam a tarefa mais fácil ou mais difícil para o modelo.
 - ***Overfitting* em dados específicos:**
 - O modelo pode ter “decorado” padrões de um *fold* onde os exemplos eram muito fáceis, elevando a acurácia (ex: Algoritmo B no *fold* 8).
 - ***Underfitting* ou dados difíceis no *fold*:**
 - Um *fold* pode ter exemplos raros ou difíceis de classificar, derrubando a performance (ex: Algoritmo C no *fold* 10).
 - **Pequeno tamanho do conjunto de dados:**
 - Amplifica o impacto de poucos exemplos mal classificados.

Gráfico	Melhor para ...
Boxplot	• Ver mediana, outliers e variabilidade
Violin Plot	• Parecido com o <i>boxplot</i>, mas mostra a distribuição dos dados (densidade). • Útil para visualizar simetria, multimodalidade e espalhamento. • Ver forma da distribuição.
Swarm Plot	• Mostra cada ponto individual. • Útil para ver sobreposição de valores e a dispersão real. • Ver cada resultado individual.
Line Plot	• Mostra a evolução do desempenho ao longo dos <i>folds</i>. • Pode revelar tendência de aprendizado, variabilidade por <i>fold</i> ou instabilidade. • Ver como cada algoritmo variou nos <i>folds</i>.
Barplot + Erro	• Resume a média de cada algoritmo com barras e mostra o desvio padrão como erro. • Comparar médias e incertezas (desvios padrão).

Avaliação

Visualização dos *folds*

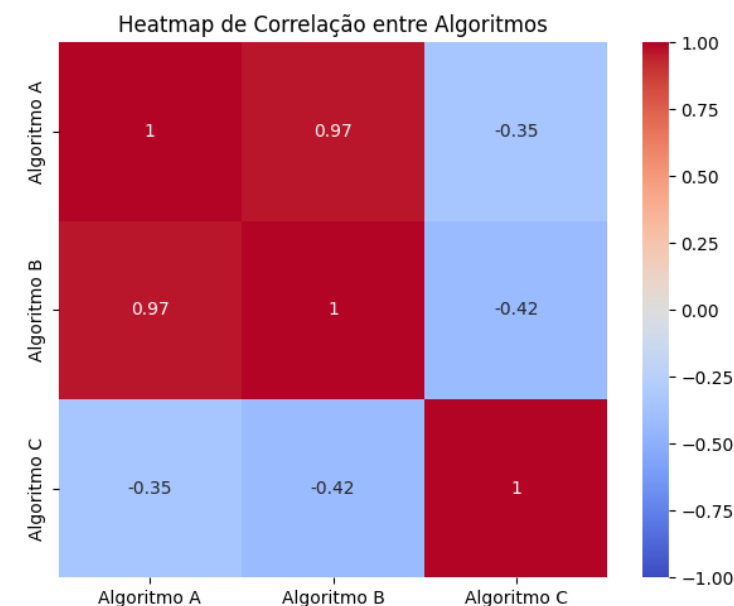
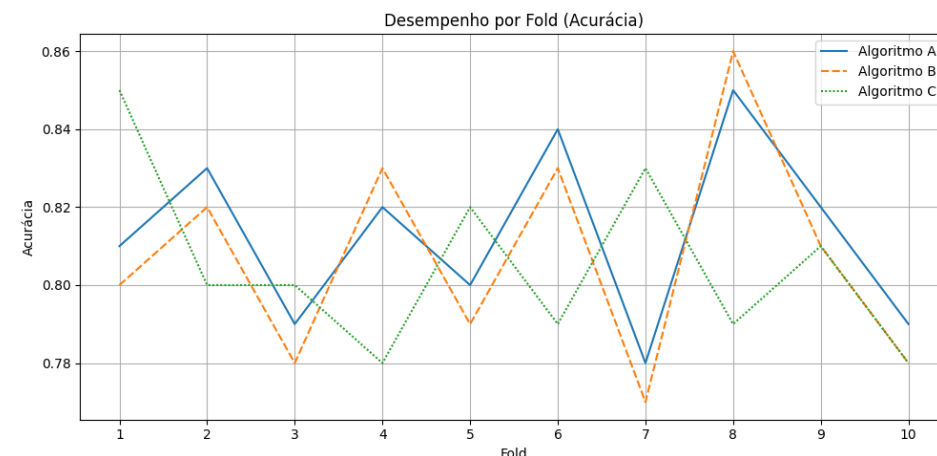


- **Heatmap** da matriz de correlação entre algoritmos:
 - Mostra a **correlação** entre as acurácias *fold a fold* dos algoritmos.
 - Ou seja: Ele mede **como os desempenhos variam juntos ao longo dos 10 *folds***.
 - A correlação varia de **-1 (fortemente inversa)** a **+1 (fortemente direta)**.
 - Mostra o quanto os desempenhos dos algoritmos estão correlacionados *fold a fold*;
 - Útil para ver **se eles erram/acertam nos mesmos casos**;
 - Por exemplo, **se dois algoritmos tendem a ir bem ou mal nos mesmos *folds***, sua correlação será alta.
- Suponha:
 - # Algoritmos A e B são altamente correlacionados
 - alg_A = [0.81, 0.83, 0.79, 0.82, 0.80, 0.84, 0.78, 0.85, 0.82, 0.79]
 - alg_B = [0.80, 0.82, 0.78, 0.83, 0.79, 0.83, 0.77, 0.86, 0.81, 0.78]
 - # Algoritmo C tem comportamento muito diferente (baixa correlação com A e B)
 - alg_C = [0.85, 0.80, 0.80, 0.78, 0.82, 0.79, 0.83, 0.79, 0.81, 0.78]

Avaliação

Visualização dos *folds*

- Calculamos a **correlação de Pearson** entre cada desempenho.
- Conclusão da análise do heatmap:
 - **Algoritmos A e B** são consistentes entre si — seus desempenhos têm **correlação positiva forte**;
 - **Algoritmo C** se comporta **de forma diferente**, provavelmente por ter outliers — e isso é visível tanto no boxplot quanto na correlação fraca no heatmap.
- Por que isso é útil?
 - Ajuda a **identificar comportamentos complementares**. Se um algoritmo vai bem onde o outro vai mal, pode indicar uma **boa combinação para *ensemble* (ou comitê)**;
 - Pode **revelar algoritmos instáveis**, como o Algoritmo C, que teve **outliers e baixa correlação com os demais**.
 - Fornece um **nível extra de análise além da média de desempenho**.



- A correlação de *Pearson* (r) **mede a força e a direção da relação linear entre duas variáveis**:
 - Varia de -1 a +1
 - **+1: correlação linear perfeita positiva** (à medida que uma variável aumenta, a outra também);
 - 0: nenhuma correlação linear;
 - **-1: correlação linear perfeita negativa** (uma aumenta, a outra diminui).
 - A fórmula é:

$$r = \frac{\sum (x_i - \bar{x}) (y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \times \sqrt{\sum (y_i - \bar{y})^2}}$$

- Numerador: Covariância.
 - $\sum (x_i - \bar{x}) (y_i - \bar{y})$
- Esse é o **numerador** e representa a **covariância não normalizada** entre as duas variáveis (ou vetores de acurácia, por exemplo).
- Intuição:
 - Mede **como x e y variam juntos**;
 - Se os dois aumentam juntos (acima de suas médias), o produto $\sum (x_i - \bar{x}) (y_i - \bar{y})$ é positivo;
 - Se um sobe e o outro desce, o produto é negativo;
 - Assim, a soma indica **a direção e intensidade da relação conjunta**.

- Denominador: Produto dos desvios padrão.
 - $\sqrt{\sum (x_i - \bar{x})^2} \times \sqrt{\sum (y_i - \bar{y})^2}$
- Esse é o **denominador**, e representa o produto das medidas de **dispersão (ou variabilidade) dos dois vetores**.
- Intuição:
 - Ele **normaliza a covariância**, ou seja, ajusta o numerador para o quão espalhados são os dados;
 - Se os dados forem muito variáveis, o denominador será maior, portanto, a correlação tende a diminuir;
 - Se forem pouco variáveis, o denominador será menor, portanto, a correlação tende a aumentar;
 - Esse fator garante que r fique entre **-1 e 1**, sempre.

- Na verdade a fórmula da correlação de *Pearson* supprime a divisão n ou $(n - 1)$ das fórmulas de correção:

$$r = \frac{\frac{1}{n} \sum (x_i - \bar{x}) (y_i - \bar{y})}{\sqrt{\frac{1}{n} \sum (x_i - \bar{x})^2} \times \sqrt{\frac{1}{n} \sum (y_i - \bar{y})^2}}$$

- Os fatores $\frac{1}{n}$ se cancelam, então o valor final não muda se usar n ou $n - 1$, desde que você use o mesmo nos dois (covariância e desvio).
- Importante: o que não pode é misturar — por exemplo, dividir a covariância por $n - 1$ e os desvios por n . Isso mudaria o valor da correlação.

- Passo 1: Dados:
 - Vamos comparar os seguintes vetores:
 - # Algoritmos A e B são altamente correlacionados
 - alg_A = [0.81, 0.83, 0.79, 0.82, 0.80, 0.84, 0.78, 0.85, 0.82, 0.79]
 - alg_B = [0.80, 0.82, 0.78, 0.83, 0.79, 0.83, 0.77, 0.86, 0.81, 0.78]
- Passo 2: Médias:

$$\bar{A} = \frac{0,81 + 0,83 + \dots + 0,79}{10} = 0,813$$

$$\bar{B} = \frac{0,80 + 0,82 + \dots + 0,78}{10} = 0,807$$

- Passo 3: Tabela com os desvios e produtos:

A	B	A - 0.813	B - 0.807	$(A_i - \bar{A})(B_i - \bar{B})$	$(A_i - \bar{A})^2$	$(B_i - \bar{B})^2$
0.81	0.80	-0.003	-0.007	0.000021	0.000009	0.000049
0.83	0.82	0.017	0.013	0.000221	0.000289	0.000169
0.79	0.78	-0.023	-0.027	0.000621	0.000529	0.000729
0.82	0.83	0.007	0.023	0.000161	0.000049	0.000529
0.80	0.79	-0.013	-0.017	0.000221	0.000169	0.000289
0.84	0.83	0.027	0.023	0.000621	0.000729	0.000529
0.78	0.77	-0.033	-0.037	0.001221	0.001089	0.001369
0.85	0.86	0.037	0.053	0.001961	0.001369	0.002809
0.82	0.81	0.007	0.003	0.000021	0.000049	0.000009
0.79	0.78	-0.023	-0.027	0.000621	0.000529	0.000729

- Passo 4: Somatórios:
 - Soma dos produtos:
 - $\sum (A_i - \bar{A}) (B_i - \bar{B}) = 0.00567$.
 - Soma dos quadrados (A):
 - $\sum (A_i - \bar{A})^2 = 0.00481$.
 - Soma dos quadrados (B):
 - $\sum (B_i - \bar{B})^2 = 0.00721$.

$$r = \frac{\sum (A_i - \bar{A}) (B_i - \bar{B})}{\sqrt{\sum (A_i - \bar{A})^2 \sum (B_i - \bar{B})^2}} = \frac{0.00567}{\sqrt{0.00481 \times 0.00721}} = \frac{0.00567}{\sqrt{0.00003467}} \approx \frac{0.00567}{0.00589} \approx 0.962 \approx 0.97$$

Avaliação

Visualização para múltiplas bases de dados?

- Você pode montar uma **matriz (algoritmo × base)**.
- Para isso, você pode seguir o seguinte processo:
 - Para cada base de dados, aplique o *k-fold cross validation*.
 - Para cada algoritmo, calcule as métricas médias (e desvios padrão, se quiser) dos *folds*:
 - Acurácia;
 - F1-score;
 - E outras métricas, se desejar.

Algoritmo	Iris (F1)	Wine (F1)	Digits (F1)
KNN	0.95	0.89	0.97
SVM	0.94	0.90	0.98
Naive Bayes	0.91	0.88	0.89
Decision Tree	0.92	0.85	0.91
Neural Network	0.93	0.91	0.95

Análise estatística

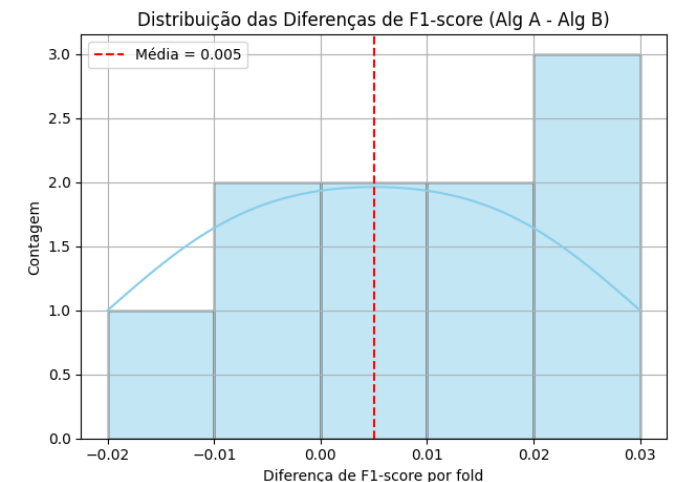
- Quando dois algoritmos são avaliados usando *k-fold cross-validation* ou *Holdout* na mesma base de dados, os resultados podem ser **comparados estatisticamente** porque temos **amostras pareadas** (mesmos *folds* para ambos). Isso permite aplicar testes estatísticos apropriados.
- Etapas para comparação estatística entre dois algoritmos:
 1. Executar o *k-fold* ou *holdout* para ambos os algoritmos;
 - Use o mesmo particionamento (ou seja, mesma semente aleatória) para garantir que ambos os algoritmos foram testados nos mesmos *folds*;
 - Isso garante comparações pareadas, o que é crucial.
 2. Obter a métrica de interesse para cada *fold*:
 - Por exemplo: acurácia, *f1-score*, etc;
 - Você terá dois vetores de tamanho k com os resultados (um para cada algoritmo).
 - Exemplo ($k = 5$):
 - Algoritmo A: [0.82, 0.84, 0.80, 0.79, 0.83]
 - Algoritmo B: [0.78, 0.80, 0.77, 0.76, 0.79]

3. Aplicar o teste estatístico apropriado:
 1. Se os dados forem aproximadamente normais:
 - Use o teste t pareado (*paired t-test*)
 - Hipóteses:
 - H_0 : A média das diferenças entre os algoritmos é zero.
 - H_1 : Existe diferença significativa entre os algoritmos.
 2. Se os dados não forem normais ou se quiser uma abordagem mais conservadora:
 - Use o teste de Wilcoxon *signed-rank* (não paramétrico).
4. Interpretar o p-valor:
 - Se $p < 0.05$:
 - Há diferença estatística significativa entre os dois algoritmos.
 - Se $p \geq 0.05$:
 - Não há evidência suficiente para dizer que eles diferem estatisticamente.

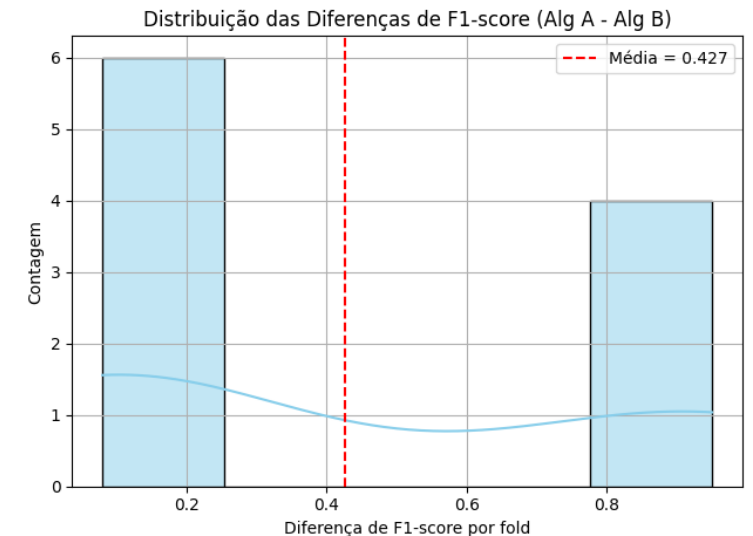
- Exemplo prático:
 - Suponha:
 - `scipy.stats import ttest_rel`
 - `A = [0.82, 0.84, 0.80, 0.79, 0.83]`
 - `B = [0.78, 0.80, 0.77, 0.76, 0.79]`
 - `stat, p = ttest_rel(A, B)`
 - `print(f"p-valor: {p:.4f}")`
 - Se $p < 0.05$, você pode concluir que os algoritmos são estatisticamente diferentes.
 - Caso positivo, o melhor algoritmo é o algoritmo que obteve melhor valor médio de performance, sendo estatisticamente superior ao algoritmo concorrente.

- Quando falamos que os **dados são normais ou aproximadamente normais**, estamos nos referindo à **distribuição dos dados**:
 - Mais especificamente, se eles seguem ou se parecem com uma **distribuição normal** (ou gaussiana), também conhecida como curva de sino.
- O que é uma distribuição normal?
 - É uma distribuição **simétrica** em torno da média.
 - Suas principais características:
 - A maioria dos valores está próxima da média;
 - Poucos valores estão nas extremidades (caudas);
 - A média, mediana e moda são (ou quase são) iguais.

- Exemplos de dados **aproximadamente** normais:
 - Imagine a diferença de acurácia entre dois algoritmos em cada *fold*:
 - [0.02, -0.01, 0.00, 0.03, -0.02, 0.01, 0.00, -0.01, 0.01, 0.02]
 - Esses dados são:
 - Centrado em zero;
 - Simétricos;
 - Sem valores extremos (*outliers*).
 - Provavelmente seguem uma distribuição aproximadamente normal:
 - Podemos usar o teste t.

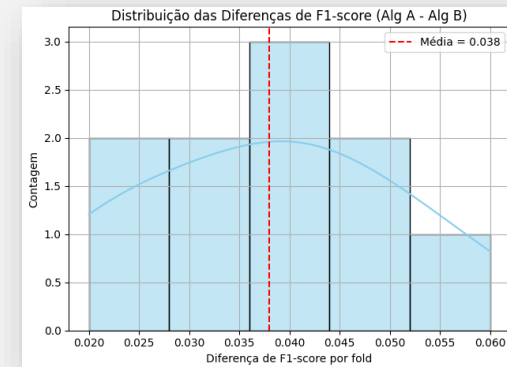


- Exemplos de dados não normais:
 - Imagine a diferença de acurácia entre dois algoritmos em cada *fold*:
 - [0.10, 0.08, 0.09, 0.12, 0.11, 0.09, 0.90, 0.92, 0.95, 0.91].
 - Esses dados são:
 - Há uma cauda longa à direita (valores muito altos);
 - Os dados são assimétricos;
 - Valores extremos (outliers) distorcem a média.
 - Neste caso, não é seguro usar o teste t, e o melhor é usar o teste de Wilcoxon (não depende da normalidade dos dados).



- Como verificar se os dados são normais?
 - Você pode:
 - Visualizar com histogramas ou gráfico de densidade;
 - Usar testes estatísticos como:
 - Shapiro-Wilk;
 - Kolmogorov-Smirnov;
 - Normal test (D'Agostino).

- **Importante:**
 - Quando falamos em **verificar se os dados são normais** nesse contexto, os “dados” são:
 - Os valores de desempenho (por exemplo, acurácia, precisão, F1-score etc.) obtidos por um algoritmo em cada *fold* da validação cruzada.
- Exemplo:
 - Imagine que você está comparando dois algoritmos usando *k-fold cross-validation* com $k = 10$.
 - Você mede o F1-score em cada *fold*:
 - Algoritmo A: [0.82, 0.85, 0.81, 0.83, 0.84, 0.80, 0.82, 0.83, 0.81, 0.84];
 - Algoritmo B: [0.78, 0.79, 0.77, 0.80, 0.81, 0.78, 0.79, 0.80, 0.77, 0.79].
 - Para comparar A e B com um teste estatístico pareado, você primeiro calcula as diferenças:
 - Diferenças A - B: [0.04, 0.06, 0.04, 0.03, 0.03, 0.02, 0.03, 0.03, 0.04, 0.05];
 - Agora **esses valores de diferença** são os **dados** que você precisa testar se são **aproximadamente normais**.
- Por quê?
 - Se essas diferenças forem **simétricas e bem distribuídas ao redor de zero**, o **teste t pareado** é apropriado;
 - Se forem **assimétricas, com outliers ou distribuição esquisita**, melhor usar o **teste de Wilcoxon**, que não assume normalidade.



- Diferenças A - B: [0.04, 0.06, 0.04, 0.03, 0.03, 0.02, 0.03, 0.03, 0.04, 0.05].
- Resultados dos Testes Estatísticos:
 - Shapiro-Wilk (teste de normalidade das diferenças):
 - Estatística = 0.916;
 - p-valor = 0.328;
 - Como o p-valor é maior que 0.05, as diferenças podem ser consideradas aproximadamente normais.
 - Testes de Comparação:
 - Teste t pareado:
 - Estatística = 10.09;
 - p-valor = 3.32e-06
 - Diferença significativa entre os algoritmos ($p < 0.05$).
 - Teste de Wilcoxon:
 - Estatística = 0.0;
 - p-valor = 0.00195
 - Também indica diferença significativa
 - Diferença média:
 - Alg A – Alg B:
 - Média = 0.037 (ou seja, Algoritmo A teve desempenho médio superior em *F1-score*, sendo estatisticamente superior ao algoritmo B).

Teste	Estatística	p-valor	Conclusão
Shapiro-Wilk	0.916	0.328	Normalidade assumida
t de Student pareado	10.091	0.000003	Diferença significativa
Wilcoxon	0.000	0.001953	Diferença significativa

- Isso indica que as diferenças são normais e que o algoritmo A foi estatisticamente superior ao B.

- Se a normalidade foi assumida, por que calculamos ambos os testes t-pareado e Wilcoxon ao invés de usar apenas o t-pareado?
 - Mesmo quando a normalidade é assumida, muitas vezes os pesquisadores optam por reportar os dois testes — t de Student pareado (paramétrico) e Wilcoxon (não paramétrico) — pelas seguintes razões:
 1. Reforço da confiabilidade dos resultados:
 - Se ambos os testes concordam (como no seu caso), isso fortalece a evidência de que a diferença entre os algoritmos é estatisticamente significativa, mesmo que pequenas violações na normalidade passem despercebidas.
 2. Robustez a outliers e pequenas amostras:
 - O teste de Wilcoxon é menos sensível a outliers ou amostras pequenas (como 10 *folds*). Mesmo que a normalidade não seja rejeitada, isso não garante que o teste t seja ideal em todos os casos.
 3. Prática comum em Machine Learning experimental:
 - Na área de aprendizado de máquina, é comum usar testes não paramétricos (como Wilcoxon ou Nemenyi) mesmo quando os dados parecem normais, por serem mais seguros em termos de suposições.

- Situação:
 - Você quer comparar, por exemplo, os algoritmos A e B em três bases de dados usando *k-fold cross-validation*, e depois avaliar estatisticamente qual é melhor no geral.
 - O que não se deve fazer:
 - Juntar os *folds* das três bases e aplicar o teste estatístico sobre todos os valores juntos:
 - Isso mistura populações diferentes, violando o pressuposto de independência e homogeneidade do teste.
 - Fazer o teste em cada base separadamente e depois tirar a média dos p-valores:
 - Isso não tem base estatística válida;
 - O p-valor não é uma métrica agregável como média.
 - Estratégia correta (mais aceita na literatura):
 - Usar os valores agregados por base de dados.
 - Exemplo:
 - Para cada algoritmo em cada base de dados, obtenha o desempenho médio em cada métrica (ex: F1-score médio via *k-fold*);
 - Com isso, você terá uma única pontuação por base de dados para cada algoritmo.
 - Com esses valores, aplique o teste estatístico:
 - Para dois algoritmos: use teste t pareado ou Wilcoxon com os valores médios em cada base;
 - Para mais de dois algoritmos: use Friedman + Nemenyi.

- Exemplo:
 - Com esses três pares de valores, você aplica o teste de Wilcoxon pareado ou t-pareado.
 - Vantagens dessa abordagem:
 - Leva em conta que cada base é um universo diferente;
 - Garante que os testes respeitem os pressupostos (mesmo número de pares, independência);
 - É a estratégia usada em artigos com múltiplas bases (veja Demsar 2006, referência clássica na área).

Base de Dados	Algoritmo A (F1)	Algoritmo B (F1)
Iris	0.96	0.94
Wine	0.89	0.87
Digits	0.95	0.96

- Nesse caso, temos que verificar se a distribuição é normal usando Shapiro-Wilk? Temos poucos valores, ou seja, apenas 3? O Shapiro-Wilk vai funcionar?
- O que acontece quando temos apenas três pares de valores (por exemplo, resultados em três bases de dados)?
 - Sobre o teste de normalidade (Shapiro-Wilk):
 - O teste de Shapiro-Wilk é projetado para verificar se uma amostra vem de uma distribuição normal;
 - Entretanto, com $N = 3$, o teste praticamente não tem poder estatístico suficiente para dizer algo confiável;
 - O próprio teste pode nem retornar um resultado válido ou simplesmente nunca rejeitar a hipótese nula (normalidade), mesmo que a distribuição não seja normal.
 - O que fazer, então?
 - Dado o pequeno número de amostras (3 bases), a estratégia mais segura e aceita é:
 - Usar teste não paramétrico diretamente, como o teste de Wilcoxon pareado, sem testar normalidade;
 - O Wilcoxon não assume normalidade;
 - Ele é mais robusto e indicado para N pequeno, desde que as diferenças entre pares possam ser ordenadas (o que geralmente é o caso).

Base de Dados	Algoritmo A (F1)	Algoritmo B (F1)
Iris	0.96	0.94
Wine	0.89	0.87
Digits	0.95	0.96

- Resumo da abordagem para N pequeno (ex: 3 bases de dados):
 - Não faça o teste de normalidade (Shapiro-Wilk);
 - Aplique diretamente o teste de Wilcoxon pareado;
 - Compare os valores médios de desempenho por base entre os algoritmos.

Base de Dados	Algoritmo A (F1)	Algoritmo B (F1)
Iris	0.96	0.94
Wine	0.89	0.87
Digits	0.95	0.96

- Caso tenha muitas bases de dados, por exemplo, 10 bases, seria mais confiável realizar o Shapiro-Wilk?
 - Sim, exatamente;
 - Quando você tem mais bases de dados — como dez ou mais — o teste de Shapiro-Wilk se torna muito mais confiável e é altamente recomendado para verificar a normalidade.
- Por que o Shapiro-Wilk funciona melhor com mais bases?
 - O teste de Shapiro-Wilk precisa de uma quantidade mínima de dados para ter poder estatístico suficiente para detectar desvios da normalidade;
 - Com $N \geq 10$, ele já começa a produzir resultados estatisticamente úteis e interpretáveis;
 - Entre 10 e 50 observações, ele é considerado um dos testes mais sensíveis e confiáveis para normalidade.
- Resumo:

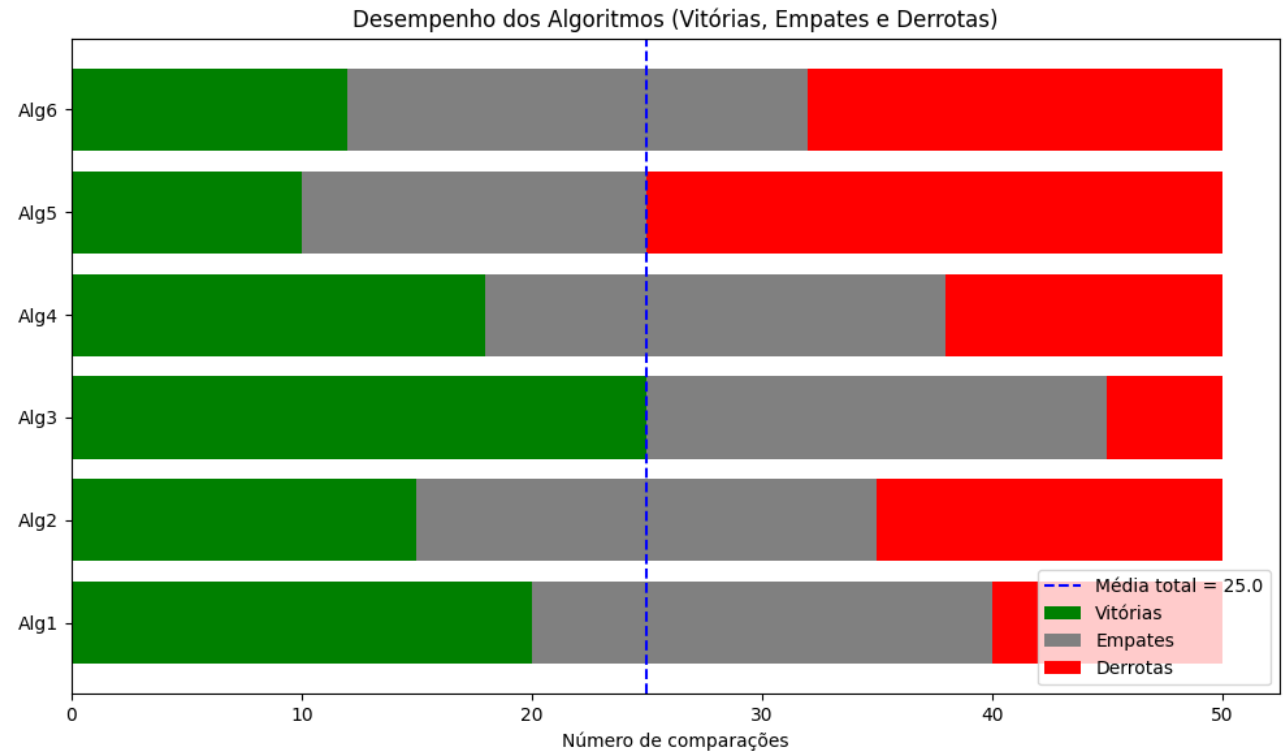
Número de Bases (ou pares de valores)	Deve usar Shapiro-Wilk?	Melhor teste estatístico
2 ou 3	Não	Wilcoxon pareado
4 a 7	Com cautela	Melhor usar Wilcoxon
≥ 10	Sim	t-pareado ou Wilcoxon (dependendo do resultado de normalidade)

- Exemplo de fluxo com 10 bases:
 - Obtenha as médias (ex: *F1-score* médio) por base para cada algoritmo;
 - Aplique o teste de Shapiro-Wilk nas diferenças entre algoritmos A e B;
 - Se a normalidade for confirmada → use o t-pareado;
 - Se não houver normalidade → use o Wilcoxon pareado.

Base	Algoritmo A	Algoritmo B	Diferença (A - B)
Base 1	0.8299	0.7807	0.0492
Base 2	0.8172	0.7807	0.0365
Base 3	0.8330	0.7948	0.0381
Base 4	0.8505	0.7517	0.0987
Base 5	0.8153	0.7555	0.0598
Base 6	0.8153	0.7788	0.0366
Base 7	0.8516	0.7697	0.0818
Base 8	0.8353	0.7963	0.0391
Base 9	0.8106	0.7718	0.0388
Base 10	0.8309	0.7618	0.0691

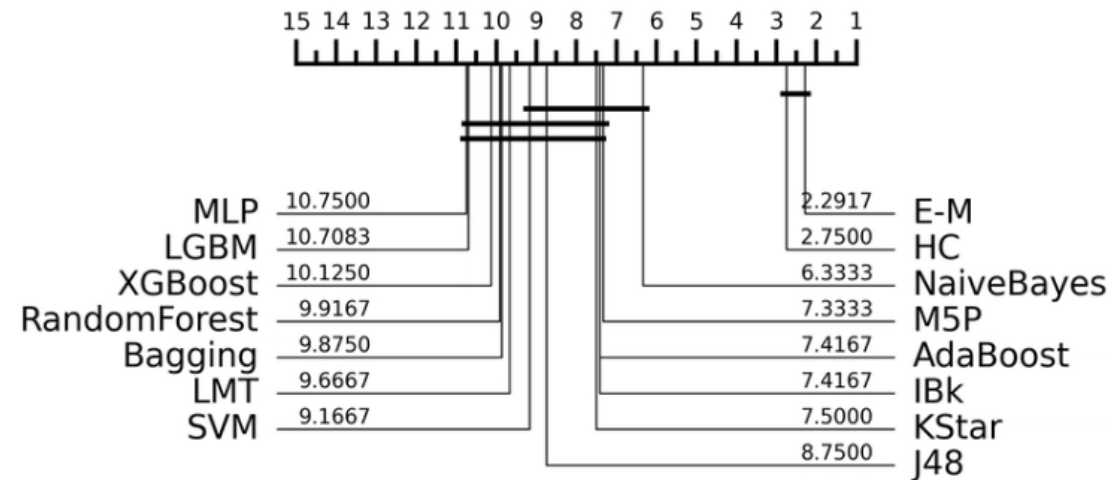
- Teste de Normalidade das Diferenças:
 - Shapiro-Wilk: estatística = 0.8292;
 - p-valor = 0.0327;
 - Como o p-valor < 0.05 , as diferenças não seguem uma distribuição normal.
- Teste Estatístico Usado:
 - Teste de Wilcoxon (não paramétrico, pois os dados não são normais)
 - Estatística = 0.0;
 - p-valor = 0.00195.
- Conclusão:
 - Há diferença estatística significativa entre os algoritmos A e B;
 - O Algoritmo A teve desempenho superior com base nos *F1-scores* médios em dez bases de dados.

- **Importante:**
 - Para 3 ou mais algoritmos, é necessário fazer essa análise par a par.
- Podemos mostrar um gráfico com o número de vitórias, derrotas e empates, para cada algoritmo.
 - O gráfico ao lado ilustra 6 algoritmos em 10 bases de dados.
- Personalizações possíveis:
 - Colocar o nome do algoritmo ao lado da barra;
 - Adicionar os valores numéricos nas barras;
 - Usar proporções em vez de números absolutos (com percentual = vitórias / total, etc).



- Podemos fazer uma análise conjunto com 3 ou mais algoritmos ao invés de calcular par a par?
 - Sim, podemos utilizar testes estatísticos mais robustos:
 - O teste de Friedman, que verifica se há diferença significativa entre três ou mais algoritmos.
 - Caso o Friedman seja significativo, aplicamos o teste de Nemenyi como pós-teste.
 - Visualizamos os resultados com gráfico de ranks médios e tabela de F1-score com média e desvio padrão.
 - Explicação rápida:
 - Friedman testa se pelo menos um algoritmo tem desempenho estatisticamente diferente.
 - Nemenyi compara todos os pares. Se a diferença de ranks for maior que o *critical difference* (CD), então há significância.

- Gráfico de Ranking Médio (*Critical Difference Diagram* ou *CD plot*):
 - Objetivo:
 - Exibir os rankings médios dos algoritmos em todos os datasets;
 - Ferramenta: Usa o teste de Friedman + teste de Nemenyi (ou outro pós-teste);
 - Exibe: Conjuntos de algoritmos não significativamente diferentes conectados por uma linha;
 - Ótimo para: Comparar todos os algoritmos de forma conjunta e identificar grupos de desempenho similar;
 - Ferramentas: Orange, scikit-posthocs, cd-diagram (Python).



Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. Journal of Machine learning research, 7(Jan), 1-30.

Statistical Comparisons of Classifiers over Multiple Data Sets

Janez Demšar

*Faculty of Computer and Information Science
Tržaška 25
Ljubljana, Slovenia*

JANEZ.DEMSAR@FRI.UNI-LJ.SI

Editor: Dale Schuurmans

Abstract

While methods for comparing two learning algorithms on a single data set have been scrutinized for quite some time already, the issue of statistical tests for comparisons of more algorithms on multiple data sets, which is even more essential to typical machine learning studies, has been all but ignored. This article reviews the current practice and then theoretically and empirically examines several suitable tests. Based on that, we recommend a set of simple, yet safe and robust non-parametric tests for statistical comparisons of classifiers: the Wilcoxon signed ranks test for comparison of two classifiers and the Friedman test with the corresponding post-hoc tests for comparison of more classifiers over multiple data sets. Results of the latter can also be neatly presented with the newly introduced CD (critical difference) diagrams.

Keywords: comparative studies, statistical methods, Wilcoxon signed ranks test, Friedman test, multiple comparisons tests

Outros cenários

- A **curva ROC** e a **AUC** são calculadas para uma **única classe positiva por vez**.
- Ou seja, **você define quem é a “classe positiva”**, e todo o cálculo de TPR e FPR gira em torno disso.
- Agora, se você tem **mais de duas classes** (classificação multiclasse), você pode sim **calcular curvas ROC para todas as classes**.
- Para isso, existem abordagens padronizadas:
 - Para problemas **binários**;
 - Para problemas **multiclasse**.

- Para problemas **binários**:
 - Você calcula **uma única curva ROC**, definindo **qual é a classe positiva**.
 - Exemplo:
 - Classes: $[0, 1]$;
 - Você escolhe 1 como positiva $\Rightarrow$ curva ROC será em relação à classe 1.
 - Se quiser ver a curva ROC em relação à classe 0, você pode:
 - Inverter os rótulos (tornar 0 a positiva e 1 a negativa);
 - Recalcular tudo.

- Para problemas **multiclasse**.
 - Existem duas abordagens principais:
 - 1. **One-vs-Rest (OvR)**:
 - Você trata **cada classe individualmente** como a classe positiva, e as outras como negativas;
 - Para cada classe, você obtém: Uma curva ROC e Uma AUC;
 - Por exemplo, com três classes [0, 1, 2], você obtém:
 - Curva ROC e AUC para classe 0 vs (1 e 2)
 - Curva ROC e AUC para classe 1 vs (0 e 2)
 - Curva ROC e AUC para classe 2 vs (0 e 1)
 - Você pode visualizar cada curva separadamente ou calcular uma média entre as AUCs (veja abaixo).
 - 2. **Média das AUCs** (macro e micro):
 - Depois de calcular as AUCs com OvR, você pode resumir tudo:
 - **AUC Macro (*macro-average*)**:
 - Faz a **média aritmética simples** das AUCs individuais de cada classe;
 - **Trata todas as classes com o mesmo peso**, independentemente do número de exemplos.
 - **AUC Micro (*micro-average*)**:
 - Junta todas as predições e **calcula uma única curva ROC**, somando os verdadeiros e falsos positivos de **todas as classes**;
 - Leva em conta o desequilíbrio de classes.

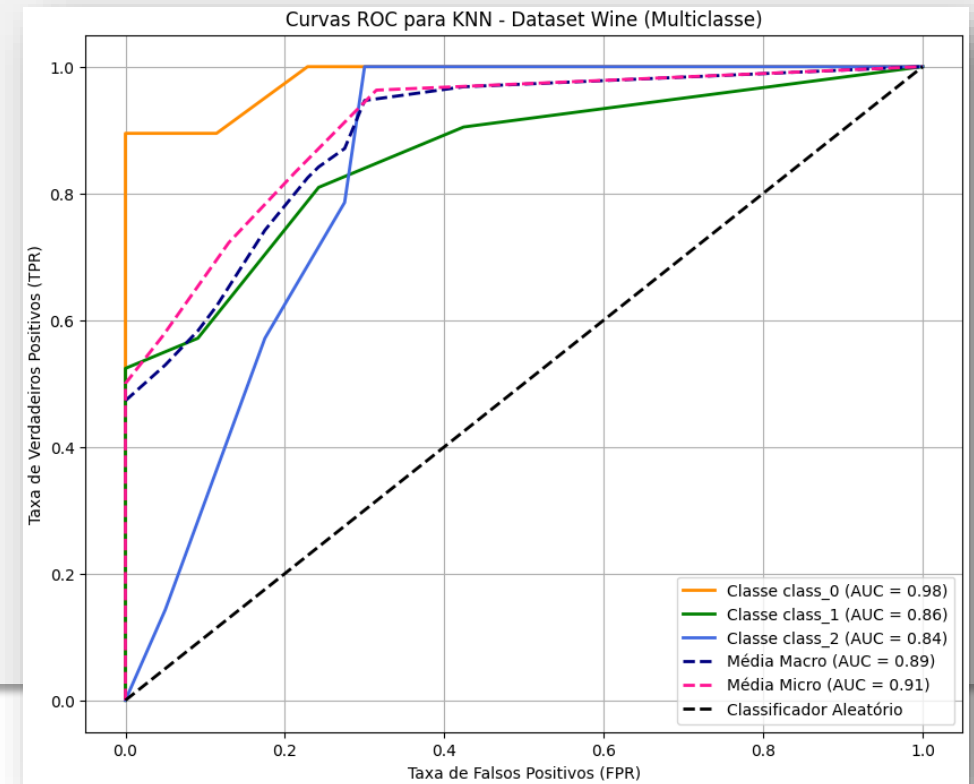
- Em resumo:

Cenário	AUC/ROC
Binário	Uma única curva ROC e AUC
Multiclasse (OvR)	Uma curva e AUC por classe
Multiclasse (macro)	Média das AUCs de cada classe
Multiclasse (micro)	AUC considerando todos os exemplos

Avaliação

Curva ROC

- Vamos fazer o seguinte com a base de dados Wine e o classificador k -NN:
 - Calcular e plotar:
 - Curvas ROC individuais;
 - AUC por classe;
 - AUC macro e micro.
 - AUCs individuais por classe:
 - Classe class_0: AUC = 0.9820
 - Classe class_1: AUC = 0.8586
 - Classe class_2: AUC = 0.8384
 - AUC Macro média: 0.8930
 - AUC Micro média: 0.9077
- Para comparar esse resultado com outros classificadores:
 - Utilizar e plotar a AUC Macro ou Micro;
 - Comparar classe a classe.



Questões importantes

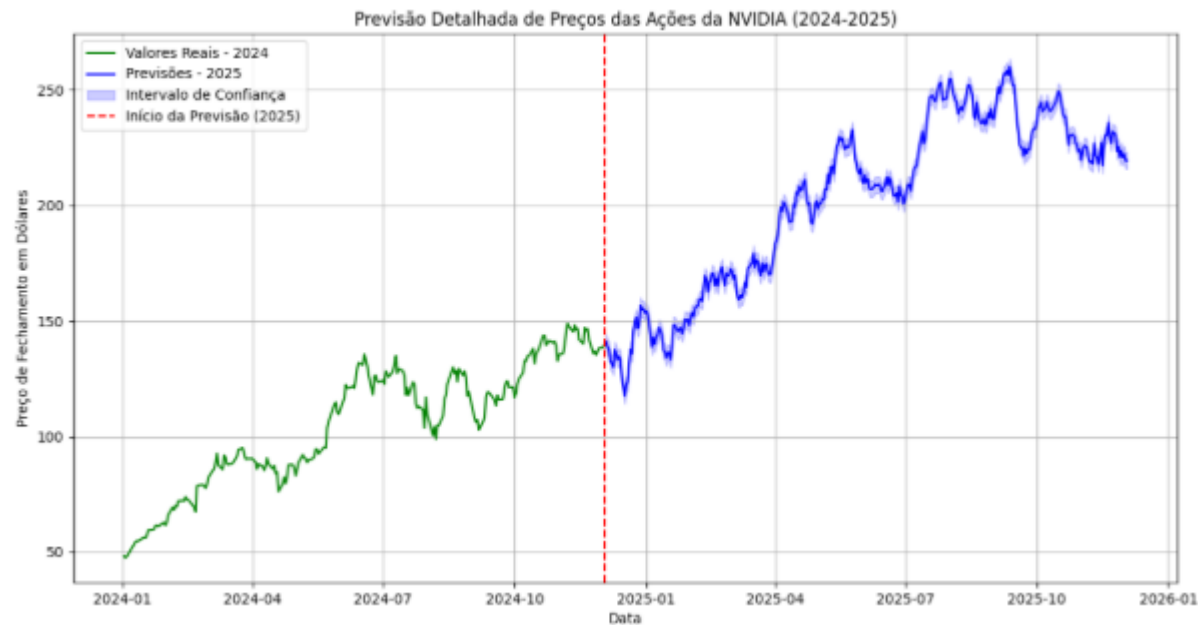
Como melhorar os modelos de classificação?

- Ao analisar a matriz de confusão, algumas estratégias podem ser empregadas para melhorar a performance do seu modelo:
 - **Rebalanceamento de Dados:** Se o modelo apresenta muitos falsos negativos ou falsos positivos, pode ser um indicativo de que as classes estão desbalanceadas. Reamostrar o conjunto de dados ou usar técnicas como SMOTE (*“Synthetic Minority Over-sampling Technique”*) pode ajudar.
 - **Ajuste de *Threshold*:** Dependendo do contexto do problema, alterar o limiar de decisão pode ser uma estratégia útil para equilibrar precisão e recall.
 - **Escolha do Algoritmo:** Alguns algoritmos são mais adequados para diferentes contextos. Modelos baseados em árvores de decisão, por exemplo, são ótimos para explicar suas previsões, enquanto redes neurais têm tendência a ser mais precisas, mas menos interpretáveis.

Questões importantes

Outros cenários

- Nem todos os cenários se encaixam nessas metodologias:
 - Previsão de mercado de ações.

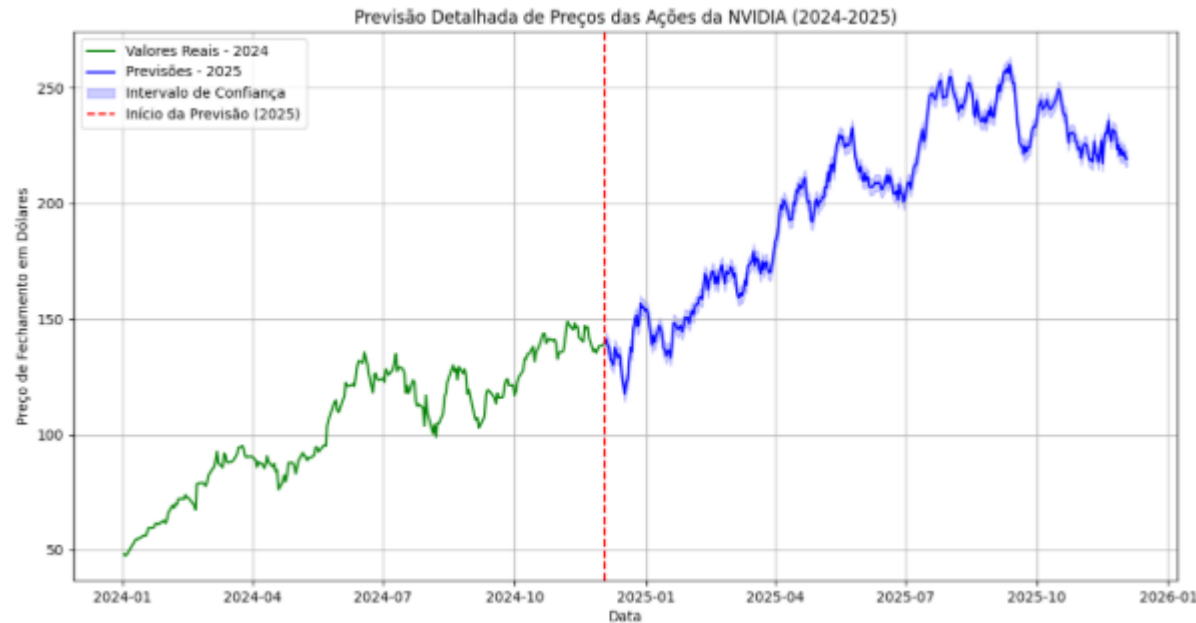


Questões importantes

Outros cenários

- Nem todos os cenários se encaixam nessas metodologias
 - Previsão de mercado de ações.

- Comparar algoritmos de **previsão de alta ou queda do mercado de ações com base apenas na próxima hora** geralmente **não é a forma mais eficaz ou informativa** de avaliar o desempenho de modelos preditivos.

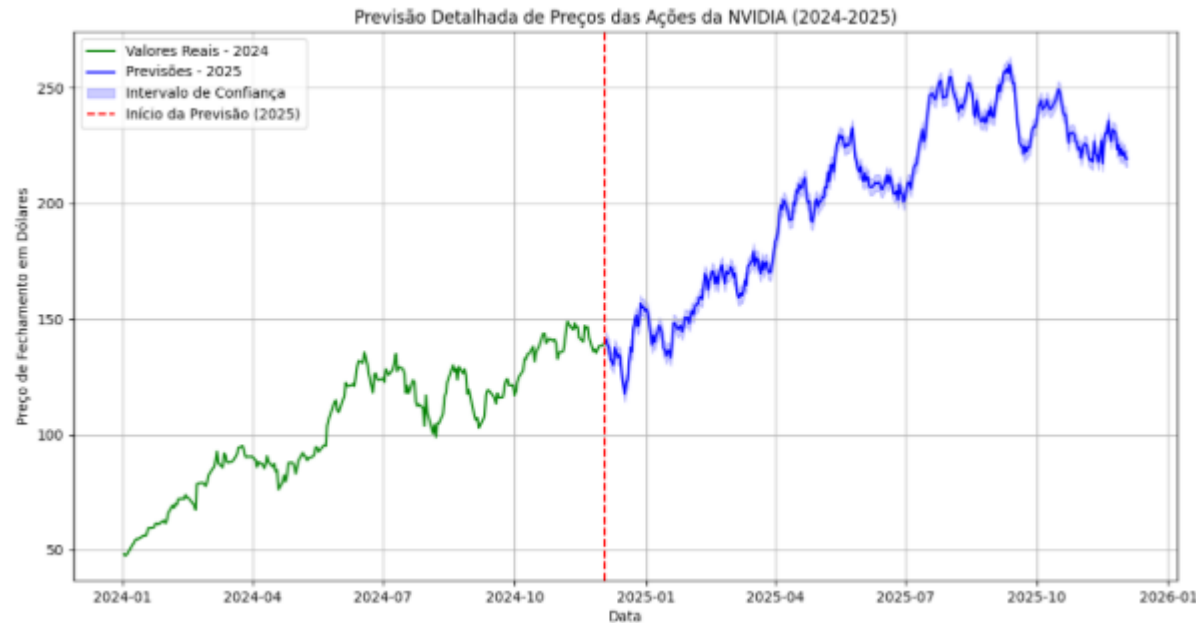


Questões importantes

Outros cenários

- Nem todos os cenários se encaixam nessas metodologias
 - Previsão de mercado de ações.

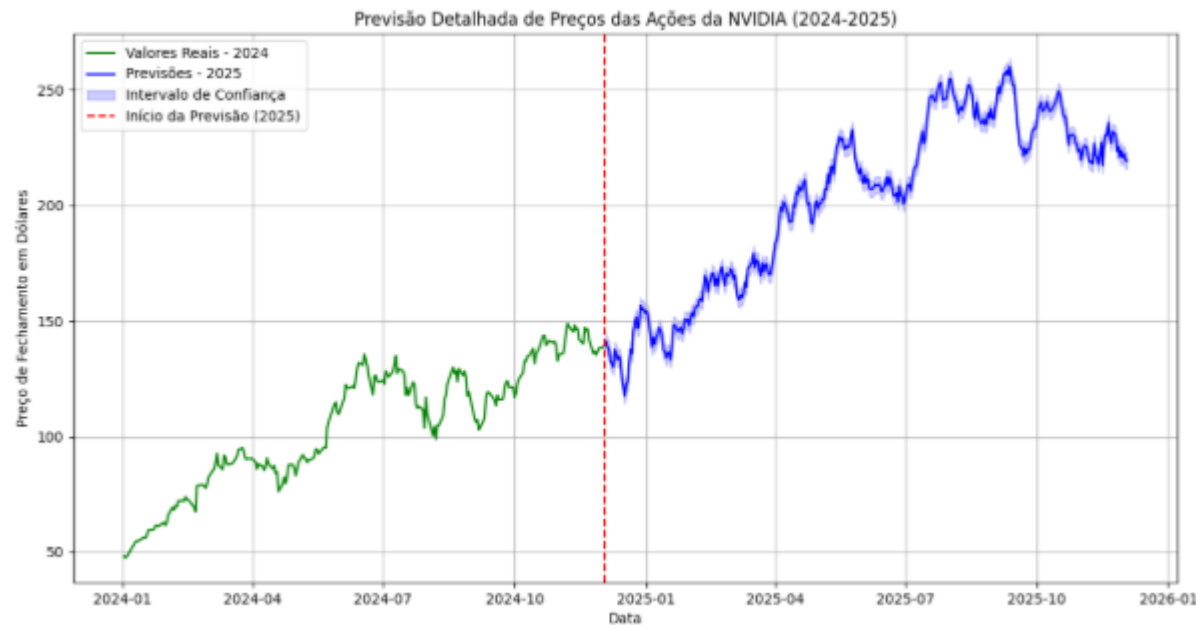
- Principal ponto: O famoso problema do **One Step Ahead**.
- In time series forecasting, a "one-step ahead" prediction refers to using a model to predict the next value in the sequence, given the current value and potentially past values.*



Questões importantes

Outros cenários

- Nem todos os cenários se encaixam nessas metodologias:
 - Previsão de mercado de ações.



- É fácil prever o próximo valor de uma série:
 - Basta eu pegar a moda/média dos três valores anteriores.

1. Curto prazo é extremamente ruidoso:

- O mercado de ações é altamente volátil e, especialmente em intervalos curtos (como uma hora), **o ruído é muito maior do que o sinal**;
- Por isso, **modelos simples** (como um preditor baseado em média ou até aleatoriedade com viés) podem obter **resultados parecidos com modelos sofisticados**, sem que isso signifique que os métodos são equivalentes em termos de capacidade preditiva.

2. Previsões de um único passo à frente (one-step-ahead) limitam a análise:

- Comparar algoritmos apenas na tarefa de **prever o próximo ponto** não mostra o verdadeiro potencial do modelo;
- É possível que um modelo seja apenas ligeiramente melhor que o aleatório no curto prazo, mas tenha **uma vantagem acumulativa clara em janelas maiores** (como ao longo de várias horas ou dias).

3. Previsão em janelas maiores permite capturar padrões reais:

- Ao usar janelas maiores (ex: previsão de tendência para as próximas **6h, 12h, 24h ou até 3 dias**), você:
 - Reduz a interferência do ruído de curto prazo;
 - Dá espaço para que o modelo capture **padrões mais estruturais e significativos**, como movimentos de tendência ou reversões.

4. Métricas de avaliação são mais confiáveis em janelas maiores:

- Com janelas maiores, o impacto de **erros aleatórios se dilui**, e é mais provável que **modelos superiores se destaquem** em métricas como **acurácia, F1-score, AUC-ROC, lucro simulado (*backtesting*)**, etc.

5. Modelos simples tendem a performar bem em horizontes curtos:

- Modelos baseados em regras simples (ex: média móvel, persistência, *random walk* com viés) têm desempenho razoável no curtíssimo prazo, especialmente em séries com **alta autocorrelação**. Isso **mascara a superioridade dos métodos mais avançados**.

- Sugestão prática:
 - Você pode avaliar os modelos com **diferentes horizontes de previsão**, como:
 - Previsão da direção (alta/queda) nas próximas:
 - 1 hora;
 - 6 horas;
 - 12 horas;
 - 24 horas;
 - 3 dias.
 - Além disso, experimente calcular o desempenho com uma **estratégia de *backtesting***, simulando operações de compra/venda com base nas previsões, para ver se os modelos realmente têm **valor prático**.

- O que é *Backtesting*?
 - **Backtesting** é o processo de testar **uma estratégia de investimento** ou **modelo de previsão com dados históricos**, como se estivéssemos aplicando essa estratégia no passado, para ver como ela teria se saído;
 - Em outras palavras:
 - “*Se eu tivesse seguido essa estratégia nos últimos meses/anos, quanto eu teria ganhado ou perdido?*”
- Por que o *backtesting* é importante?
 - Porque ele te permite:
 - Avaliar a **eficácia realista** de um modelo de previsão (mesmo que ele use aprendizado de máquina);
 - Considerar **regras práticas de operação**, como:
 - Custos de transação;
 - *Stop loss* e *take profit*;
 - Tempo de espera até o resultado do trade;
 - Comparar com benchmarks simples (como comprar e manter, ou regras heurísticas).

- Diferença entre acurácia e performance real:
 - Um classificador pode acertar 60% das vezes se o preço vai subir ou cair, mas **isso não garante lucro**.
 - Exemplo:
 - Acertou que ia subir → mas subiu só 0.1%, e você pagou 0.2% de taxa = prejuízo;
 - Errou que ia cair → mas o ativo subiu 5% e você não aproveitou = perda de oportunidade.
 - Por isso, no *backtesting* você calcula o **lucro/prejuízo de verdade**, e não apenas a acurácia do modelo.
- Como fazer *backtesting* (resumo):
 - **Você treina um modelo** com dados históricos até certo ponto;
 - Com os dados do “futuro”, você:
 - Aplica as previsões do modelo;
 - Simula compras e vendas como se fosse um *trader* real;
 - Calcula o **retorno acumulado** da estratégia.
 - Repete isso ao longo do tempo, **sem usar informações futuras**, como num ambiente real.

- Portanto:
 - Muitos cenários necessitam de estratégias necessitam de metodologias experimentais específicas;
- Mas para a grande maioria dos cenários, vocês podem utilizar as metodologias passadas nessa aula.

Análise paramétrica

- O termo mais preciso seria mesmo “análise de hiperparâmetros”. Mas, na prática, a comunidade usa “análise paramétrica” por convenção histórica e por simplificação terminológica.
- Hoje há uma tendência de maior precisão terminológica:
 - **Hyperparameter tuning:** termo mais correto;
 - **Parameter tuning:** termo mais genérico.
- Como selecionar os melhores valores dos atributos para um determinado algoritmo?
 - Rodar o algoritmo para diferentes valores de atributos e verificar o seu desempenho;
 - Selecionar o melhor valor do hiperparâmetro.
- Pode-se utilizar técnicas mais robustas como GridSearch, RandomSearch ou AutoML.

Próxima aula

- Redes Neurais

Obrigado

Dúvidas

Email: alanvalejo@ufscar.br